LEAN 4 · FORMAL THEOREM PROVING

Proving Is Programming

Lean 4 from First Principles

Lean 4.33.0

SOURCES AND LICENSE

Attribution: a compilation built on one open-source book and other cited works

Primary source: the structure, concepts and Lean code examples are adapted from

> Jeremy Avigad, Leonardo de Moura, Soonho Kong and Sebastian Ullrich, with contributions from the Lean Community. *Theorem Proving in Lean 4* (edition for Lean 4.33.0).
>
> `https://lean-lang.org/theorem_proving_in_lean4/`
> `https://github.com/leanprover/theorem_proving_in_lean4`

Licensed under the **Apache License, Version 2.0** — http://www.apache.org/licenses/LICENSE-2.0. This deck is distributed under the same License; its full text is reproduced in the Appendix at the end.

Notice of changes: this is a modified work

- Code examples copied from the book's source files (book/TPiL/*.lean), some re-wrapped to fit the slides
- Examples labelled "(illustration)" are original, written for this deck and checked with Lean 4.33.0
- Explanations, background (Part 0), summaries, questions and error walkthroughs are newly written
- Other works ([3]–[22]) are cited for background, history and further reading, marked [n] in footers; their text is not reproduced

Compiled and adapted by hohieuai, 2026 · github.com/hohieuai/awesome-formal-verification

Not affiliated with or endorsed by the authors or the Lean project.

OVERVIEW

Source and scope of this deck

Theorem Proving in Lean 4

- **Authors:** Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, and the Lean Community
- **Version:** written for Lean 4 (specifically 4.33.0); the first edition targeted Lean 2, with a separate edition for Lean 3
- **Open:** an open-access project on GitHub — leanprover/theorem_proving_in_lean4
- **Logical foundation:** a version of the Calculus of Constructions with inductive types

What we cover

- Part 0: background for newcomers
- Dependent types and “propositions as types”
- Term-style and tactic-style proofs
- Inductive types, recursion, structures, type classes
- Lean’s axioms and their computational consequences

AGENDA

Contents: Part 0 and 12 chapters in 4 parts

PART 0 Background logic · sets, types, functions · induction · functional programming · Lean architecture and setup · terminology

PART I

Foundations

1 Introduction
What Lean is and why

2 Dependent Type Theory
Types that depend on values

3 Propositions and Proofs
Propositions as types

4 Quantifiers and Equality
∀, ∃ and =

PART II

Proving in practice

5 Tactics
Proof tactics

6 Interacting with Lean
Commands and notation

PART III

Data and recursion

7 Inductive Types
Defining data

8 Induction and Recursion
Pattern matching, termination

9 Structures and Records
Single-constructor types

PART IV

Advanced mechanisms

10 Type Classes
Instance resolution

11 The Conversion Tactic Mode
The conv mode

12 Axioms and Computation
propext, choice, quotients

PART 0 · PREPARATION

0

Background

Background for newcomers

SECTIONS

0.1 What a proof is, and why formalize it

0.2 Propositional logic and natural deduction

0.3 Predicate logic: ∀, ∃, bound variables

0.4 Sets and types; functions, currying, λ

0.5 Induction and recursive definitions

0.6 Constructive vs. classical; functional programming

0.7 Lean's architecture, setup, Infoview

0.8 Terminology and a roadmap of the 12 chapters

PART 0 · §0.1 BACKGROUND

What is a proof — and why do we need formal proofs?

A **proof** is a chain of reasoning steps, each following an accepted rule, leading from hypotheses and axioms to a conclusion.

On paper. Written for a reader: “obvious” steps are skipped, relying on the reader’s intuition — like a recipe written for a chef.

Formal. Written for a machine: no step may be skipped, and every step is machine-checked — like a program that must compile.

> *A proof assistant such as Lean does not come up with ideas for us; it guarantees that no step is wrong or missing.*

1994 · The Pentium division bug (FDIV). A few missing entries in the floating-point divider’s lookup table made some divisions return wrong results. Afterwards, formal verification of arithmetic hardware became far more widespread.

CompCert. A C compiler with a proof (in Coq) that the machine code preserves the semantics of the source. Random testing (Csmith, 2011) found many bugs in popular compilers, but no miscompilation bugs in CompCert’s verified part.

Mathematics. The four colour theorem was verified in Coq (2005); the Kepler conjecture via the Flyspeck project (2014); P. Scholze’s “Liquid Tensor Experiment” was formalized in Lean (2022).

Propositions and the five logical connectives

A **proposition** is a statement that can be true or false: “it is raining”, “2 + 2 = 4”, “7 is even”. Questions and commands are not propositions. Connectives combine small propositions into larger ones. Below, p = “it is raining”, q = “the road is wet”.

Symbol	Input	Read as	Everyday sentence	True when?
`¬p`	`\not`	not p	“it is not raining”	p is false
`p ∧ q`	`\and`	p and q	“it is raining and the road is wet”	both p and q are true
`p ∨ q`	`\or`	p or q	“it is raining or the road is wet”	at least one side is true (possibly both)
`p → q`	`\to`	if p then q	“if it rains, the road is wet”	never p true while q is false
`p ↔ q`	`\iff`	p if and only if q	“rain if and only if the road is wet”	p and q are both true or both false

> *Two constant propositions: True (always true) and False (always false). In mathematics “or” is inclusive: `p ∨ q` holds when both sides are true — unlike the exclusive “either… or…” of everyday speech.*

Reading formulas: precedence, truth tables and proofs

Just as “multiplication before addition”, connectives have precedence: ¬ > ∧ > ∨ > → > ↔. The arrow → associates to the right.

How Lean parses them (checked with #check)

```
¬p ∧ q          means (¬p) ∧ q
p ∧ q ∨ r       means (p ∧ q) ∨ r
p ∨ q → r       means (p ∨ q) → r
p → q → r       means p → (q → r)

¬p ∧ q → r ∨ s
    means ((¬p) ∧ q) → (r ∨ s)
```

(p → q) → r **needs** parentheses — a different proposition.

The **truth table** of p → q: false in exactly one row.

p	q	p → q	Promise: rain → umbrella
true	true	true	rain, umbrella taken
true	false	**false**	rain, but no umbrella
false	true	true	no rain, umbrella anyway
false	false	true	no rain, no umbrella

> *A truth table tries every case (2^n rows). “For every natural number n” means infinitely many cases — they cannot be listed. So Lean does not use truth tables: each proposition needs a **proof**.*

Natural deduction: introduction and elimination rules

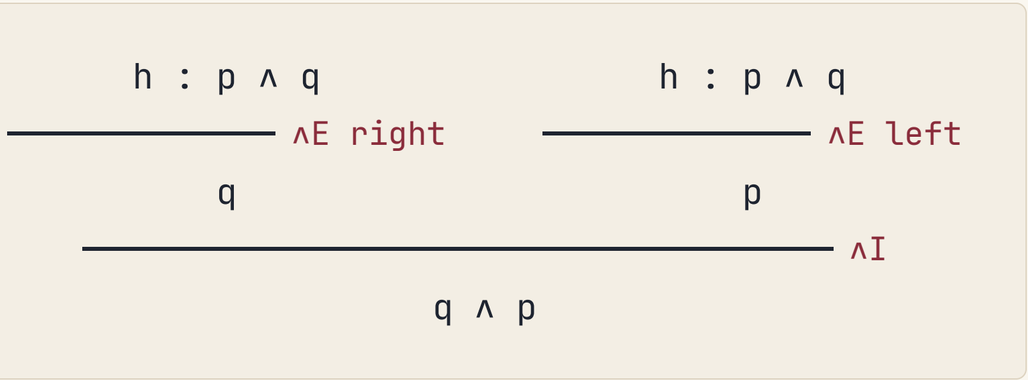
Natural deduction (G. Gentzen, 1930s): **introduction** rules say how to *prove* a formula; **elimination** rules say how to *use* a hypothesis we already have.

Conn.	Introduction — to prove…	Elimination — given…, how to use it?
∧	Have p and have q ⇒ have p ∧ q. *"I have a ticket and a passport."*	From p ∧ q, extract p alone, or q alone.
∨	Prove p (or q) ⇒ have p ∨ q.	Have p ∨ q; p implies r and q implies r ⇒ have r. *"Pizza or pasta; both places are on Main Street ⇒ tonight I am on Main Street."*
→	**Temporarily assume** p, derive q ⇒ have p → q.	Have p → q and have p ⇒ have q (modus ponens).
¬	Temporarily assume p, reach a contradiction ⇒ have ¬p.	Have p and ¬p ⇒ contradiction (False).
False	No rule — a falsehood cannot be proved.	From False, anything follows (ex falso quodlibet).

> *In Chapter 3, each rule above becomes a Lean function: And.intro, And.left, Or.elim, False.elim…*

A proof tree: from p ∧ q derive q ∧ p

Read **top to bottom**: above the bar are the premises, below it the conclusion, to its right the rule name. The leaves are hypotheses.



In words: “From h extract q; from h extract p; combine them to get q ∧ p.”

The same proof, written as a Lean term (Chapter 3)

```
variable (p q : Prop)

example (h : p ∧ q) : q ∧ p :=
  And.intro (And.right h) (And.left h)
```

Tree node	Part of the term	Type
∧E right	`And.right h`	`q`
∧E left	`And.left h`	`p`
∧I (root)	`And.intro … …`	`q ∧ p`

> *Proof tree = expression; checking the tree = type checking. This is the seed of the Curry–Howard correspondence.*

Predicates and the two quantifiers ∀ (for all), ∃ (exists)

A **predicate** is a proposition with a "blank": Even(x) — "x is even". Filling in x = 4 gives a true proposition, x = 7 a false one. In Lean: a function `P : Nat → Prop`.

Formula	Prove it by	Use it as a hypothesis by
∀ x, P x	P x for an **arbitrary** x	applying it to some x
∃ x, P x	exhibiting **one** specific x (a witness) and proving P x	naming that x and gaining the hypothesis P x

Lean reads two propositions (illustration)

```
#check ∀ n : Nat, n + 0 = n
-- ∀ (n : Nat), n + 0 = n : Prop
#check ∃ n : Nat, n > 3
-- ∃ n, n > 3 : Prop
```

Quantifier order matters

∀ person, ∃ mother, … — *everyone has a mother* (true).

∃ mother, ∀ person, … — *one mother shared by everyone* (false).

Negation flips the quantifier

"Not all swans are white" ⇔ "some swan is not white".

`¬(∀ x, P x) ↔ ∃ x, ¬P x`

The ⇐ direction is direct; the ⇒ direction needs classical logic.

> *In Chapter 4: ∀ is a dependent function type, ∃ is a pair (witness, proof).*

Free and bound variables

In `∀ x, x + y = y + x`, the variable **x** is **bound** by ∀; the variable **y** is **free** — the meaning of the formula depends on y.

Expression	Bound	Free
`∀ x, x + y = y + x`	x	y
`∃ n, n > m`	n	m
∑ i for i from 1 to n	i	n
`fun x ⇒ x + y`	x	y

Analogy: a bound variable is like a function's **parameter** in programming; a free variable is like a **global variable** the function refers to.

Renaming bound variables preserves meaning

∀ x, P x and ∀ z, P z are one proposition. Terms differing only in bound names are **α-equivalent**; Lean identifies them. Chapter 2: `fun (b : β) (x : α) ⇒ b` is exactly `fun (u : β) (z : α) ⇒ u`.

Pitfall: variable "capture"

Naively substituting x for y in ∀ x, x + y = y + x gives ∀ x, x + x = x + x — the meaning changes because the new x is "captured" by ∀. One must rename first: ∀ z, z + x = x + z. Lean handles this automatically.

Sets or types? Why Lean chooses type theory

Criterion	Set theory (ZFC)	Type theory (Lean)
Basic objects	Everything is a set — including the number 3 and functions.	Everything is a **term**; each term has **exactly one type**.
“Membership”	$x \in A$ is a proposition, true or false; $3 \in \mathbb{N}$ and $3 \in \mathbb{R}$ hold at once.	$t : T$ is a judgment checked by the machine, not a proposition. (3 : Nat) and (3 : Int) are different terms.
“Meaningless” statements	“$2 \in 3$” is syntactically valid (and true under von Neumann’s encoding of numbers).	2 + true is rejected at type checking: there is no addition of a Nat and a Bool.
Logic	Outside: first-order predicate logic, with sets built on top.	Inside: propositions are types, proofs are terms (Chapter 3).

> *Analogy: types are like **units** in physics — adding metres to seconds is blocked immediately, before anything is computed.*

> *Benefits for the machine: type checking is algorithmic; many errors are caught early; proofs and programs share one language.*

Functions from scratch: domain, codomain, application, λ

A function `f : A → B` assigns to **each** element of A (the **domain**) **exactly one** element of B (the **codomain**).

- "add 1": Nat → Nat
- "string length": String → Nat
- "is it zero?": Nat → Bool

Function application: mathematics writes f(x), Lean writes `f x` — a space means application. Nesting needs parentheses: `f (g x)`.

λ-notation: functions need no name. Maths: x ↦ x + 1; λ-calculus: λx. x + 1; Lean: `fun x ⇒ x + 1` (or λ x => …).

Named and anonymous functions (illustration)

```
def addOne (n : Nat) : Nat := n + 1

#eval addOne 41      -- 42
#check addOne      -- addOne (n : Nat) : Nat

#check fun (n : Nat) ⇒ n + 1
-- fun n ⇒ n + 1 : Nat → Nat
#eval (fun (n : Nat) ⇒ n + 1) 41
-- 42
```

> ***β-reduction:*** *applying a function = substituting the argument for the parameter. (fun n => n + 1) 41 ⟶ 41 + 1 ⟶ 42.*

Functions returning functions (currying) and higher-order functions

`Nat → Nat → Nat` means `Nat → (Nat → Nat)`: a "two-argument" function really takes **one** number and returns **a function** awaiting the second.

Analogy: a coffee machine — choose the beans and you get a machine "loaded with beans"; choose a cup size and you get your coffee.

- **Partial application**: add 3 is the function "add 3".
- Application associates left: add 3 4 = (add 3) 4.
- **Higher-order functions**: take a function (twice) or return one (add).
- In (Nat → Nat) → Nat → Nat, the left parentheses are required.

Partial application and higher-order functions (illustration)

```
def add (x y : Nat) : Nat := x + y
#check add 3      -- add 3 : Nat → Nat
#eval add 3 4      -- 7

def twice (f : Nat → Nat) (x : Nat) : Nat :=
  f (f x)
#check (twice)
-- twice : (Nat → Nat) → Nat → Nat
#eval twice (add 3) 10          -- 16
#eval twice (fun n ⟹ n * 2) 5    -- 20
```

> *Proofs use exactly this mechanism: p → q → r is a "function" taking a proof of p and returning a function that turns a proof of q into a proof of r.*

Natural numbers from two bricks: zero and succ

Following Peano, every natural number is built from **zero** (0) and **succ** (“the next one”): 3 = succ (succ (succ zero)). There is no other way to make one.

A copy of Nat (illustration)

```
inductive MyNat where
  | zero : MyNat
  | succ : MyNat → MyNat
```

Lean’s built-in Nat is defined exactly this way, with two constructors Nat.zero and Nat.succ; the numerals 0, 1, 2… are just convenient notation.

```
#eval Nat.succ 2    -- 3
```

Recursive definition: give the result at zero, and the result at succ n in terms of the result at n. Since every number is built by finitely many succ steps, such a definition always terminates.

Doubling by recursion (illustration)

```
def double : Nat → Nat
  | 0     ⇒ 0
  | n + 1 ⇒ double n + 2

#eval double 5    -- 10
```

> *By hand: double 2 = double 1 + 2 = (double 0 + 2) + 2 = 0 + 2 + 2 = 4. The pattern n + 1 matches succ n.*

Mathematical induction: the domino effect

To prove P n for **every** natural number n, two things suffice:

Base. Prove P 0 — the first domino falls.

Step. Assume P k (induction hypothesis), prove P (k + 1) — each domino topples the next.

Example: 0 + n = n. Why is it not obvious? Lean's addition recurses on the *second* argument: n + 0 = n and n + (m + 1) = (n + m) + 1 hold by definition, but 0 + n does not reduce when n is unknown.

> *Induction and recursion are two faces of one principle: the recursor Nat.rec (Chapter 7).*

A proof with the induction tactic (illustration)

```
theorem zero_add' (n : Nat) : 0 + n = n := by
  induction n with
  | zero ⇒ rfl
  | succ k ih ⇒ rw [Nat.add_succ, ih]
```

Base case

```
case zero
⊢ 0 + 0 = 0
```

Step case — with the induction hypothesis ih

```
case succ
k : Nat
ih : 0 + k = k
⊢ 0 + (k + 1) = k + 1
```

Constructive and classical: two readings of “exists”

	Classical	Constructive
Motto	“it cannot fail to exist”	“show it to me”
p ∨ ¬p	always true (excluded middle)	only if we can decide which side
∃ x, P x	enough that “none” leads to a contradiction	must exhibit a specific x
p ∨ q	no need to know which side	must know which side holds

In Lean. The core logic is constructive. Excluded middle is available as Classical.em, derived from the axioms, notably Classical.choice (Chapters 3 and 12). Classical logic is perfectly valid — the trade-off is computability.

A classic example

Claim. There are irrational numbers a, b such that a^b is rational.

Proof. Consider √2^√2. If it is rational, take a = b = √2. If it is irrational, take a = √2^√2, b = √2; then $a^b = \sqrt{2}^2 = 2$.

The proof is **valid** classically, but does not tell us which case holds — so it exhibits no specific a.

> *In practice: constructive definitions run with #eval; data definitions relying on Classical.choice must be marked noncomputable.*

Functional programming: four ideas before reading Lean code

Immutability. let x := 5 names a value; it is not an assignment, and x never changes. let x := x + 1 creates a new name shadowing the old one.

Everything is an expression. if … then … else … returns a value, so an else branch is always needed. There are no "statements".

Pattern matching. Define by the "shape" of data: the empty list [] or a head :: the rest.

Pure, recursive functions. Same input, same output; loops become recursion. So we reason about code as about equations.

> *Lean is both a functional programming language and a proof language: def, fun, match serve both.*

Three small definitions (illustration)

```
def demo : Nat :=
  let x := 5
  let y := x + 1
  x * y
#eval demo    -- 30

def describe (n : Nat) : String :=
  if n == 0 then "zero" else "positive"

def len : List Nat → Nat
  | []      ⇒ 0
  | _ :: xs ⇒ len xs + 1
#eval len [3, 1, 4]    -- 3
```

Lean's architecture: from source code to the kernel

Source code
Your def, theorem, notation, by …

→

Parser
Reads syntax; expands notation and macros

→

Elaborator
Fills implicit args, solves metavariables, finds instances, runs tactics ⇒ full term

→

Kernel
Type-checks the full term; small, simple, trusted

→

Environment
Stores accepted definitions

Why does a small kernel matter? The elaborator and tactics are complex and may have bugs — but if they produce a wrong term, the kernel rejects it. To trust a theorem, you only need to trust the small kernel (and the axioms).

Proof objects. The end result of every proof, including tactic proofs, is a concrete term. It can be printed with #print and rechecked independently by another program.

What does "type-checking a proof" mean?

Write it with tactics, then inspect the term Lean built (illustration)

```
theorem and_swap (p q : Prop) (h : p ∧ q) :
    q ∧ p := by
  constructor
  · exact h.2
  · exact h.1

#print and_swap
-- theorem and_swap : ∀ (p q : Prop),
--     p ∧ q → q ∧ p :=
-- fun p q h ⟹ ⟨h.right, h.left⟩

#print axioms and_swap
-- 'and_swap' does not depend on any axioms
```

1. **The statement is a type:** ∀ (p q : Prop), p ∧ q → q ∧ p.
2. The **elaborator** runs the tactics, yielding fun p q h => ⟨h.right, h.left⟩.
3. The **kernel** checks each node: h : p ∧ q, so h.right : q and h.left : p; hence ⟨h.right, h.left⟩ : q ∧ p; the whole function has the stated type.
4. Match ⇒ the theorem enters the environment. Mismatch ⇒ an error.

> ***Finding** a proof takes creativity; **checking** one is a mechanical procedure, like checking units in a physics equation.*

Installing Lean 4 and creating a first project

1. Install **VS Code**, then the **Lean 4** extension (ID leanprover.lean4) from the Marketplace.
2. Follow the extension's **Setup Guide**: File › New Text File, click the ∀ icon at the top right › Documentation › Docs: Show Setup Guide. The extension installs elan for you.
3. Create a project with lake and open its folder in VS Code.

Tool	Role
elan	Manages Lean versions (like rustup)
lake	Creates projects, builds, manages libraries
lean-toolchain	File recording the project's Lean version

Manual install on Linux/macOS and a project using Lean 4.33.0

```
# install elan (choose 1 = default)
curl https://elan.lean-lang.org/elan-init.sh -sSf | sh
source $HOME/.elan/env

# new project, pinned to version 4.33.0
lake +leanprover/lean4:v4.33.0 new hello
cd hello
cat lean-toolchain
# leanprover/lean4:v4.33.0
lake build
```

The project contains lakefile.toml, lean-toolchain, Main.lean, Hello.lean, Hello/Basic.lean. On Windows, use the elan-init.ps1 script in PowerShell.

Typing Unicode symbols: a backslash, then an abbreviation

Type	Get	Used for
`\and`	∧	and
`\or`	∨	or
`\not`	¬	negation
`\to`	→	implies, functions
`\iff`	↔	equivalence
`\forall`	∀	for all
`\ex`	∃	exists

Type	Get	Used for
`\la, \fun`	λ	anonymous function
`\<`	⟨	open a pair ⟨a, b⟩
`\>`	⟩	close a pair
`\x`	×	product type
`\circ`	∘	composition
`\l`	←	rw [← h]
`\.`	·	focus on one goal

Type	Get	Used for
`\a \b \g`	α β γ	type variables
`\ne`	≠	not equal
`\le`	≤	less or equal
`\ge`	≥	greater or equal
`\_1`	₁	names h_1, h_2
`\N`	ℕ	another name for Nat
`\⊢`	⊢	goal symbol

> *The symbol is replaced when you type a space or Tab. Newcomer trap: \l gives ←,* ***not*** *λ; Lean 4 code usually writes fun instead of λ. → can also be typed as \r or \->. ℕ is just notation defined by Mathlib.*

Reading the Infoview: hypotheses, goals and the ⊢ symbol

Editor — cursor at the end of the constructor line (illustration)

```
example (p q : Prop) (hp : p) (hq : q) :
    p ∧ q := by
  constructor   -- cursor here
  · exact hp
  · exact hq
```

Open the Infoview with “Lean 4: Infoview: Toggle Infoview” (Ctrl/Cmd+Shift+Enter); it usually opens beside a .lean file.

Line	Meaning
`case left`	name of the goal (branch)
`hp : p`	hypothesis: **name : type** — what we have
`⊢ p`	goal: what remains to be proved

Infoview · Tactic state · 2 goals

```
case left
p q : Prop
hp : p
hq : q
⊢ p

case right
p q : Prop
hp : p
hq : q
⊢ q
```

> *The Infoview shows the state **at the cursor**: move the cursor line by line to watch the proof progress.*

The first four messages you will meet — and how to read them

Three kinds of message: **error** (red underline), **warning** (yellow underline), **information** (results of #check, #eval). Hover over the underline to read it.

Code (illustration)	Lean 4.33.0 reports	What does it mean?
`def x : Nat := "Not a number"`	`Type mismatch "Not a number" has type String but is expected to have type Nat`	The value does not have the declared type. (Lean prints this message over several lines.)
`#check bar`	`Unknown identifier `bar``	The name is undefined, misspelt, or its namespace is not open.
`example (p q : Prop) (hp : p) : p ∧ q := by constructor exact hp`	`unsolved goals case right p q : Prop hp : p ⊢ q`	The tactics ran out but goals remain. Lean prints what is left: here nothing proves q.
`theorem foo : 1 = 1 := sorry`	`declaration uses `sorry``	Warning: the proof is a placeholder; accepted, but incomplete.

Glossary: terms, types and proofs

Term	Short explanation	Example
term	Any Lean expression: a number, function, type, proof	`fun x ⟹ x + 1`
type	A label classifying terms; each term has exactly one type	`Nat, Nat → Bool`
Prop	The type of propositions; each p : Prop is itself a type	`2 + 2 = 4 : Prop`
Sort	The family of all types: Sort 0 is Prop, Sort 1 is Type	`Sort u`
universe	Stacked levels of types, avoiding Russell-style paradoxes	`Type : Type 1`
proof	A term whose type is a proposition (Curry–Howard)	`⟨h.2, h.1⟩ : q ∧ p`
goal	What remains to be proved, after the ⊢ symbol	`⊢ q ∧ p`
hypothesis	An assumption in the context, of the form name : type	`h : p ∧ q`
tactic	A command transforming the goal; ultimately yields a term	`intro, exact, rw`
elaborator	Translates user code into a full term for the kernel	fills {α}, runs by …

Glossary: kernel, inductive types and type classes

Term	Short explanation	Example
kernel	The small, trusted part that type-checks the final term	accepting a theorem
metavariable	An unknown “hole” the elaborator must fill, shown as ?m	`?m.2`
definitional equality	Equal by definition: the machine sees it, rfl suffices	`2 + 2 = 4 := rfl`
inductive type	A type given by a list of constructors	`inductive Nat`
constructor	The only way to build elements of an inductive type	`Nat.zero, Nat.succ`
recursor	The recursion/induction principle Lean generates	`Nat.rec`
motive	The type (or proposition) the recursor is building	`motive : Nat → Sort u`
type class	An interface; Lean finds the right implementation for a type	`Add α, ToString α`
instance	A concrete implementation of a type class for a type	`instance : Add Point`
axiom	A statement accepted without proof	`propext, Classical.choice`

Roadmap: how Part 0 leads into the 12 chapters

Part 0 Background → **Ch. 1–2** Lean, type theory → **Ch. 3–4** Logic, quantifiers → **Ch. 5–6** Tactics, workflow → **Ch. 7–9** Induction, structures → **Ch. 10–12** Type classes, conv, axioms

Background	Used in	What it becomes in Lean
Formal proof, proof assistants	Ch. 1	Context: formal verification, Lean's place
Functions, currying, λ; sets and types	Ch. 2	Function types, fun, dependent types, universes
Propositional logic, natural deduction	Ch. 3, 5	Each intro/elim rule is a function or a tactic
Predicate logic, bound variables	Ch. 4	∀ is a dependent function type, ∃ a pair; equality, calc
Lean's architecture, Infoview, errors	Ch. 5, 6, 11	Reading goals while writing tactics; namespaces, notation; conv
Induction, recursion, functional programming	Ch. 7, 8, 9	inductive, recursors and motives, match, structure
Higher-order functions, interfaces	Ch. 10	Type classes and automatically found instances
Constructive vs. classical	Ch. 3, 12	Classical.em, the axiom of choice, noncomputable, quotients

PART I · CHAPTER

1

Introduction

Formal verification and where Lean fits

SECTIONS

1.1 Computers and Theorem Proving

1.2 About Lean

1.3 About this Book

1.4 Acknowledgments

CHAPTER 1 · §1.1

Formal verification

Definition

The use of logical and computational methods to establish claims expressed in precise mathematical terms. Targets:

- Ordinary mathematical theorems
- Hardware and software meeting a specification
- Network protocols
- Mechanical and hybrid systems

A blurred boundary

Systems → mathematics. Describe a system mathematically and checking its correctness becomes theorem proving.

Mathematics → computation. Proving a theorem may depend on a long computation — and that computation must be verified too.

> *Proof is the "gold standard": 20th-century logic showed that most ordinary proof methods reduce to a small set of axioms and rules of a foundational system.*

The checker as a referee who never accepts “obvious”

A **proof on paper** is written for a reader: the “obvious”, “similarly” and “clearly” gaps are left for the reader to fill.

A **proof for a machine** is like a refereed chess game: each move must follow a published rule. The referee need not understand the plan, only check each move.

A familiar example: a compiler rejects a call with an argument of the wrong type. Lean does the same — for proofs too.

> *The cost: more writing. The gain: once the machine accepts, no step has been skipped.*

Claim: “if p and q both hold, then q and p both hold”

Step	Content	Rule used
Paper	“Obvious.”	(not stated)
1	Assume p and q	temporary hypothesis
2	From “p and q” extract p	left part of “and”
3	From “p and q” extract q	right part of “and”
4	Combine q and p into “q and p”	build “and” from parts
5	Conclude the implication	discharge the hypothesis

These five steps are exactly the and_commutative example in §1.3, written in Lean.

Two roles for computers: finding and checking proofs

Criterion	Automated theorem proving	Interactive theorem proving
Goal	Find a proof	Check every reasoning step, down to the axioms and basic rules
Typical tools	Resolution, tableaux, SAT solvers, decision procedures (linear arithmetic), SMT solvers, computer algebra systems	Systems requiring every claim to have a proof in a suitable axiomatic foundation
Priority	Power and efficiency	Reliability; support for deep, complex proofs
Limitations	Often no soundness guarantee; may have bugs, results hard to confirm	Requires more input from the user
Output	A computed result — a computation can itself count as a proof	A complete “proof object” that other systems can check independently

CHAPTER 1 · §1.1 · BACKGROUND

A minimal vocabulary before we start

Term	Everyday meaning	In Lean
Proposition	A statement that is true or false: “2 + 2 = 4”, “there are infinitely many primes”	`p : Prop`
Proof	An argument from known facts to the proposition, by valid rules	a term of type p
Axiom	Something accepted without proof	`axiom`
Inference rule	A “legal move”: from p and “p implies q”, get q	function application
Theorem	A proved proposition, named for reuse	`theorem`
Type	The “kind” of an object: natural number, truth value, list of numbers	`Nat, Bool, List Nat`
Type checking	Confirming each object is used according to its kind	type checking

> *The central idea (Chapter 3): proofs are also typed objects — the type of a proof is the proposition it proves. So “checking a proof” becomes “checking a type”.*

CHAPTER 1 · §1.1–1.2

Lean: bridging automated and interactive proving

Goal: place automated tools in a framework that supports user interaction and the construction of fully axiomatic proofs — for mathematics and complex systems alike.

A programming language

Lean's logic has a computational interpretation: write programs with precise semantics and reason about what they compute.

Its own metaprogramming language

Automate and extend Lean in Lean. Further reading: *Functional Programming in Lean*.

2013

Launched by Leonardo de Moura at Microsoft Research Redmond

Apache 2.0

Open license: use, extend and redistribute the code and mathematical libraries

Lean 4

The version used in this deck; install via the official installation guide, source on GitHub

Long-term

An ongoing project; automation is added step by step

CHAPTER 1 · §1.1–1.3 · BACKGROUND

How Lean processes a declaration: elaborator, then kernel

Source code
written by the user, omitting many details

→

Elaborator
infers types, fills implicit arguments, runs tactics

→

Full term
a “proof object” with no gaps left

→

Kernel
final type check: accept or reject

Why two layers?

The elaborator is large and complex, and may have bugs. The kernel is small and knows only a few basic rules. Every result must pass the kernel, so trusting a proof only requires trusting that small part.

The full term can be rechecked by another, independent program.

What does the user see?

In VS Code, Lean checks as you type. The Infoview shows three kinds of result: **information** (#check, #eval output), **warnings** (e.g. a proof using sorry), **errors** (red underline under faulty code).

CHAPTER 1 · §1.3–1.4

The approach and a first example

- **Foundation:** dependent type theory — the Calculus of Constructions with inductive types; strong enough for most ordinary mathematics
- **The computer fills in details:** term rewriting, automatic simplification, elaboration, type inference
- **Running examples:** "Copy to clipboard" → paste into VS Code; Lean checks as you type
- **Reading in VS Code:** the command "Lean 4: Docs: Show Documentation Resources"
- **§1.4:** an open project on GitHub; many contributors fixed errors, added examples and content

Example §1.3 — commutativity of ∧

```
theorem and_commutative (p q : Prop) :
    p ∧ q → q ∧ p :=
  fun hpq : p ∧ q ⟹
  have hp : p := And.left hpq
  have hq : q := And.right hpq
  show q ∧ p from And.intro hq hp
```

Reading: assume *hpq* proves p ∧ q; extract *hp*, *hq*; combine them into a proof of q ∧ p.

Line by line: and_commutative

```
theorem and_commutative (p q : Prop) :
    p ∧ q → q ∧ p :=
  fun hpq : p ∧ q ⟹
  have hp : p := And.left hpq
  have hq : q := And.right hpq
  show q ∧ p from And.intro hq hp
```

Context and expected type at the show line

```
p q : Prop
hpq : p ∧ q
hp : p
hq : q
⊢ q ∧ p
```

Line	Term	Meaning and type
1–2	`(p q : Prop)`	p, q are any propositions; the goal p ∧ q → q ∧ p is a **function type**
3	`fun hpq : p ∧ q ⟹`	Assume *hpq*, a proof of p ∧ q; the body must have type q ∧ p
4	`And.left hpq`	Left conjunct: a term of type p, named *hp*
5	`And.right hpq`	Right conjunct: a term of type q, named *hq*
6	`And.intro hq hp`	Combine q then p into a term of type q ∧ p — matching the type after show, so Lean accepts

Why type checking works: propositions are types

The Curry–Howard correspondence

Logic	Lean
proposition p	type p : Prop
proof of p	term hp : p
p ∧ q → q ∧ p	function type: turns a proof of p ∧ q into a proof of q ∧ p
“assume p ∧ q”	`fun hpq : p ∧ q ⇒ …`
split / build p ∧ q	`And.left, And.right` / `And.intro`
proof checking	type checking

The same proof without have / show (illustration)

```
theorem and_commutative' (p q : Prop) :
    p ∧ q → q ∧ p :=
  fun hpq ⇒
    And.intro (And.right hpq) (And.left hpq)
```

Common error. And.intro hp hq: Lean expects q first but gets hp : p — a type error. And.intro takes its arguments in the order of the sides of ∧.

> *have merely names an intermediate result, and show merely states the type being proved; both are covered in detail in Chapter 3.*

When a proof is wrong: Lean shows where and why

Swapping the two arguments on the last line (illustration, deliberately wrong)

```
theorem and_commutative (p q : Prop) :
    p ∧ q → q ∧ p :=
  fun hpq : p ∧ q ⇒
  have hp : p := And.left hpq
  have hq : q := And.right hpq
  show q ∧ p from And.intro hp hq
```

Lean underlines **hp** in red. The core of the message (the heading line varies by version):

```
  hp
has type
  p
but is expected to have type
  q
```

Reading the message in three steps

1. **Where?** The red underline marks the term Lean rejects: hp.
2. **What do we have?** “has type p”: the type of what we supplied.
3. **What is needed?** “expected … q”: the type that position requires. show q ∧ p forces the first argument of And.intro to prove q.

Fix: And.intro hq hp. The argument order of And.intro is the order of the two sides of ∧.

> *Tip: place the cursor inside the proof so the Infoview shows the context (available hypotheses) and the expected type there.*

CHAPTER 1 · CHAPTER SUMMARY

Chapter 1 summary: computers, proofs and Lean

Key ideas

- Formal verification: using logic and computation to establish precise mathematical claims — about theorems and about systems.
- Computers help **find** proofs (automated) and **check** proofs (interactive); Lean combines both.
- Foundation: dependent type theory (the Calculus of Constructions with inductive types).
- Proofs are terms, propositions types; a small kernel checks them.

Syntax seen in and_commutative

Syntax	Meaning
`theorem t (p q : Prop) : T`	claim T, named t
`fun h : A ⟹ …`	“assume h is a proof of A”
`have h : A := e`	a named intermediate result
`show A from e`	state the type being proved
`And.left / And.right`	extract one side of ∧
`And.intro a b`	combine two sides into ∧

CHAPTER 1 · QUESTIONS TO REFLECT ON

Questions to reflect on for Chapter 1

1. A SAT solver answers “this formula is satisfiable”. How far do we trust that answer? What more does an interactive prover such as Lean demand?

2. Why is trusting the kernel enough, even though the elaborator and tactics do most of the work?

3. Drop the two have lines in and_commutative and write And.intro (And.right hpq) (And.left hpq) directly. Is the proof still correct? Is it “different” from the original?

4. Verifying software and proving mathematical theorems are regarded as the same activity. Why?

Suggested answers for Chapter 1

1. Only as far as we trust the whole solver program, which favours speed and may have bugs. Lean demands a proof reaching down to the axioms and basic rules, as a proof object that other systems can recheck.

2. The elaborator and tactics only *build* terms; they cannot declare “proved” on their own. The final term must pass the kernel. If the elaborator has a bug, the worst outcome is that the kernel rejects the term — not that a false theorem is accepted.

3. Still correct: have only names an intermediate result, show only annotates the type. have h : A := e; t is (fun h => t) e — β-reduction yields exactly the direct term, so the two are definitionally equal. The have version is easier to read and debug.

4. Once hardware and software are described mathematically, “the system meets its specification” becomes a theorem. Conversely, a theorem may rely on a long computation — then the computation itself, i.e. a program, must be verified.

> *The common thread: a proof is a concrete, machine-checkable object — Chapter 2 builds the language for writing such objects.*

PART I · CHAPTER

2

Dependent Type Theory

Types, functions and universes

SECTIONS

2.1 Simple Type Theory

2.2 Types as objects

2.3 Function Abstraction and Evaluation

2.4 Definitions

2.5 Local Definitions

2.6 Variables and Sections

2.7 Namespaces

2.8 What makes dependent type theory dependent?

2.9 Implicit Arguments

Types resemble sets — and where the analogy stops

The similarity: read types as sets

Lean	Set-theoretic picture
`x : Nat`	$x \in \mathbb{N}$
`Bool`	{true, false}
`α × β`	Cartesian product $\alpha \times \beta$
`α → β`	the set of all functions from α to β
`List Nat`	finite sequences of natural numbers

This picture suffices for most examples in §2.1–2.4.

Where the analogy breaks down

- **Each term has exactly one type.** In set theory, $2 \in \mathbb{N}$ and $2 \in \mathbb{Z}$ at once; in Lean, (2 : Nat) and (2 : Int) are different terms.
- **"x : α" is not a proposition.** You cannot write "if x : Nat then …"; it is a judgment Lean checks, not something we prove.
- **No built-in union, intersection or subset.** Nat is not inside Int; conversion goes through a function (coercion).
- **Types are objects too:** Nat : Type — leading to universes (§2.2).

Every expression has a type

- **def** declares a new constant; **#check** asks for a type; **#eval** computes a value — query commands start with #
- Function types α → β (type \to) and Cartesian products α × β (type \times)
- Arrows associate to the right: Nat → Nat → Nat is Nat → (Nat → Nat) — "curried" functions, which can be partially applied
- Pairs (m, n); projections p.1 and p.2
- Nat is the natural numbers with arbitrary precision

Example — type checking and evaluation

```
#check Nat.add
-- Nat.add : Nat → Nat → Nat
#check Nat.add 3
-- Nat.add 3 : Nat → Nat
#check (5, 9).1
-- (5, 9).fst : Nat
#eval Nat.add 5 2 -- 7
#eval (5, 9).2 -- 9
```

#check and #eval: reading each line of output

Declaring constants and querying — comments are Lean's output

```
def m : Nat := 1 -- m is a natural number
def n : Nat := 0
def b1 : Bool := true -- b1 is a Boolean
def b2 : Bool := false

#check m -- m : Nat
#check n + 0 -- n + 0 : Nat
#check b1 && b2 -- b1 && b2 : Bool
#check true -- Bool.true : Bool

#eval 5 * 4 -- 20
#eval m + 2 -- 3
#eval b1 && b2 -- false
```

- **def m : Nat := 1** — name : type := value. Lean checks that 1 has type Nat, then adds the constant m to the environment.
- **#check** only infers the type, without computing: n + 0 is printed as is, n + 0 : Nat.
- **true** is printed as **Bool.true**: the full name, since true is a constructor of the type Bool (Chapter 7).
- **#eval** actually runs: m + 2 unfolds the definition m := 1 and computes 3.
- **&&**, **||** are operations on Bool: the result is a Bool value, not yet a proposition in Prop.
- **Common errors:** #eval on a function lacking arguments (e.g. #eval Nat.add) — Lean reports an error, since it cannot display a function.

Currying and partial application

Types of a function and of each application (Lean's output)

```
#check Nat.succ -- Nat.succ (n : Nat) : Nat
#check Nat.add -- Nat.add : Nat → Nat → Nat
#check Nat.add 3 -- Nat.add 3 : Nat → Nat
#check Nat.add 5 2 -- Nat.add 5 2 : Nat

#check Nat → Nat → Nat
#check Nat → (Nat → Nat) -- same type as above
#check Nat × Nat → Nat
```

Naming a partially applied function (illustration)

```
def add3 : Nat → Nat := Nat.add 3
#eval add3 4 -- 7
```

Reading the type argument by argument

```
Nat.add       : Nat → (Nat → Nat)
Nat.add 5     : Nat → Nat
(Nat.add 5) 2 : Nat
```

- Arrows associate to the right, application to the left: Nat.add 5 2 is (Nat.add 5) 2. The two conventions match, so a two-argument function is just a function returning a function.
- Nat.add 3 is a function "waiting" for its second argument.
- Nat × Nat → Nat takes a pair: it cannot be partially applied.
- #check on a constant such as Nat.succ prints a signature with the parameter name (n : Nat); its type is still Nat → Nat.

Why types need many “levels”: boxes within boxes

In Lean every expression has a type — Nat and Bool included. So what is the type of Nat? And what is the type of that type?

Picture: objects sit in boxes; boxes sit in a crate; crates sit in a warehouse… Each level holds the one below, and no box contains itself.

Object	Lives in
`3, true`	`Nat, Bool`
`Nat, Bool, List Nat`	`Type`
`Type`	`Type 1`
`Type 1`	`Type 2, …`

Background: Russell’s paradox in three lines

> Let R be “the set of all sets that do not contain themselves”.
>
> If $R \in R$ then by definition $R \notin R$; if $R \notin R$ then $R \in R$.
>
> Contradiction: “the set of all sets” is too big.

So too for types: a “type of all types” containing itself makes the system inconsistent. Lean stratifies: Type lies not in Type but in Type 1.

> *In practice we almost always use Type; higher levels appear when functions take or return types themselves.*

Types are objects too: the universe hierarchy

- Nat, Bool have type Type; functions on types: List : Type → Type
- Type : Type 1, Type 1 : Type 2, … — Type abbreviates Type 0
- Universe polymorphism: List.{u}, Prod.{u, v}; via universe u

```
#check Type -- Type : Type 1
#check Type 1 -- Type 1 : Type 2
#check List
-- List.{u} (α : Type u) : Type u
```

sort	Prop (Sort 0)	Type (Sort 1)	Type 1 (Sort 2)	Type 2 (Sort 3)
type	True	Bool	Nat → Type	Type → Type 1
term	True.intro	true	fun n ⇒ Fin n	fun (_ : Type) ⇒ Type

Across: change universe. Down: sort → type → term.

Why no Type : Type? The universe hierarchy

> Allowing Type : Type makes the system **inconsistent**: Girard's paradox (akin to the Russell / Burali-Forti paradoxes of set theory) builds a term of type False — every proposition becomes "provable".

- **Solution:** a countable sequence of universes Type 0 : Type 1 : Type 2 : … — each universe lives in the one just above it, and none contains itself.
- **Type** abbreviates Type 0: the universe of "small", ordinary types such as Nat, Bool, List Nat.
- **Non-cumulative:** Nat : Type, but Lean does not automatically treat Nat as an element of Type 1.
- **Universe polymorphism:** a universe variable u lets List, Prod work at every level; Prod α β lives in Type (max u v) so it can hold both α and β.

Hierarchy and universe polymorphism (Lean's output)

```
#check Type -- Type : Type 1
#check Type 1 -- Type 1 : Type 2
#check Type 2 -- Type 2 : Type 3

#check List
-- List.{u} (α : Type u) : Type u
#check Prod
-- Prod.{u, v} (α : Type u) (β : Type v)
--   : Type (max u v)

universe u
def F (α : Type u) : Type u := Prod α α
#check F -- F.{u} (α : Type u) : Type u
```

Sort, Prop and Type: reading the table column by column

Kind	term	: type	: sort	as Sort
proof	`True.intro`	`True`	`Prop`	`Sort 0`
data	`true`	`Bool`	`Type`	`Sort 1`
type family	`fun n ⇒ Fin n`	`Nat → Type`	`Type 1`	`Sort 2`
function returning a universe	`fun (_ : Type) ⇒ Type`	`Type → Type 1`	`Type 2`	`Sort 3`

Sort in Lean's output (illustration; last line from §2.9)

```
#check Prop -- Prop : Type
#check Sort 0 -- Prop : Type
#check Sort 1 -- Type : Type 1
#check @id -- @id : {α : Sort u_1} → α → α
```

- **Sort u** is the general form: Prop = Sort 0, Type u = Sort (u+1).
- **Prop** is the bottom level, for propositions; its terms are proofs (Chapter 3). Prop itself : Type.
- **Nat → Type lives in Type 1:** the function's values are types, and Type : Type 1, so the function type moves up a level.
- **id** uses Sort u_1, so it applies to both data and proofs.

CHAPTER 2 · §2.3–2.4

Function abstraction and definitions

- **fun** (or λ) creates a function; Lean usually infers the variable's type
- Two terms differing only in bound variable names are **alpha equivalent**
- Every term has computational behaviour (normalization); terms reducing to the same value are **definitionally equal**
- **def** is a "named fun"; state the type explicitly; it accepts type parameters and higher-order functions
- def is *semireducible*: unfolded only when needed — with def Natural := Nat, declaring five : Natural := 5 fails for lack of OfNat Natural 5

Example — λ and def

```
#check fun (x : Nat) ⇒ x + 5
-- fun x ⇒ x + 5 : Nat → Nat
#eval (λ x : Nat ⇒ x + 5) 10 -- 15

def double : Nat → Nat :=
  fun x ⇒ x + x

-- equivalent form:
-- def double (x : Nat) : Nat := x + x
```

Reading λ-expressions: type inference and substitution

```
def f (n : Nat) : String := toString n
def g (s : String) : Bool := s.length > 0
#check fun x ⟹ g (f x) -- fun x ⟹ g (f x) : Nat → Bool
#check fun x y ⟹ if not y then x + 1 else x + 2
-- fun x y ⟹ if (!y) = true then x + 1 else x + 2 : Nat → Bool → Nat
#check
  (fun (α β γ : Type) (u : β → γ) (v : α → β) (x : α) ⟹ u (v x)) Nat String Bool g f 0
```

Type inference. x : Nat because f takes a Nat; y : Bool because of not y. Lean prints not y as (!y) = true: if needs a proposition, so the Bool value becomes an equation with true.

Substitution on application. α := Nat, β := String, γ := Bool, u := g, v := f, x := 0. The body u (v x) becomes g (f 0), so the whole expression has type Bool.

Bound variables. Names after fun are mere placeholders: fun (b : β) (x : α) => b and fun (u : β) (z : α) => u are alpha equivalent — Lean treats them as one term.

CHAPTER 2 · §2.3–2.4

Definitional equality: reduction, #reduce and rfl

Every term can be computed

```
#eval (fun x : Nat ⇒ x) 1 -- 1
#eval (fun x : Nat ⇒ true) 1 -- true
def double (x : Nat) : Nat :=
  x + x
#eval double 3 -- 6
```

Reduction and proof by rfl (illustration)

```
#reduce (fun x : Nat ⇒ x + 5) 10 -- 15
#reduce fun x : Nat ⇒ x + 0 -- fun x ⇒ x
example : double 3 = 6 := rfl
example (x : Nat) : x + 0 = x := rfl
```

Reduction step	Example
β — apply a λ	`(fun x ⇒ x + 5) 10 → 10 + 5`
δ — unfold a definition	`double 3 → 3 + 3`
ζ — unfold a let	`let y := 2; y * y → 2 * 2`

- Two terms reducing to the same form are **definitionally equal**; the type checker treats them as one, so rfl proves the equation between them.
- **#eval** compiles and runs — fast, for testing programs. **#reduce** uses a kernel-like reducer — slower, but it can reduce expressions containing variables.
- **Common errors:** x + 0 = x closes by rfl, but 0 + x = x does not: Nat.add recurses on its second argument, so with a variable x the computation gets “stuck”.

A complete example: functions taking functions, functions taking types

Four definitions and three evaluations (comments are Lean's output)

```
def double (x : Nat) : Nat := x + x
def square (x : Nat) : Nat := x * x
def add (x : Nat) (y : Nat) := x + y
def doTwice (f : Nat → Nat) (x : Nat)
    : Nat := f (f x)
def compose (α β γ : Type) (g : β → γ)
    (f : α → β) (x : α) : γ :=
  g (f x)

#eval add (double 3) (7 + 9) -- 22
#eval doTwice double 2 -- 8
#eval compose Nat Nat Nat double square 3
-- 18
```

Step-by-step reduction

```
add (double 3) (7 + 9)
→ add 6 16 → 6 + 16 → 22
```

```
doTwice double 2
→ double (double 2) → double 4 → 8
```

```
compose Nat Nat Nat double square 3
→ double (square 3) → double 9 → 18
```

doTwice takes a *function* f : Nat → Nat as argument — a higher-order function.

compose takes three *types* α β γ. The types of g and f force f's output to match g's input; here g := double, f := square, so square is computed first.

> *The type of compose mentions the earlier arguments α β γ — the seed of dependent types (§2.8).*

CHAPTER 2 · §2.5–2.7

Local definitions, variables, sections and namespaces

Construct	Syntax	Meaning
let	`let a := t1; t2`	Definitionally equal to t2 with every a replaced by t1; not the same as (fun a => t2) t1
variable	`variable (α : Type)`	Inserted automatically as a parameter of definitions that use it; #print shows the result
section	`section … end`	Limits the scope of variable; can be named and nested
namespace	`namespace Foo … end Foo`	Hierarchical names Foo.a; open Foo for short names; can be reopened, even in another file

let is stronger than λ — this line type-checks:

```
def foo := let a := Nat; fun x : a ⟹ x + 2
```

Namespaces organize data; sections declare variables. Both limit the scope of variable, open and set_option.

let and fun: similar, but not the same

let in a definition; foo type-checks, bar does not

```
def twice_double (x : Nat) : Nat :=
  let y := x + x; y * y
#eval twice_double 2 -- 16

def foo := let a := Nat; fun x : a ⟹ x + 2
/-
  def bar := (fun a ⟹ fun x : a ⟹ x + 2) Nat
-/
```

*twice_double x is definitionally equal to (x + x) * (x + x) (illustration)*

```
example (x : Nat) :
    twice_double x = (x + x) * (x + x) := rfl
```

	foo: let a := Nat; …	**bar: (fun a => …) Nat**
a when checking the body	abbreviates Nat — known	an arbitrary variable, type unknown
x + 2 with x : a	valid: Nat has + and the numeral 2	error: no + for an arbitrary type a
Result	add-2 function on Nat	does not type-check

> *fun a => t2 must make sense for* ***every*** *a, regardless of the value passed later. But let a := t1; t2 checks t2 knowing that a is t1 — so let is a stronger abbreviation.*

variable and section: Lean inserts the parameters

Declare once, use throughout the section

```
section useful
  variable (α β γ : Type)
  variable (g : β → γ) (f : α → β) (h : α → α)
  variable (x : α)

  def compose := g (f x)
  def doTwice := h (h x)
  def doThrice := h (h (h x))
end useful
```

Same effect as writing the parameters explicitly

```
def compose (α β γ : Type) (g : β → γ)
    (f : α → β) (x : α) : γ :=
  g (f x)

def doTwice (α : Type) (h : α → α)
    (x : α) : α :=
  h (h x)
```

Only variables that are **used** get inserted, in declaration order: compose takes α β γ g f x but not h; doTwice takes only α h x. Check with #print compose.

> *After end useful, α, g, x… go out of scope. Sections may be anonymous or nested; a named section must be closed with that same name.*

namespace: full names, short names and open

Excerpt from the namespace Foo example

```
namespace Foo
  def a : Nat := 5
  def f (x : Nat) : Nat := x + 7
  def fa : Nat := f a
  #check a
  #check Foo.fa
end Foo

-- #check a -- error
#check Foo.a

open Foo
#check a
```

Location	short name a	Foo.a
inside namespace Foo	works	works
after end Foo	error	works
after open Foo	works	works

- Every name declared in namespace Foo gets a full name Foo.a, Foo.f, Foo.fa.
- Nesting: namespace Bar inside Foo gives the name Foo.Bar.ffa; close them in reverse order. A closed namespace can be reopened, even in another file.
- The library uses the same mechanism: List.nil, List.cons, List.map; after open List you can write nil, cons, map.
- Unlike sections, a namespace must be named; variable and open inside it also lapse when it is closed.

Functions whose result type depends on the input value

An ordinary function is like a coffee machine: whichever button you press, you get "a cup of coffee". The output type is fixed: Nat → Nat.

A dependent function is like a currency exchange: say "USD" and you get USD, say "EUR" and you get EUR. The *kind* of thing returned is decided by the *value* given.

Mathematical example: "given n, return a vector of length n" has type (n : Nat) → Vector α n. Vector α 3 and Vector α 5 are different types, so the length is recorded in the type and Lean can check it.

> *Notation: (b : Bool) → T b — the argument is named b so the type after it can refer to it.*

A miniature currency exchange (illustration)

```
def T : Bool → Type
  | true  ⇒ Nat
  | false ⇒ String

def pick : (b : Bool) → T b
  | true  ⇒ (42 : Nat)
  | false ⇒ ("hi" : String)

#check pick true  -- pick true : T true
#check pick false -- pick false : T false
```

T is a **type family**: each b gives a type. pick true has type T true, which is by definition Nat; pick false has type String. pick cannot be given type Bool → Nat or Bool → String.

CHAPTER 2 · §2.8

What makes type theory “dependent”?

Types can depend on parameters: List α, Vector α n.

Dependent function types (Π)

`(a : α) → β a`

with β : α → Type; α → β is the case where β does not depend on a

Dependent pair types (Σ)

`(a : α) × β a` or `Σ a : α, β a`

built with ⟨a, b⟩ or Sigma.mk a b

Example — the type of cons α depends on α

```
def cons (α : Type) (a : α) (as : List α)
    : List α :=
  List.cons a as

#check cons Nat
-- cons Nat : Nat → List Nat → List Nat
#check cons
-- cons (α : Type) (a : α) (as : List α)
-- : List α
```

Dependent function types: reading the type of cons

A first guess, **Type → α → List α → List α**, makes no sense: α refers to no argument. The first argument must be named so later types can refer to it:

```
(α : Type) → α → List α → List α
```

Applying one argument at a time (illustration)

```
cons : (α : Type) → α → List α → List α
cons Nat : Nat → List Nat → List Nat
cons Nat 1 : List Nat → List Nat
cons Nat 1 [] : List Nat
```

In general: **(a : α) → β a** where β : α → Type is a type family. When β does not depend on a, (a : α) → β is just α → β.

Types of cons and of List functions (Lean's output)

```
#check cons Bool
-- cons Bool : Bool → List Bool → List Bool
#check @List.cons
-- @List.cons :
--   {α : Type u_1} → α → List α → List α
#check @List.nil
-- @List.nil : {α : Type u_1} → List α
#check @List.length
-- @List.length :
--   {α : Type u_1} → List α → Nat
```

> *Chapter 4: ∀ x : α, p x is a dependent function type, with p x : Prop.*

Σ types: dependent pairs, computing h1 5 step by step

Two ways to build a dependent pair

```
universe u v

def f (α : Type u) (β : α → Type v)
    (a : α) (b : β a) : (a : α) × β a :=
  ⟨a, b⟩
def g (α : Type u) (β : α → Type v)
    (a : α) (b : β a) : Σ a : α, β a :=
  Sigma.mk a b

def h1 (x : Nat) : Nat :=
  (f Type (fun α ⟹ α) Nat x).2
#eval h1 5 -- 5
```

Inside h1 5: substitution and type checking

```
α := Type,  β := fun α ⟹ α
a := Nat,   b := 5
need b : β a = (fun α ⟹ α) Nat ≡ Nat ✓
f … = ⟨Nat, 5⟩ : (a : Type) × a
.2 : β Nat ≡ Nat,  value 5
```

- Σ generalizes α × β: the type of the second component depends on the **value** of the first.
- ⟨a, b⟩ (type \< \>) and Sigma.mk a b are one pair: f and g are the same.
- Intuition: ⟨Nat, 5⟩ and ⟨Bool, true⟩ have the same type (α : Type) × α — “a value bundled with its type”.
- Chapter 4: ∃ x, p x is the Prop version of Σ.

Comparison: → and × versus dependent Π and Σ

	Non-dependent	Dependent
Function	`α → β`	`(a : α) → β a`
reads as	every output has the same type β	the output at a has type β a
example	`Nat.succ : Nat → Nat`	`cons : (α : Type) → α → List α → List α`
Pair	`α × β`	`(a : α) × β a`
reads as	the second component always has type β	the second component has type β a, where a is the first
example	`(5, 9) : Nat × Nat`	`⟨Nat, 5⟩ : (α : Type) × α`
Prop version (ch. 3–4)	`p → q,  p ∧ q`	`∀ x : α, p x,  ∃ x : α, p x`

The left column is a special case of the right: when β does not mention a, (a : α) → β is just α → β; likewise × is a non-dependent Σ.

The last row looks ahead: ∀ is Π with a proposition as body; ∃ is the Prop version of Σ — "a value together with evidence about it".

Implicit arguments and elaboration

- _ asks Lean to fill it in: Lst.cons _ 0 (Lst.nil _)
- **{α : Type u}**: an argument implicit by default; also works with variable {α : Type u}
- **Elaboration**: the process of filling in “holes” (metavariables) — polymorphic expressions such as id, List.nil
- Type ascription **(e : T)**: (List.nil : List Nat); numerals default to Nat, (2 : Int) is an integer
- **@foo**: every argument becomes explicit

Example — exposing implicit arguments

```
#check @id
-- @id : {α : Sort u_1} → α → α
#check @id Nat 1
-- id 1 : Nat

#check (List.nil : List Nat)
#check (2 : Int)
```

From explicit arguments to _ and { }

Final version: α in curly braces

```
universe u
def Lst (α : Type u) : Type u := List α

def Lst.cons {α : Type u} (a : α) (as : Lst α)
    : Lst α := List.cons a as
def Lst.nil {α : Type u} : Lst α := List.nil

#check Lst.cons 0 Lst.nil
def bs : Lst Nat := Lst.cons 5 Lst.nil
```

Declaring α	Call building [5] : Lst Nat
(α : Type u)	`Lst.cons Nat 5 (Lst.nil Nat)`
(α : Type u), with _	`Lst.cons _ 5 (Lst.nil _)`
{α : Type u}	`Lst.cons 5 Lst.nil`

With the explicit version, Lean fills in _ and prints it back:

```
#check Lst.append _ as bs
-- Lst.append Nat as bs : Lst Nat
```

How does Lean infer α? The first hole: from 5 : Nat. The hole in Lst.nil _: not from itself, but from its position — Lst.cons needs a Lst α there. {...} makes _ the default, also with variable {α : Type u}.

Elaboration: metavariables, type ascriptions and @

Comments are Lean's actual output

```
universe u
def ident {α : Type u} (x : α) := x
#check (ident) -- ident : ?m.22 → ?m.22
#check ident 1 -- ident 1 : Nat
#check @ident -- @ident : {α : Type u_1} → α → α

#check (List.nil) -- [] : List ?m.2
#check (List.nil : List Nat) -- [] : List Nat
#check (id : Nat → Nat) -- id : Nat → Nat

#check 2 -- 2 : Nat
#check (2 : Int) -- 2 : Int
#check @id Nat -- id : Nat → Nat
#check @id Bool true -- id true : Bool
```

- **?m.22, ?m.2** are metavariables: "holes" the elaborator could not fill for lack of context; the number is just an internal index.
- **ident 1**: the hole α is solved via 1 : Nat — a numeral without context defaults to Nat.
- **(e : T)** gives the elaborator an expected type: matching List ?m with List Nat yields ?m := Nat.
- **(ident)** in parentheses forces Lean to elaborate it as a term, instead of printing the declared signature.
- @ makes every argument explicit, {α} included; u_1 is a universe variable Lean generates.
- Puzzling error? set_option pp.explicit true prints the hidden arguments too (Chapter 6).

Type mismatch and Unknown identifier

A value that does not match the declared type

```
def x : Nat := "Not a number"
```

```
error: Type mismatch
  "Not a number"
has type
  String
but is expected to have type
  Nat
```

Line 2: the offending term. **has type**: the type Lean infers from the term itself. **expected**: the type the context demands — here, the type after the colon. Fix the value or the declared type.

A name out of scope (from the namespace Foo example, §2.7)

```
namespace Foo
  def a : Nat := 5
end Foo
#check a     -- error
#check Foo.a -- Foo.a : Nat
```

```
error: Unknown identifier `a`
```

Lean cannot find the name in the current scope. Common causes: a typo; a missing open or namespace prefix; using a variable after its section has closed; using a name before its declaration (Lean reads top to bottom).

failed to synthesize and the metavariable ?m

```
def Natural := Nat
def five : Natural := 5
```

```
failed to synthesize instance of type class
  OfNat Natural 5
numerals are polymorphic in Lean, but the
numeral `5` cannot be used in a context
where the expected type is
  Natural
due to the absence of the instance above
```

(followed by a hint to enable trace.Meta.synthInstance; line breaks added to fit)

Reading: the numeral 5 is polymorphic — Lean needs a "way to read 5 in the type Natural", i.e. an instance OfNat Natural 5 (type classes, Chapter 10). Natural is a def, unfolded only when truly needed; instance search does not unfold it, so it does not see Nat's instance.

Fix (illustration): write abbrev Natural := Nat (a definition that is always unfolded), or write (5 : Nat).

?m in the output

#check (List.nil) prints [] : List ?m.2. ?m is a "hole" the elaborator could not fill (§2.9). Seeing ?m in an error message means Lean lacks information: add a type ascription (e : T) or explicit arguments with @.

CHAPTER 2 · CHAPTER SUMMARY

Chapter 2 summary: dependent type theory

Types have types too: Nat : Type : Type 1 : … Prop = Sort 0.

Functions are objects: built with fun, used by application; currying.

Computation: same reduced form ⇒ definitionally equal.

Dependency: types can mention values: Π and Σ.

Syntax	Meaning	Syntax	Meaning
`def f (x : α) : β := t`	define a constant/function	`variable (x : α)`	shared parameter
`#check e / #eval e`	show type / run	`section … end`	limit scope
`fun x ⟹ t`	anonymous function (λ)	`namespace N / open N`	name N.a / short name a
`let a := t1; t2`	local definition	`universe u`	universe variable
`(a : α) → β a`	dependent function	`{α : Type u}`	implicit argument
`(a : α) × β a`	dependent pair ⟨a, b⟩	`_ / @f / (e : T)`	Lean fills in / explicit / type ascription

CHAPTER 2 · QUESTIONS TO REFLECT ON

Questions to reflect on for Chapter 2

1. Natural is defined as Nat itself. Why does def five : Natural := 5 still fail?

2. Why does Lean not allow Type : Type? What is the type of Nat → Type, and why is it not Type?

3. let a := Nat; fun x : a => x + 2 type-checks, but (fun a => fun x : a => x + 2) Nat does not. Explain in words.

4. Why can the type of cons not be written Type → α → List α → List α? More generally: why is α → β just a special case of (a : α) → β a?

Suggested answers to the Chapter 2 questions

1. The numeral 5 has no fixed type: Lean must find an instance OfNat Natural 5. def is semireducible — unfolded only when truly needed to compare two terms — so instance search does not see that Natural is Nat. abbrev, or (5 : Nat), solves it.

2. Type : Type allows building Girard's paradox (a relative of Russell's paradox), yielding a term of type False. Nat → Type has type Type 1: the function's values are types, and Type : Type 1, so the function type lives one level up.

3. With let, Lean checks the body knowing that a is Nat, so x + 2 makes sense. With fun a => ..., the body must type-check for every unknown a : Type; an arbitrary type has no + or numeral 2. Passing Nat in afterwards cannot rescue it.

4. In Type → α → ..., the α refers to no argument. It must be named: (α : Type) → α → List α → List α. When the output type does not mention the argument's name, naming it is redundant, and (a : α) → β shortens to α → β.

> *Chapter 3 uses this very machinery for logic: propositions are types in Prop, proofs are terms.*

PART I · CHAPTER

3

Propositions and Proofs

Curry–Howard and propositional logic

SECTIONS

3.1 Propositions as Types

3.2 Working with Propositions as Types

3.3 Propositional Logic

3.4 Introducing Auxiliary Subgoals

3.5 Classical Logic

3.6 Examples of Propositional Validities

— Further problems

Propositional logic on one page

Symbol	Read as	True when
$p \wedge q$	p and q	both are true
$p \vee q$	p or q	at least one side is true
$\neg p$	not p	p is false
$p \to q$	if p then q	false only when p is true and q false
$p \leftrightarrow q$	p if and only if q	p, q both true or both false
True / False	true / false	always true / never true

Tautology: true under every assignment of true/false to the variables, e.g. $p \wedge q \to q \wedge p$.

Two ways to establish a tautology

Truth tables: try every true/false combination — 2^n rows for n variables. This is the "semantic" view.

Deduction: proceed step by step by rules. Each connective has **introduction** rules (how to build evidence) and **elimination** rules (how to use it). E.g. modus ponens: from $p \to q$ and p, get q.

> *Lean takes the second route: rather than computing whether p is true, it builds evidence of the right type. This extends to ∀ and ∃, where truth tables no longer work.*

Propositions are types: the Curry–Howard correspondence

Logic	Type theory
Proposition p	Type p : Prop
Proof of p	Term t : p
Implication p → q	Function type p → q
Proof of p → q	Function turning proofs of p into proofs of q
Proof checking	Type checking

Prop is Sort 0; **Type u** is Sort (u+1). Prop is closed under →.

Two readings: constructive (proofs are data) and a "coding trick" (type inhabited ⇔ proposition true).

Proof irrelevance: any t1 t2 : p are definitionally equal.

> *To state a claim: give p : Prop. To prove it: give t : p — Lean checks that t has the right type.*

From the naive Proof p encoding to Curry–Howard

Naive approach: a separate proof type for each proposition

```
def Implies (p q : Prop) : Prop := p → q
structure Proof (p : Prop) : Type where
  proof : p
#check Proof  -- Proof (p : Prop) : Type

axiom and_commut (p q : Prop) :
  Proof (Implies (And p q) (And q p))

axiom modus_ponens (p q : Prop) :
  Proof (Implies p q) → Proof p →
  Proof q

axiom implies_intro (p q : Prop) :
  (Proof p → Proof q) → Proof (Implies p q)
```

Step 1 · Encode. Each p : Prop has a type Proof p. An “axiom” is a constant of type Proof …; checking a proof t means checking t : Proof p.

Step 2 · Spot the structure. modus_ponens is just **function application**; implies_intro is just **λ-abstraction**: a function Proof p → Proof q gives Proof (Implies p q).

Step 3 · Identify Proof p with p. Read t : p as “t is a proof of p”. Then Implies p q is just p → q, so drop Implies and Proof entirely.

> *The inference rules for → coincide with the rules for λ and application: the Curry–Howard correspondence of the Calculus of Constructions.*

Proof irrelevance: you need a ticket, not a particular ticket

Picture: the ticket inspector only needs to know you hold a valid ticket. Two tickets with different serial numbers grant exactly the same right: entry.

So it is with a proposition: all that matters is that it *has* a proof. Two different proofs of 2 + 2 = 4 — one by computation, one via lemmas — are treated by Lean as one.

Data is different: 3 and 5 both have type Nat but differ, and that difference is the whole information.

	Data (Type)	Proofs (Prop)
Key question	which element?	is there one or not?
Two elements	may differ: 3 ≠ 5	always definitionally equal
At run time	kept	erased at compile time

Consequences we will meet again

- theorem need not unfold its proof (§3.2).
- From a proof of ∃ x, p x one generally cannot extract x as a value (Chapters 4, 12).

Two readings of “propositions as types” and proof irrelevance

Question	Constructive reading	“Coding trick” reading
What is p : Prop?	A specification of the data that constitutes a proof of p	An empty type if p is false; a type with exactly one element * if p is true
What is t : p?	An abstract mathematical object, denoted by t	“The fact that p is true”; the well-typed expression itself is the proof
Proof of p → q	A function from proof data for p to proof data for q	Uses “the fact that p is true” to get “the fact that q is true”

Proof irrelevance. For p : Prop, the kernel treats any t1 t2 : p as *definitionally equal* — just as it identifies (fun x => t) s with t[s/x]. A proof carries no information beyond the truth of p.

Prop is Sort 0, Type u is Sort (u+1); Prop is closed under →: p q : Prop ⇒ p → q : Prop.

The kernel accepts any two proofs as one (illustration)

```
example (p : Prop) (t1 t2 : p) :
    t1 = t2 := rfl
```

Consequence: proofs need not be “unfolded” — see theorem.

Working with propositions as types: theorem

- **theorem** is a form of def; the kernel treats them alike
- Practical differences: proofs are marked *irreducible* and can be checked in parallel; only section variables occurring in the *type* become parameters
- **#print** shows the proof term; **show p from e** makes the type explicit
- Apply theorems like functions: t1 hp
- **axiom** postulates a term — it can make the system unsound (axiom unsound : False)

Example — a theorem is a function

```
variable {p : Prop}
variable {q : Prop}

theorem t1 : p → q → p :=
  fun hp : p ⟹ fun hq : q ⟹ hp

#print t1
#check @t1
-- ∀ {p q : Prop}, p → q → p
```

Comparing theorem and def

Aspect	def	theorem
To the kernel	Defines a constant t : T	The same: proving p defines an element of p
Unfolding	Unfolded when computing	Marked irreducible: by proof irrelevance, no need to unfold
Checking	In dependency order	Can be parallel: one proof needs no details of another
Section variables	All variables used in the body	Only those in the *type*

Why the last row: how something is proved must not change what is stated.

Parameters left of the colon give the same term

```
variable {p : Prop}
variable {q : Prop}

theorem t1 (hp : p) (hq : q) : p := hp

#print t1
```

Lean returns

```
theorem t1 : ∀ {p q : Prop}, p → q → p :=
  fun {p q} hp hq ⟹ hp
```

{p q} are added because p, q occur in the type; hp, hq are temporary hypotheses — that is, λs.

A general theorem is a function: instantiate step by step

Explicit p, q: apply t1 to any propositions

```
theorem t1 (p q : Prop) (hp : p) (hq : q) : p := hp

variable (p q r s : Prop)

#check t1 p q
#check t1 r s
#check t1 (r → s) (s → r)

variable (h : r → s)

#check t1 (r → s) (s → r) h
```

h : r → s is read as the hypothesis “r → s holds”; putting h in place of hp uses it.

Lean returns, in order

```
t1 p q : p → q → p
t1 r s : r → s → r
t1 (r → s) (s → r) :
  (r → s) → (s → r) → r → s
t1 (r → s) (s → r) h :
  (s → r) → r → s
```

Each argument “consumes” the first arrow of the type: set p := r → s, q := s → r, then supply hp := h.

> *axiom hp : p declares p true without proof. axiom unsound : False makes everything provable: False.elim unsound : 1 = 0.*

Curry–Howard as a dictionary: translating paper proofs into terms

Theorem. If q implies r and p implies q, then p implies r.

(a) Assume h_1: q → r and h_2: p → q.

(b) To prove p → r, further assume h_3: p.

(c) From h_2 and h_3 we get q.

(d) From h_1 and q we get r. Hence p → r.

```
variable (p q r s : Prop)

theorem t2 (h₁ : q → r) (h₂ : p → q) : p → r :=
  fun h₃ : p ⟹
  show r from h₁ (h₂ h₃)
```

Line	Lean	Type · rule
(a)	`(h₁ : q → r) (h₂ : p → q)`	parameters = hypotheses
(b)	`fun h₃ : p ⟹ …`	→ introduction = λ
(c)	`h₂ h₃`	q · → elimination = application
(d)	`h₁ (h₂ h₃)`	r · apply once more
“Hence”	`fun h₃ : p ⟹ h₁ (h₂ h₃)`	p → r

Reading the dictionary backwards: t2 is just **function composition** $h_1 \circ h_2$ — the same idea as compose in Chapter 2, except the types live in Prop.

> *Translation rules: “assume” ↦ fun; “from A and A → B, get B” ↦ function application; “hence” ↦ close the λ.*

Logical connectives: introduction and elimination rules

Connective	Type as	Introduction (intro)	Elimination (elim)
∧ And	\and	And.intro hp hq, ⟨hp, hq⟩	And.left h, And.right h
∨ Or	\or	Or.inl hp, Or.inr hq	Or.elim h f g
¬ Not	\not	fun hp => … (¬p is p → False)	hnp hp, absurd hp hnp
→	\to	fun hp => …	application: h hp
↔ Iff	\iff	Iff.intro h1 h2	Iff.mp, Iff.mpr
False	—	(none)	False.elim (ex falso)
True	—	True.intro	(none)

Precedence: ¬ > ∧ > ∨ > → > ↔; binary connectives associate to the right. Dot notation: h.left is And.left h. **example** checks a claim without naming it.

CHAPTER 3 · §3.3.1–3.3.3

Examples: ∧, ∨ and ¬ in practice

∧ — two ways to write the same proof

```
variable (p q : Prop)
example (h : p ∧ q) : q ∧ p :=
  And.intro (And.right h) (And.left h)
example (h : p ∧ q) : q ∧ p :=
  ⟨h.right, h.left⟩
```

∨ — proof by cases

```
example (h : p ∨ q) : q ∨ p :=
  Or.elim h
    (fun hp : p ⇒
      show q ∨ p from Or.intro_right q hp)
    (fun hq : q ⇒
      show q ∨ p from Or.intro_left p hq)
```

¬ — negation is p → False; ex falso with absurd

```
example (hpq : p → q) (hnq : ¬q) : ¬p :=
  fun hp : p ⇒ show False from hnq (hpq hp)
example (hp : p) (hnp : ¬p) : q := absurd hp hnp
```

Line by line: swapping p ∧ q

```
example (h : p ∧ q) : q ∧ p :=
  And.intro (And.right h) (And.left h)
example (h : p ∧ q) : q ∧ p :=
  ⟨h.right, h.left⟩
```

```
example (h : p ∧ q) : q ∧ p ∧ q :=
  ⟨h.right, ⟨h.left, h.right⟩⟩
example (h : p ∧ q) : q ∧ p ∧ q :=
  ⟨h.right, h.left, h.right⟩
```

Subterm	Type	Rule / reason
`And.right h`	`q`	right ∧ elimination, from h : p ∧ q
`And.left h`	`p`	left ∧ elimination
`And.intro (And.right h) (And.left h)`	`q ∧ p`	∧ introduction from proofs of q, p
`⟨h.right, h.left⟩`	`q ∧ p`	anonymous constructor; h.right is And.right h
`⟨h.right, h.left, h.right⟩`	`q ∧ (p ∧ q)`	∧ is right-associative, so nested ⟨⟩ flatten

> *Like swapping a pair (a, b) : α × β — but And lives in Prop, Prod in Type; Lean keeps the two types separate.*

Or.elim is case analysis

Fully type-annotated version, then the short one

```
example (h : p ∨ q) : q ∨ p :=
  Or.elim h
    (fun hp : p ⇒
      show q ∨ p from Or.intro_right q hp)
    (fun hq : q ⇒
      show q ∨ p from Or.intro_left p hq)

example (h : p ∨ q) : q ∨ p :=
  h.elim (fun hp ⇒ Or.inr hp) (fun hq ⇒ Or.inl hq)
```

Short version: h.elim is Or.elim h; Lean infers the types of hp, hq. Type annotations aid reading and catch errors.

The rule, with r := q ∨ p

```
Or.elim : p ∨ q → (p → r) →
  (q → r) → r
```

Subterm	Type
`h`	`p ∨ q`
`Or.intro_right q hp`	`q ∨ p`
`fun hp : p ⇒ …`	`p → q ∨ p`
`Or.intro_left p hq`	`q ∨ p`
`fun hq : q ⇒ …`	`q → q ∨ p`

Or has two constructors (inl, inr), so ⟨⟩ cannot be used; Or.inr is Or.intro_right _.

¬p is p → False: a worked example

Contraposition, ex falso and absurd

```
example (hpq : p → q) (hnq : ¬q) : ¬p :=
  fun hp : p ⟹
  show False from hnq (hpq hp)

example (hp : p) (hnp : ¬p) : q := False.elim (hnp hp)

example (hp : p) (hnp : ¬p) : q := absurd hp hnp

example (hnp : ¬p) (hq : q) (hqp : q → p) : r :=
  absurd (hqp hq) hnp
```

False has only an elimination rule (False.elim); True has only an introduction rule: True.intro : True.

Subterm (example 1)	Type
`hp`	p — temporary hypothesis (λ)
`hpq hp`	q
`hnq (hpq hp)`	False, since hnq : q → False
`fun hp : p ⟹ …`	p → False, i.e. ¬p

```
False.elim : False → C  (C implicit)
absurd     : a → ¬a → b
```

> ***Common error:*** *absurd hnp hp — wrong order. The proof of a comes first, then ¬a. In the last example, a := p with hqp hq : p.*

A complete example: ↔ is two implications packaged together

```
variable (p q : Prop)

theorem and_swap : p ∧ q ↔ q ∧ p :=
  Iff.intro
    (fun h : p ∧ q ⟹
     show q ∧ p from
       And.intro (And.right h) (And.left h))
    (fun h : q ∧ p ⟹
     show p ∧ q from
       And.intro (And.right h) (And.left h))

#check and_swap p q
-- and_swap p q : p ∧ q ↔ q ∧ p
variable (h : p ∧ q)
example : q ∧ p := Iff.mp (and_swap p q) h
```

Constant	Type
`Iff.intro`	`(a → b) → (b → a) → (a ↔ b)`
`Iff.mp`	`(a ↔ b) → a → b`
`Iff.mpr`	`(a ↔ b) → b → a`

Build: Iff.intro takes two functions, one per direction; here both directions use the same swap “recipe”.

Use: and_swap p q is a proof of ↔; Iff.mp extracts the forward direction, of type p ∧ q → q ∧ p, and applies it to h.

Short form: ⟨fun h => ⟨h.right, h.left⟩, fun h => ⟨h.right, h.left⟩⟩ and (and_swap p q).mp h.

CHAPTER 3 · §3.4–3.5

Auxiliary goals and classical logic

§3.4 — have (forward) and suffices (backward)

```
example (h : p ∧ q) : q ∧ p :=
  have hp : p := h.left
  suffices hq : q from And.intro hq hp
  show q from And.right h
```

- have h : p := s; t is just (fun (h : p) => t) s
- suffices: “it suffices to prove q”, then prove q

§3.5 — double-negation elimination from excluded middle

```
open Classical
theorem dne {p : Prop} (h : ¬¬p) : p :=
  Or.elim (em p)
    (fun hp : p ⟹ hp)
    (fun hnp : ¬p ⟹ absurd hnp h)
```

- The earlier rules are constructive; open Classical gives em p : p ∨ ¬p
- Classical patterns: byCases, byContradiction — e.g. ¬(p ∧ q) → ¬p ∨ ¬q

Goal states through have and suffices

have — forward reasoning

```
example (h : p ∧ q) : q ∧ p :=
  have hp : p := h.left
  have hq : q := h.right
  show q ∧ p from And.intro hq hp
```

Context and goal after each line

```
1: h : p ∧ q      ⊢ q ∧ p
2: h, hp : p      ⊢ q ∧ p
3: h, hp, hq : q  ⊢ q ∧ p
4: And.intro hq hp : q ∧ p ✓
```

suffices — backward reasoning (last two lines swapped)

```
example (h : p ∧ q) : q ∧ p :=
  have hp : p := h.left
  suffices hq : q from And.intro hq hp
  show q from And.right h
```

Line 3 creates two obligations

```
2: h, hp : p     ⊢ q ∧ p
3a: + hq : q     ⊢ q ∧ p — from closes
3b: h, hp        ⊢ q — remaining
4: And.right h : q ✓
```

> *Why it type-checks: have h : p := s; t is just (fun (h : p) => t) s — a λ applied immediately.*

CHAPTER 3 · §3.5 CLASSICAL LOGIC

¬¬p → p two ways: byCases and byContradiction

After open Classical; variable (p : Prop)

```
example (h : ¬¬p) : p :=
  byCases
    (fun h1 : p ⟹ h1)
    (fun h1 : ¬p ⟹ absurd h1 h)
```

- **byCases** : (p → q) → (¬p → q) → q — just Or.elim (em p)
- Branch 1: h1 : p, need p — return h1
- Branch 2: h1 : ¬p contradicts h : ¬¬p; absurd h1 h uses a := ¬p

Same context

```
example (h : ¬¬p) : p :=
  byContradiction
    (fun h1 : ¬p ⟹
     show False from h h1)
```

- **byContradiction** : (¬p → False) → p
- Assume h1 : ¬p, need False
- h : ¬¬p is (p → False) → False; applied to h1 it gives h h1 : False

> *Why Classical is needed: a constructive "or" means knowing which side holds. With RH the Riemann hypothesis, a classical mathematician asserts RH ∨ ¬RH without knowing which — em p allows exactly that.*

A classical proof: ¬(p ∧ q) → ¬p ∨ ¬q

After open Classical; variable (p q : Prop)

```
example (h : ¬(p ∧ q)) : ¬p ∨ ¬q :=
  Or.elim (em p)
    (fun hp : p ⇒
      Or.inr
        (show ¬q from
          fun hq : q ⇒
          h ⟨hp, hq⟩))
    (fun hp : ¬p ⇒
      Or.inl hp)
```

Subterm	Type
`em p`	`p ∨ ¬p`
`⟨hp, hq⟩`	`p ∧ q`
`h ⟨hp, hq⟩`	`False`
`fun hq : q ⇒ …`	`¬q`
`Or.inr (…)`	`¬p ∨ ¬q`
`Or.inl hp`	`¬p ∨ ¬q`

Branch where p holds: q cannot hold (else h is violated), so choose the right side. Branch ¬p: choose the left side at once.

> *Why classical logic is needed: knowing p and q are not both true does not tell us which is false; em p "picks" a branch for us.*

A complete classical example: ¬(p ∧ ¬q) → (p → q)

```
open Classical

example (p q : Prop) :
    ¬(p ∧ ¬q) → (p → q) :=
  fun h : ¬(p ∧ ¬q) ⇒
  fun hp : p ⇒
  show q from
    Or.elim (em q)
      (fun hq : q ⇒ hq)
      (fun hnq : ¬q ⇒
        absurd (And.intro hp hnq) h)
```

Skeleton: two λs for the two arrows (assume h, assume hp), then the two cases q or ¬q supplied by em q.

Subterm	Type
`h`	`p ∧ ¬q → False`
`em q`	`q ∨ ¬q`
branch 1: hq	`q — done`
`And.intro hp hnq`	`p ∧ ¬q`
`absurd (…) h`	q — since p ∧ ¬q and ¬(p ∧ ¬q) clash

> ***Why classical logic is needed:*** *from h and hp, constructively we only get ¬¬q (assuming ¬q gives p ∧ ¬q, a contradiction). Going from ¬¬q to q is exactly the dne step that needs em.*

Representative propositional tautologies

Constructively provable (selection)

```
p ∧ q ↔ q ∧ p
(p ∧ q) ∧ r ↔ p ∧ (q ∧ r)
p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r)
p ∨ (q ∧ r) ↔ (p ∨ q) ∧ (p ∨ r)
(p → (q → r)) ↔ (p ∧ q → r)
((p ∨ q) → r) ↔ (p → r) ∧ (q → r)
¬(p ∨ q) ↔ ¬p ∧ ¬q
¬(p ∧ ¬p)
(p → q) → (¬q → ¬p)
¬(p ↔ ¬p)
```

Require classical reasoning

```
(p → r ∨ s) → ((p → r) ∨ (p → s))
¬(p ∧ q) → ¬p ∨ ¬q
¬(p → q) → p ∧ ¬q
(p → q) → (¬p ∨ q)
(¬q → ¬p) → (p → q)
p ∨ ¬p
((p → q) → p) → p
```
(Peirce)

Tip: `sorry` “proves” anything (Lean warns) — use it to build a skeleton top-down; `_` makes Lean report the goal to fill, with its context.

The sorry and _ workflow: building proofs top-down

Skeleton for the distributive law (illustration)

```
example (p q r : Prop) :
    p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) :=
  Iff.intro
    (fun h : p ∧ (q ∨ r) ⇒ _)
    (fun h : (p ∧ q) ∨ (p ∧ r) ⇒ sorry)
```

1. Write the skeleton with sorry in every gap; Lean must accept it (warnings only). If it errors, the skeleton is wrong.
2. Replace one sorry with _: Lean reports the exact subgoal there.
3. Fill in gradually, repeating until no sorry remains.

Lean reports at _

```
don't know how to synthesize placeholder
context:
p q r : Prop
h : p ∧ (q ∨ r)
⊢ p ∧ q ∨ p ∧ r
```

The message gives both the context and the type to build — exactly what we need to write the next step (here: have hp : p := h.left, then Or.elim h.right).

> ***Common error:*** *a leftover sorry. It can "prove" even False; Lean warns "declaration uses `sorry`" on every declaration that uses or imports it.*

Filling the sorry skeleton: distributivity, forward direction

Forward direction complete; backward left as sorry for now

```
example (p q r : Prop) :
    p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) :=
  Iff.intro
    (fun h : p ∧ (q ∨ r) ⇒
      have hp : p := h.left
      Or.elim (h.right)
        (fun hq : q ⇒
          show (p ∧ q) ∨ (p ∧ r) from Or.inl ⟨hp, hq⟩)
        (fun hr : r ⇒
          show (p ∧ q) ∨ (p ∧ r) from Or.inr ⟨hp, hr⟩))
    (fun h : (p ∧ q) ∨ (p ∧ r) ⇒ sorry)
```

After have hp

```
h : p ∧ (q ∨ r),  hp : p
⊢ p ∧ q ∨ p ∧ r
```

The q branch of Or.elim

```
… hq : q
⊢ p ∧ q ∨ p ∧ r  ← Or.inl ⟨hp, hq⟩
```

The r branch of Or.elim

```
… hr : r
⊢ p ∧ q ∨ p ∧ r  ← Or.inr ⟨hp, hr⟩
```

Just as _ suggested on the previous slide: split off p first, then consider the two cases of q ∨ r.

Backward direction: Or.elim h; in each branch split the pair with have, then build ⟨hp, Or.inl hq⟩ or ⟨hp, Or.inr hr⟩.

Six frequent errors in term-style proofs

Situation	Lean reports	Reading and fixing
And.intro hp hq when q ∧ p is needed	hp “has type p but is expected to have type q”	Misplaced argument; argument order = order of the sides
absurd hnp hp	similar: hnp fixes a to ¬p, so hp : p is flagged (¬¬p expected)	absurd : a → ¬a → b — the proof of a comes first
Using em without open Classical	`Unknown identifier `em``	Add open Classical, or write Classical.em
Using hp outside fun hp => …	`Unknown identifier `hp``	A λ’s hypothesis lives only in the λ’s body
Leaving _ where Lean cannot fill it	`don't know how to synthesize placeholder`	Not really an error: the ⊢ line in the message is the goal to fill
A leftover sorry	`declaration uses `sorry``	Only a warning, but the theorem is not yet proved

> *Good habit: annotate types (fun hp : p => …, show … from …). Errors then appear at the annotation, not somewhere distant and obscure.*

CHAPTER 3 · CHAPTER SUMMARY

Chapter 3 summary: propositions and proofs

Curry–Howard: propositions are types in Prop, proofs are terms, proof checking is type checking.

Each connective has introduction and elimination rules; → is λ and application; ¬p is p → False.

Proof irrelevance: all proofs of the same proposition are one.

Classical = add em p : p ∨ ¬p; needed for ¬¬p → p and one direction of De Morgan.

Syntax	Meaning	Syntax	Meaning
`theorem / example`	named / unnamed	`Or.inl / Or.inr / Or.elim`	build / use ∨
`fun h : p ⇒ …`	assume p	`absurd / False.elim`	anything from a contradiction
`have / suffices`	forward / backward reasoning	`Iff.intro / .mp / .mpr`	build / use ↔
`show p from e`	state the type	`em / byCases / byContradiction`	classical reasoning
`⟨hp, hq⟩, h.left`	build / use ∧	`sorry / _`	gap / ask for the goal

CHAPTER 3 · QUESTIONS TO REFLECT ON

Questions to reflect on for Chapter 3

1. Why is defining ¬p as p → False enough, with no need for a new connective with its own rules?

2. If all proofs of p are equal, why do we still care how a proof is written?

3. ¬¬p → p needs em, but p → ¬¬p does not. Write a term for the direction without em and explain the asymmetry.

4. What does axiom unsound : False break? Why is a forgotten sorry dangerous in exactly the same way?

Suggested answers to the Chapter 3 questions

1. The rules for ¬ come for free from those for →: a proof of ¬p is a λ turning a hypothesis p into False (introduction); given hnp : ¬p and hp : p, hnp hp : False (elimination). Everything else comes from False.elim.

2. The kernel does not distinguish them, but people do: clear proofs are easier to read, easier to repair when definitions change, and faster to check. Also, a technique written for p can be reused for other propositions.

4. False implies anything (False.elim), so a false axiom makes every theorem worthless. sorry acts as an axiom that "proves anything"; Lean only warns, so read warnings carefully.

3. p → ¬¬p is p → ((p → False) → False): given hp, take hnp, return hnp hp.

```
-- (illustration)
example (p : Prop) : p → ¬¬p :=
  fun hp hnp ⟹ hnp hp
```

The converse requires building a proof of p from nothing but a function (p → False) → False — which holds no proof of p to extract. Constructively there is no way; em p lets us consider "p or ¬p" and rule out the ¬p branch.

PART I · CHAPTER

4

Quantifiers and Equality

∀, ∃, equality and calc

SECTIONS

4.1 The Universal Quantifier

4.2 Equality

4.3 Calculational Proofs

4.4 The Existential Quantifier

4.5 More on the Proof Language

— Further problems

∀ is a machine: feed it any x, get back a proof

"Every natural number n satisfies n + 0 = n" is a **promise:** "give me any n and I will return a proof of n + 0 = n".

That promise is a **function.** The result type n + 0 = n varies with n — exactly the dependent function of §2.8.

- **Proving ∀** = writing the machine: fun n => ..., where n is arbitrary — nothing more may be assumed about n.
- **Using ∀** = feeding the machine a concrete value: h 5.

The machine for n + 0 = n (illustration)

```
theorem add_zero' : ∀ n : Nat, n + 0 = n :=
  fun n ⇒ rfl

#check add_zero' 5
-- add_zero' 5 : 5 + 0 = 5
```

	Ordinary function	∀
type	`Nat → Nat`	`∀ n : Nat, p n`
build	`fun n ⇒ n + 1`	`fun n ⇒ (… : p n)`
use	`f 5 : Nat`	`h 5 : p 5`

The universal quantifier ∀ is a dependent function type

- Predicate p : α → Prop; relation r : α → α → Prop
- **Introduction:** from p x for arbitrary x, infer ∀ x : α, p x — λ-abstraction
- **Elimination:** from ∀ x, p x and t : α, infer p t — function application
- ∀ x : α, p is notation for (x : α) → p; p → q is the case where q does not depend on x
- Typing rule: α : Sort i, β : Sort j ⇒ (x : α) → β : Sort (imax i j); imax i 0 = 0, so **Prop is impredicative**

Example — introducing and eliminating ∀

```
example (α : Type) (p q : α → Prop) :
    (∀ x : α, p x ∧ q x) → ∀ y : α, p y :=
  fun h : ∀ x : α, p x ∧ q x ⇒
  fun y : α ⇒
  show p y from (h y).left
```

Reading: h is a function; applying h to y gives a proof of p y ∧ q y, then take the left part.

Introducing and eliminating ∀: the type of each subterm

```
example (α : Type) (p q : α → Prop) :
    (∀ x : α, p x ∧ q x) → ∀ y : α, p y :=
  fun h : ∀ x : α, p x ∧ q x ⇒
  fun y : α ⇒
  show p y from (h y).left
```

Subterm	Type · rule
`h`	(x : α) → p x ∧ q x — a function
`y`	α, arbitrary — ∀ introduction
`h y`	p y ∧ q y — ∀ elimination = application
`(h y).left`	p y — ∧ elimination
`fun y : α ⇒ …`	∀ y : α, p y

```
variable (α : Type) (r : α → α → Prop)
variable (trans_r : ∀ x y z,
    r x y → r y z → r x z)
variable (a b c : α)
variable (hab : r a b) (hbc : r b c)
```

```
#check after each additional argument is applied
trans_r : ∀ (x y z : α), r x y → r y z → r x z
trans_r a b c : r a b → r b c → r a c
trans_r a b c hab : r b c → r a c
trans_r a b c hab hbc : r a c
```

Instantiate x y z := a b c, then use hab, hbc as hypotheses of the two arrows.

CHAPTER 4 · §4.1–4.2

Equivalence relations: from a general r to equality

Arguments x y z are implicit — inferred from the hypotheses

```
variable (α : Type) (r : α → α → Prop)

variable (refl_r : ∀ x, r x x)
variable (symm_r : ∀ {x y}, r x y → r y x)
variable (trans_r : ∀ {x y z},
    r x y → r y z → r x z)

example (a b c d : α) (hab : r a b)
    (hcb : r c b) (hcd : r c d) : r a d :=
  trans_r (trans_r hab (symm_r hcb)) hcd
```

> ***Note:*** *#check trans_r on its own gives r ?m.1 ?m.2 → r ?m.2 ?m.3 → r ?m.1 ?m.3 — nothing yet determines x y z.*

Subterm	Type
`symm_r hcb`	`r b c`
`trans_r hab (symm_r hcb)`	`r a c`
`trans_r (…) hcd`	`r a d`

Replace r with = : Eq.symm, Eq.trans play the roles of symm_r, trans_r

```
variable (α : Type) (a b c d : α)
variable (hab : a = b) (hcb : c = b)
         (hcd : c = d)
example : a = d :=
  Eq.trans (Eq.trans hab (Eq.symm hcb)) hcd
example : a = d :=
  (hab.trans hcb.symm).trans hcd
```

Impredicativity in plain words: Prop may talk about itself

For data (predicative): a type that talks about *every type in Type* must live above Type. (α : Type) → α → α has type Type 1 — it does not land back in Type.

For propositions (impredicative): a proposition about *every proposition* is still a proposition. ∀ p : Prop, p → p has type Prop — it falls within the range of its own "every p".

Picture: a parliament passes a law that applies to "every law", including itself. For Prop, Lean allows this.

Defining a predicate by quantifying over all predicates (illustration)

```
def Ind (n : Nat) : Prop :=
  ∀ P : Nat → Prop,
    P 0 → (∀ k, P k → P (k + 1)) → P n
```

"n has every property that 0 has and that passes from k to k + 1." Ind is a predicate on Nat, yet it is defined by mentioning *every* predicate P on Nat — Ind included. Thanks to imax, the result still has type Prop.

> *The price: proofs in Prop generally cannot be used to extract data — Prop is the world of "true/false", not of values.*

Why Prop is impredicative: the imax rule

```
α : Sort i, β : Sort j ⇒ (x : α) → β : Sort (imax i j)
imax i j = max i j if j ≠ 0;  imax i 0 = 0
```

Function type / ∀	i	j	Result
`Nat → Prop`	1	1	Sort 1 = Type
`∀ x : Nat, x = x`	1	0	Sort 0 = Prop
`∀ p : Prop, p → p`	1	0	Sort 0 = Prop
`(α : Type) → α → α`	2	1	Sort 2 = Type 1

j ≠ 0: the dependent function is data and lives in the larger universe. j = 0: the body is a proposition, so ∀ x : α, β is still a proposition, however large α is — even α = Prop.

Checking the table on the left (illustration)

```
#check Nat → Prop          -- Type
#check ∀ x : Nat, x = x    -- Prop
#check ∀ p : Prop, p → p   -- Prop
#check (α : Type) → α → α -- Type 1
```

> ***Context:*** *Poincaré and Russell blamed the set-theoretic paradoxes on a "vicious circle" — defining a property by quantifying over a collection that contains it. In Prop we may quantify over α → Prop (all predicates on α) to define a new predicate: exactly that kind of circularity.*

Equality is an inductive type with a single constructor

A copy of the library's Eq (illustration)

```
universe u

inductive MyEq {α : Sort u} : α → α → Prop where
  | refl (a : α) : MyEq a a

#check MyEq.refl 3
-- MyEq.refl 3 : MyEq 3 3
```

Reading: for all a b : α, MyEq a b is a proposition. The *only* way to build evidence is refl a, and it proves only MyEq a a.

Build: to get a = b, a and b must coincide definitionally. Hence 2 + 3 = 5 := rfl is accepted: 2 + 3 reduces to 5.

Use: given h : a = b, h can only come from refl, i.e. "b is really a". So every property of a transfers to b — that is Eq.subst and ▸.

Consequence: symmetry, transitivity and congrArg all follow from this substitution; they need no separate axioms.

> *Inductive types and the "use" principle (the recursor) are covered fully in Chapter 7.*

Equality: reflexivity, substitution and congruence

Tool	Meaning
`Eq.refl a`, `rfl`	a = a; proves equalities that hold by computation
`Eq.symm`, `Eq.trans`	Symmetry, transitivity (h.symm, h.trans)
`Eq.subst`, `h1 ▸ h2`	From a = b and p a, infer p b
`congrArg`, `congrFun`, `congr`	Replace the argument, the function, or both

rfl is notation for Eq.refl _: it closes any equality whose two sides reduce to the same term, such as 2 + 3 = 5 or (a, b).1 = a. Library: Nat.add_comm, Nat.mul_add, … Type ▸ as \t.

Examples

```
example : 2 + 3 = 5 := rfl

example (α : Type) (a b : α) (p : α → Prop)
    (h1 : a = b) (h2 : p a) : p b :=
  h1 ▸ h2

-- with f : α → Nat, h₁ : a = b
example : f a = f b := congrArg f h₁
```

Substituting equals: Eq.subst, ▸ and the congr family

From h1 : a = b and h2 : p a, infer p b

```
example (α : Type) (a b : α) (p : α → Prop)
        (h1 : a = b) (h2 : p a) : p b :=
  Eq.subst h1 h2

example (α : Type) (a b : α) (p : α → Prop)
    (h1 : a = b) (h2 : p a) : p b :=
  h1 ▸ h2
```

Eq.subst has an implicit parameter motive : α → Prop and type motive a → motive b. Lean must find a motive such that motive a matches p a — **higher-order unification**, undecidable in general. The ▸ macro (typed \t) builds on Eq.subst and Eq.symm with better heuristics, so it often succeeds where Eq.subst fails.

Replace the argument, the function, or both

```
variable (α : Type)
variable (a b : α)
variable (f g : α → Nat)
variable (h₁ : a = b)
variable (h₂ : f = g)

example : f a = f b := congrArg f h₁
example : f a = g a := congrFun h₂ a
example : f a = g b := congr h₂ h₁
```

All three are defined from Eq.subst, with a motive chosen in advance for terms of the form s t; e.g. congrArg f h_1 corresponds to the motive fun x => f a = f x.

A computation: $(x + y)^2$ by substitution and associativity

```
example (x y : Nat) :
    (x + y) * (x + y) =
    x * x + y * x + x * y + y * y :=
  have h1 : (x + y) * (x + y) = (x + y) * x + (x + y) * y :=
    Nat.mul_add (x + y) x y
  have h2 : (x + y) * (x + y) = x * x + y * x + (x * y + y * y) :=
    (Nat.add_mul x y x) ▸ (Nat.add_mul x y y) ▸ h1
  h2.trans (Nat.add_assoc (x * x + y * x) (x * y) (y * y)).symm
```

The right-hand side at each step (▸ associates to the right: applied from the inside out)

```
h1 : … = (x + y) * x + (x + y) * y          Nat.mul_add with a := x + y, b := x, c := y
▸ : … = (x + y) * x + (x * y + y * y)       Nat.add_mul x y y : (x + y) * y = x * y + y * y
▸ : … = x * x + y * x + (x * y + y * y)     Nat.add_mul x y x : (x + y) * x = x * x + y * x
trans : … = x * x + y * x + x * y + y * y   add_assoc flipped by .symm: drops last parens
```

Calculational proofs with calc

```
variable (a b c d e : Nat)
theorem T
    (h1 : a = b)
    (h2 : b = c + 1)
    (h3 : c = d)
    (h4 : e = 1 + d) :
    a = e :=
  calc
    a = b     := h1
    _ = c + 1 := h2
    _ = d + 1 := congrArg Nat.succ h3
    _ = 1 + d := Nat.add_comm d 1
    _ = e     := Eq.symm h4
```

- A chain of steps `_ op eᵢ := proofᵢ`, at the same indentation
- Each step may use **by rw [...]** or **by simp**
- **rw** rewrites the goal, then tries to close it by reflexivity; ← rewrites right to left
- calc works for any transitive relation and can mix =, <, ≤
- Extend it with **instance : Trans ...** — e.g. for divisibility

Justifying each calc step in theorem T

#	Step proposition	Proof	Why it type-checks
1	`a = b`	`h1`	The hypothesis, verbatim
2	`b = c + 1`	`h2`	_ is the previous right-hand side (b); matches h2
3	`c + 1 = d + 1`	`congrArg Nat.succ h3`	c + 1 is Nat.succ c by definition; congrArg applies Nat.succ to h3 : c = d
4	`d + 1 = 1 + d`	`Nat.add_comm d 1`	Commutativity a + b = b + a, instantiated at a := d, b := 1
5	`1 + d = e`	`Eq.symm h4`	h4 : e = 1 + d points the wrong way; Eq.symm flips it

Chaining. calc joins the five steps by transitivity (for =, Eq.trans) into a = e.

Syntax. All lines align; you may write calc a and start the first line with _ = … to line things up.

Mixing relations. =, <, ≤ can be combined; a new relation needs a Trans instance.

rw inside calc: the goal at each step

Theorem T, each step proved by rw

```
theorem T
    (h1 : a = b)
    (h2 : b = c + 1)
    (h3 : c = d)
    (h4 : e = 1 + d) :
    a = e :=
  calc
    a = b      := by rw [h1]
    _ = c + 1  := by rw [h2]
    _ = d + 1  := by rw [h3]
    _ = 1 + d  := by rw [Nat.add_comm]
    _ = e      := by rw [h4]
```

Goal before → after rw, then rfl closes it

```
⊢ a = b
  rw [h1] ⇒ ⊢ b = b ✓
⊢ b = c + 1
  rw [h2] ⇒ ⊢ c + 1 = c + 1 ✓
⊢ c + 1 = d + 1
  rw [h3] ⇒ ⊢ d + 1 = d + 1 ✓
⊢ d + 1 = 1 + d
  rw [Nat.add_comm] ⇒ ⊢ 1 + d = 1 + d ✓
⊢ 1 + d = e
  rw [h4] ⇒ ⊢ 1 + d = 1 + d ✓
```

rw replaces the left-hand side of an equation with its right-hand side; ←h rewrites the other way. Steps combine: a = d + 1 := by rw [h1, h2, h3], or the whole theorem: by rw [h1, h2, h3, Nat.add_comm, h4].

A complete example: calc mixing =, < and ≤

```
variable (a b c d : Nat)
example (h1 : a = b) (h2 : b ≤ c)
    (h3 : c + 1 < d) : a < d :=
  calc
    a = b     := h1
    _ < b + 1 := Nat.lt_succ_self b
    _ ≤ c + 1 := Nat.succ_le_succ h2
    _ < d     := h3
```

Each _ is the right-hand side of the line above; each line may use a different relation.

Combining relations step by step

```
a = b,     b < b + 1  ⇒ a < b + 1
a < b + 1, b + 1 ≤ c + 1 ⇒ a < c + 1
a < c + 1, c + 1 < d   ⇒ a < d ✓
```

#	Step	Why it holds
1	a = b	hypothesis h1
2	b < b + 1	Nat.lt_succ_self b : b < succ b; b + 1 is exactly succ b
3	b + 1 ≤ c + 1	Nat.succ_le_succ : n ≤ m → succ n ≤ succ m, applied to h2
4	c + 1 < d	hypothesis h3

> *calc chains adjacent steps using the built-in Trans instances for the pairs (=, <), (<, ≤), (<, <). A new relation — such as "divides" — needs its own Trans instance.*

∃ is a pair: a witness together with evidence

"Someone in the room speaks French." The most convincing way: point to that person (the **witness** w) and let us hear them speak (the **evidence** h : p w).

Building ∃: the pair ⟨w, h⟩ — like the dependent pair Σ of §2.8.

Using ∃: "call that person w" and keep reasoning. But we do not know who w actually is, so the conclusion must not depend on who w is.

> *Unlike Σ: by proof irrelevance, all proofs of ∃ x, p x are equal. A function returning the witness used would give two different results on two proofs that are equal — a contradiction.*

Building and using ∃ (illustration)

```
example : ∃ n : Nat, n + n = 4 :=
  ⟨2, rfl⟩

example (h : ∃ n : Nat, n + n = 4) :
    ∃ m : Nat, 4 = m + m :=
  match h with
  | ⟨w, hw⟩ ⇒ ⟨w, hw.symm⟩
```

Term	Type
`rfl`	`2 + 2 = 4`
`w, hw`	`Nat,  w + w = 4`
`hw.symm`	`4 = w + w`

The existential quantifier ∃

```
example (x y z : Nat) (hxy : x < y) (hyz : y < z) : ∃ w, x < w ∧ w < z :=
  ⟨y, hxy, hyz⟩

example (h : ∃ x, p x ∧ q x) : ∃ x, q x ∧ p x :=
  match h with
  | ⟨w, hw⟩ ⇒ ⟨w, hw.right, hw.left⟩

theorem even_plus_even (h1 : IsEven a) (h2 : IsEven b) : IsEven (a + b) :=
  match h1, h2 with
  | ⟨w1, hw1⟩, ⟨w2, hw2⟩ ⇒ ⟨w1 + w2, by rw [hw1, hw2, Nat.mul_add]⟩
```

Introduction: Exists.intro t h, or ⟨t, h⟩

Elimination: Exists.elim, match, let ⟨w, hw⟩ := h

Essence: ∃ is the Prop version of Σ; ∃ x, p x is Exists (fun x => p x)

Introducing ∃: the witness and the implicit predicate

```
example : ∃ x : Nat, x > 0 :=
  have h : 1 > 0 := Nat.zero_lt_succ 0
  Exists.intro 1 h

example (x : Nat) (h : x > 0) : ∃ y, y < x :=
  Exists.intro 0 h

#check @Exists.intro
```

```
@Exists.intro : ∀ {α : Sort u_1}
  {p : α → Prop} (w : α), p w → Exists p
```

Example 2: x > 0 is just 0 < x, so h : p 0 with p := fun y => y < x. Shorthand: ⟨1, h⟩, ⟨0, h⟩.

The same term ⟨0, hg⟩, with hg : g 0 0 = 0 and g : Nat → Nat → Nat, proves all four theorems — because p is inferred from the *expected type*:

Expected type	p inferred by Lean
`gex1: ∃ x, g x x = x`	`fun x ⟹ g x x = x`
`gex2: ∃ x, g x 0 = x`	`fun x ⟹ g x 0 = x`
`gex3: ∃ x, g 0 0 = x`	`fun x ⟹ g 0 0 = x`
`gex4: ∃ x, g x x = 0`	`fun x ⟹ g x x = 0`

Substituting x := 0 into each p yields g 0 0 = 0. To see the actual p: set_option pp.explicit true, then #print gex1.

> *Exists.intro "hides" the witness inside the assertion.*

Eliminating ∃: Exists.elim and the forms of match

```
variable (α : Type) (p q : α → Prop)

example (h : ∃ x, p x ∧ q x) : ∃ x, q x ∧ p x :=
  Exists.elim h
    (fun w ⇒
     fun hw : p w ∧ q w ⇒
     show ∃ x, q x ∧ p x from ⟨w, hw.right, hw.left⟩)
```

```
Exists.elim : (∃ x, p x) → (∀ a, p a → b) → b
```

Subterm	Type
`w`	α — an arbitrary witness
`hw`	`p w ∧ q w`
`⟨w, hw.right, hw.left⟩`	`∃ x, q x ∧ p x`

The same proof, written with match, let, fun

```
match h with
| ⟨w, hw⟩ ⇒ ⟨w, hw.right, hw.left⟩

match h with
| ⟨w, hpw, hqw⟩ ⇒ ⟨w, hqw, hpw⟩

let ⟨w, hpw, hqw⟩ := h
⟨w, hqw, hpw⟩

fun ⟨w, hpw, hqw⟩ ⇒ ⟨w, hqw, hpw⟩
```

The last line is for statements of the form (∃ x, p x ∧ q x) → … ; the pattern ⟨w, hpw, hqw⟩ also splits the ∧.

> *∃ x, p x is like a big "or" over all a : α; the conclusion may not mention w. ∃ is the Prop version of Σ.*

even_plus_even: a step-by-step trace

```
def IsEven (a : Nat) := ∃ b, a = 2 * b

theorem even_plus_even (h1 : IsEven a) (h2 : IsEven b) :
    IsEven (a + b) :=
  Exists.elim h1 (fun w1 (hw1 : a = 2 * w1) ⇒
  Exists.elim h2 (fun w2 (hw2 : b = 2 * w2) ⇒
    Exists.intro (w1 + w2)
      (calc a + b
        _ = 2 * w1 + 2 * w2 := by rw [hw1, hw2]
        _ = 2 * (w1 + w2)   := by rw [Nat.mul_add])))
```

IsEven a unfolds to ∃ b, a = 2 * b; Lean adds a, b automatically as implicit arguments.

> *Short form: match h1, h2 with | ⟨w1, hw1⟩, ⟨w2, hw2⟩ => ⟨w1 + w2, by rw [hw1, hw2, Nat.mul_add]⟩*

After two Exists.elim

```
w1 : Nat, hw1 : a = 2 * w1
w2 : Nat, hw2 : b = 2 * w2
⊢ IsEven (a + b)
```

Exists.intro (w1 + w2) still needs

```
⊢ a + b = 2 * (w1 + w2)
```

calc step 1 — rw [hw1, hw2]

```
⊢ a + b = 2 * w1 + 2 * w2
⇒ 2 * w1 + 2 * w2 = 2 * w1 + 2 * w2 ✓
```

calc step 2 — rw [Nat.mul_add]

```
⊢ 2 * w1 + 2 * w2 = 2 * (w1 + w2)
⇒ 2 * w1 + 2 * w2 = 2 * w1 + 2 * w2 ✓
```

Classical ∃: from ¬∀ x, ¬p x infer ∃ x, p x

Proof by contradiction

```
open Classical
variable (p : α → Prop)

example (h : ¬ ∀ x, ¬ p x) : ∃ x, p x :=
  byContradiction
    (fun h1 : ¬ ∃ x, p x ⇒
      have h2 : ∀ x, ¬ p x :=
        fun x ⇒
        fun h3 : p x ⇒
        have h4 : ∃ x, p x := ⟨x, h3⟩
        show False from h1 h4
      show False from h h2)
```

byContradiction: assume the opposite

```
h1 : ¬ ∃ x, p x  ⊢ False
```

Lemma h2, inside two λs

```
x : α, h3 : p x  ⊢ False
⟨x, h3⟩ : ∃ x, p x;  h1 h4 : False ✓
```

Finish

```
h2 : ∀ x, ¬ p x;  h h2 : False ✓
```

> *Why classical logic is needed: knowing "not every x satisfies ¬p" does not hand us a concrete x satisfying p. Constructive ∃ demands a witness — just as constructive ∨ demands knowing which side holds.*

A complete example: ∃ distributes over ∨

```
variable (α : Type) (p q : α → Prop)

example : (∃ x, p x ∨ q x) ↔
    (∃ x, p x) ∨ (∃ x, q x) :=
  Iff.intro
    (fun ⟨a, (h1 : p a ∨ q a)⟩ ⇒
      Or.elim h1
        (fun hpa : p a ⇒ Or.inl ⟨a, hpa⟩)
        (fun hqa : q a ⇒ Or.inr ⟨a, hqa⟩))
    (fun h : (∃ x, p x) ∨ (∃ x, q x) ⇒
      Or.elim h
        (fun ⟨a, hpa⟩ ⇒ ⟨a, (Or.inl hpa)⟩)
        (fun ⟨a, hqa⟩ ⇒ ⟨a, (Or.inr hqa)⟩))
```

Subterm	Type
`fun ⟨a, h1⟩ ⇒ …`	opens ∃ right in the parameter: a : α, h1 : p a ∨ q a
`⟨a, hpa⟩`	`∃ x, p x`
`Or.inl ⟨a, hpa⟩`	`(∃ x, p x) ∨ (∃ x, q x)`
`⟨a, Or.inl hpa⟩`	`∃ x, p x ∨ q x`

Forward: open ∃, split the two cases of ∨, close with ∃ on the right side.

Backward: split ∨ first, open ∃ in each branch, then build ∃ with the predicate “p x ∨ q x”.

> *No need for em: the witness a comes from the hypothesis. ∃ and ∨ are both “or” — one over all x, the other over two sides.*

A complete example: a chain of anonymous have and this

```
variable (f : Nat → Nat)
variable (h : ∀ x : Nat, f x ≤ f (x + 1))

example : f 0 ≤ f 3 :=
  have : f 0 ≤ f 1 := h 0
  have : f 0 ≤ f 2 := Nat.le_trans this (h 1)
  show f 0 ≤ f 3 from Nat.le_trans this (h 2)
```

Variant: replace this with (by assumption) — Lean searches the context for a hypothesis of exactly the needed type.

Term	Type
`h 0`	`f 0 ≤ f (0 + 1)`
this (1st)	`f 0 ≤ f 1`
`h 1`	`f 1 ≤ f (1 + 1)`
this (2nd)	`f 0 ≤ f 2`
`h 2`	`f 2 ≤ f (2 + 1)`
result	`f 0 ≤ f 3` ✓

Nat.le_trans : n ≤ m → m ≤ k → n ≤ k. Lean accepts f (0 + 1) where f 1 is needed because 0 + 1 reduces to 1 — they are definitionally equal.

> *An unnamed have is called this; this always refers to the most recent have, shadowing the previous this.*

CHAPTER 4 · §4.5 AND BEYOND

The proof language and further problems

- An unnamed **have** ⇒ the result is called **this**
- **by assumption** finds a hypothesis matching the goal
- **‹p›** (typed \f< \f>) is show p by assumption — refers to a hypothesis by its type, data too: ‹Nat›

this and ‹p› in one proof (illustration)

```
example (p q : Prop) (hp : p) (hq : q) :
    p ∧ q :=
  have : q := hq
  ⟨‹p›, this⟩
```

Further problems

1. ∀ distributes over ∧, → and ∨ (one direction for ∨ cannot be proved)
2. Barber paradox: from ∀ x, shaves barber x ↔ ¬ shaves x x infer False
3. Definitions in Prop: even, prime, infinitely_many_primes, Fermat_prime, Goldbach, Fermat's last theorem
4. ∃ identities: (∃ x, p x) ↔ ¬ (∀ x, ¬ p x), (¬ ∀ x, p x) ↔ (∃ x, ¬ p x), …

Some directions need open Classical and an element of α (a : α).

Errors with quantifiers and equality

An equation the wrong way round (illustration, deliberately wrong)

```
example (d e : Nat) (h4 : e = 1 + d) :
    1 + d = e :=
  h4
```

```
error: Type mismatch
  h4
has type
  e = 1 + d
but is expected to have type
  1 + d = e
```

Compare the two types: same two sides, swapped. Fix: h4.symm (Eq.symm h4).

Symptom	Cause · fix
Using h : ∀ x, p x where p 0 is needed: Type mismatch, has type ∀ (x : Nat), p x	∀ is a function: apply it to a value — write h 0
rfl cannot close 0 + a = a	computation gets stuck on the variable a; use the lemma Nat.zero_add a
Opening h : ∃ x, p x to return x as a value (of type α)	∃ can be opened only while proving a Prop; for data use Σ or Classical.choose (ch. 12)
Eq.subst / ▸ cannot find the motive	state the expected type with show, or use rw
rw cannot find the pattern in the goal	the equation's left-hand side does not occur; try ← to rewrite the other way

CHAPTER 4 · CHAPTER SUMMARY

Chapter 4 summary: quantifiers and equality

∀ is a dependent function: built with fun, used by application. Prop is impredicative thanks to imax.

= has only refl; every other property comes from substitution.

calc chains =, <, ≤ … via transitivity (Trans).

∃ is a pair of witness + evidence; it can be opened only when proving a Prop.

Syntax	Meaning	Syntax	Meaning
`fun x : α ⇒ …`	prove ∀	`calc a = b := … _ < c := …`	chain of calculations
`h t`	use ∀ at t	`by rw [h] / by simp [h]`	rewrite / simplify
`rfl, Eq.symm, Eq.trans`	reflexivity, symmetry, transitivity	`⟨w, h⟩, Exists.intro`	build ∃
`h ▸ e, Eq.subst`	substitute equals	`match h with | ⟨w, hw⟩ ⇒`	open ∃ (also: let, Exists.elim)
`congrArg, congrFun, congr`	replace argument / function	`have : … / this / ‹p›`	anonymous results

CHAPTER 4 · QUESTIONS TO REFLECT ON

Questions to reflect on for Chapter 4

1. Why can p → q be seen as a special case of ∀? Write it in ∀ form.

2. rfl proves 2 + 3 = 5 and x + 0 = x, but not 0 + x = x. Why?

3. Inferring ∃ x, p x from ¬∀ x, ¬p x needs classical logic, but inferring ¬∀ x, ¬p x from ∃ x, p x does not. Why?

4. The type ∀ p : Prop, p → p lives in Prop, while (α : Type) → α → α lives in Type 1. What does this difference say about the role of Prop?

Suggested answers to the Chapter 4 questions

1. p → q is just ∀ _ : p, q: a function taking a proof of p, where q does not depend on that proof. Just as α → β is (a : α) → β when β does not mention a.

2. Nat.add recurses on its second argument. 2 + 3 has only constants, so it computes all the way to 5; x + 0 matches the rule "n + 0 = n" directly. But 0 + x needs to know whether x is 0 or k + 1 — with a variable x, computation gets stuck; it needs a proof by induction (Nat.zero_add).

3. Easy direction: given ⟨w, hw⟩ and h : ∀ x, ¬p x, h w hw : False. Hard direction: knowing "not every x fails p" does not hand us a concrete x; constructive ∃ demands a witness, so byContradiction is needed.

4. Prop is the world of propositions: quantifying over all propositions still yields a proposition (impredicative). Data must be stratified to avoid paradoxes; propositions are allowed this, at the cost of not extracting data from proofs.

> *So far every proof has been a hand-written term. Chapter 5 introduces tactics — a way to build these very terms goal by goal.*

PART II · CHAPTER

5

Tactics

Writing proofs interactively

SECTIONS

5.1 Entering Tactic Mode

5.2 Basic Tactics

5.3 More Tactics

5.4 Structuring Tactic Proofs

5.5 Tactic Combinators

5.6 Rewriting

5.7 Using the Simplifier

5.8 Split Tactic

5.9 Extensible Tactics

5.10 Further problems

CHAPTER 5 · INTUITION FOR NEWCOMERS

Write the answer outright, or converse with Lean?

Term mode — hand in the full solution

Swapping the two sides of ∧ (illustration)

```
example (p q : Prop) : p ∧ q → q ∧ p :=
  fun h ⟹ ⟨h.2, h.1⟩
```

- We build the whole term ourselves and give it to Lean; Lean only **checks**: does the term have type p ∧ q → q ∧ p?
- Like writing a complete solution on paper: concise, but we must track everything we have and still need.

Tactic mode — a step-by-step dialogue

The same proposition, written with tactics (illustration)

```
example (p q : Prop) : p ∧ q → q ∧ p := by
  intro h      -- Lean: now ⊢ q ∧ p
  constructor -- Lean: ⊢ q and ⊢ p
  · exact h.2
  · exact h.1
```

- After each command, Lean replies: “here is what remains to prove”. We only think about **one step** at a time.

> *Both yield the same kind of object: a term whose type is a proposition. Tactics are not “a different logic” — just an assistant that helps build the term; the unfinished part shows up as a goal ⊢.*

CHAPTER 5 · BACKGROUND

Reading a goal state

Lean's display after cases h, with h : p ∨ q

```
case inl
p q : Prop
h† : p
⊢ q ∨ p

case inr
p q : Prop
h† : q
⊢ q ∨ p
```

Component	Read as
`case inl`	The goal's tag — usually the name of the constructor that produced it
`p q : Prop`	Context: "let p, q be two propositions"
`h† : p`	Hypothesis: "we already have a proof of p". The `†` mark: a name Lean chose, which cannot be typed back
`⊢ q ∨ p`	Goal: "we still have to build a term of type q ∨ p"
Two blocks	Two independent goals; the next tactic acts on the **first** block

> *Read each block as a sentence: "Given propositions p, q and assuming p, prove q ∨ p." Everything above ⊢ is what we may use; the ⊢ line is what we must produce.*

When no blocks remain, Lean reports **No goals**: the proof is done. Read x : T via Curry–Howard: x is an element of T, i.e. a proof of T when T is a proposition.

Entering tactic mode: goals, apply and exact

Example — each · solves one subgoal

```
theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by
  apply And.intro
  · exact hp
  · apply And.intro
    · exact hq
    · exact hp
```

Initial goal state

```
p q : Prop
hp : p
hq : q
⊢ p ∧ q ∧ p
```

by can replace a term anywhere; tactics are separated by newlines or ;

apply e: matches the conclusion of e with the goal; missing arguments become subgoals. **exact e:** must close the goal entirely.

case tag => or a · to focus; the result is still a proof term (#print test)

The goal state at each step of test

```
theorem test (p q : Prop) (hp : p) (hq : q) :
    p ∧ q ∧ p := by
  apply And.intro
  exact hp
  apply And.intro
  exact hq
  exact hp
```

- **apply And.intro:** And.intro has type a → b → a ∧ b. Matching a ∧ b with p ∧ (q ∧ p) gives a := p, b := q ∧ p; the two missing arguments become two subgoals.
- The tags **left**, **right** come from the parameter names of And.intro.
- **exact** always acts on the goal at the head of the list; the other goals wait in order.
- Every goal carries the context p q : Prop, hp : p, hq : q (omitted in the boxes below).

After apply And.intro

```
case left
⊢ p
case right
⊢ q ∧ p
```

After exact hp

```
case right
⊢ q ∧ p
```

After apply And.intro (2nd time)

```
case left
⊢ q
case right
⊢ p
```

After exact hq

```
case right
⊢ p
```

final exact hp: no goals left

Why tactics still produce a proof term

Short form: tactics also accept compound expressions

```
theorem test (p q : Prop) (hp : p) (hq : q) :
    p ∧ q ∧ p := by
  apply And.intro hp
  exact And.intro hq hp

#print test
```

#print test: type and term (written explicitly; Lean displays And.intro with the ⟨...⟩ notation)

```
theorem test :
  ∀ (p q : Prop), p → q → p ∧ q ∧ p :=
fun p q hp hq ⟹ And.intro hp (And.intro hq hp)
```

Step	Term under construction (?right: open goal)
Start	`?g : p ∧ q ∧ p`
`apply And.intro hp`	`And.intro hp ?right`
`exact And.intro hq hp`	`And.intro hp (And.intro hq hp)`

> *Curry–Howard: the goal is a type; each tactic fills in part of a term of that type, and the unfilled part is a metavariable. The kernel checks only the final term — tactics add no logical power.*

The parameters (p q : Prop) (hp : p) (hq : q) become fun p q hp hq => ...; the five-step version on the previous slide produces exactly this term.

“unsolved goals”: the by block ends before the work is done

Forgetting the second goal (illustration)

```
example (p q : Prop) (hp : p) (hq : q) :
    p ∧ q := by
  apply And.intro
  exact hp
```

Lean underlines by in red and prints

```
error: unsolved goals
case right
p q : Prop
hp : p
hq : q
⊢ q
```

- **Read:** what follows “unsolved goals” is the **exact** list of open goals, each with its context — the same format as the goal state.
- **case right:** apply And.intro creates two goals, left and right; exact hp closes only left.
- **Fix:** add exact hq. The error is reported at by (or at the end of a · / case block), not at the missing line.

Related message	Meaning and fix
`No goals to be solved`	An extra tactic: all goals are closed (e.g. exact hp written twice) — delete the extra line
`Type mismatch hp has type p but is expected to have type q`	(Printed over several lines.) exact hp on goal ⊢ q: the term is well-formed but has the wrong type — check the goal at the head

CHAPTER 5 · INTUITION FOR NEWCOMERS

Reason backward from the goal, forward from hypotheses

Backward: apply turns the goal into "easier" goals (illustration)

```
example (p q r : Prop) :
    (p → q) → (q → r) → p → r := by
  intro hpq hqr hp
  apply hqr
  apply hpq
  exact hp
```

Forward: have builds new facts from hypotheses

```
  intro hpq hqr hp
  have hq : q := hpq hp
  exact hqr hq
```

Goal after each step (context: hpq : p → q, hqr : q → r, hp : p)

```
⊢ r   after intro: "need r"
⊢ q   after apply hqr: "q gives r"
⊢ p   after apply hpq: "p gives q"
exact hp: done
```

- **Backward:** look at the conclusion to reach, find a hypothesis/lemma whose **conclusion matches**, apply it — the goal is replaced by its premises.
- **Forward:** from what we have, derive new facts (have) until we reach the goal.
- The question to ask: "what **shape** does the goal have?" — the shape suggests a tactic (table on the slide after §5.3).

> *The first version builds the term fun hpq hqr hp => hqr (hpq hp); the have version builds the same content, just naming the middle step hq. Reading the term outside-in is backward reasoning; inside-out is forward.*

Basic tactics

Tactic	Effect
`intro`	Moves variables and hypotheses into the context; accepts patterns ⟨w, hpw, hqw⟩
`intros`	Introduces everything, with generated names (access via rename_i)
`assumption`	Closes the goal with a matching hypothesis
`rfl`	Closes a goal whose sides are definitionally equal
`repeat t`	Repeats t until it fails
`revert h`	Inverse of intro: moves h back into the goal
`generalize e = x`	Replaces the expression e with a new variable x

Example — intro with a pattern destructuring ∃ and ∧

```
example (p q : α → Prop) :
    (∃ x, p x ∧ q x) → ∃ x, q x ∧ p x := by
  intro ⟨w, hpw, hqw⟩
  exact ⟨w, hqw, hpw⟩
```

Compare with §4.4: the same idea, written step by step.

intro, intros and rename_i: naming hypotheses

```
example : ∀ a b c : Nat, a = b → a = c → c = b := by
  intro a b c h₁ h₂
  exact Eq.trans (Eq.symm h₂) h₁
```

```
example : ∀ a b c d : Nat,
    a = b → a = d → a = c → c = b := by
  intros
  rename_i h1 _ h2
  apply Eq.trans
  apply Eq.symm
  exact h2
  exact h1
```

Example 1 · after intro a b c h_1 h_2

```
a b c : Nat
h₁ : a = b
h₂ : a = c
⊢ c = b
```

Example 2 · after rename_i h1 _ h2 († dropped)

```
h1 : a = b
a† : a = d
h2 : a = c
⊢ c = b
```

intros gives “hygienic” names (marked †) that cannot be typed back ⇒ proofs never depend on generated names. **rename_i** names the most recent inaccessible names; _ skips one.

revert and generalize: moving back into the goal

```
example (x y : Nat) (h : x = y) : y = x := by
  revert h
  intro h₁
  apply Eq.symm
  assumption
```

After revert h

```
x y : Nat
⊢ x = y → y = x
```

After intro h_1

```
h₁ : x = y
⊢ y = x
```

revert h is the inverse of intro: the hypothesis h : x = y returns to the goal as the premise of an implication. **revert x** also takes along everything depending on x (here h): ⊢ ∀ (x : Nat), x = y → y = x.

```
example : 2 + 3 = 5 := by
  generalize h : 3 = x
  rw [← h]
```

After generalize

```
x : Nat
h : 3 = x
⊢ 2 + x = 5
```

generalize replaces the expression 3 with a new variable x. Without the label h, the goal ⊢ 2 + x = 5 is false for arbitrary x! The label h records 3 = x, so rw [← h] brings 3 back and rfl closes the goal.

> *Only context entries (variables, hypotheses) can be reverted; to abstract an arbitrary expression in the goal, use generalize — and take care to preserve truth.*

cases, constructor, exists, contradiction

Splitting ∨ into two cases

```
example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h with
  | inl hp ⇒ apply Or.inr; exact hp
  | inr hq ⇒ apply Or.inl; exact hq
```

Building ∧ with constructor (illustration)

```
example (p q : Prop) : p ∧ q → q ∧ p := by
  intro ⟨hp, hq⟩
  constructor
  · exact hq
  · exact hp
```

- **cases** splits ∨, ∧, ∃ and data too: Nat into zero and succ m′
- Structured form cases h with | inl … | inr …; or cases h followed by case / ·
- **constructor:** applies the first matching constructor (splits ∧, ↔)
- **exists x:** supplies a witness for ∃
- **contradiction:** closes the goal when the hypotheses are inconsistent
- **match** works inside tactic blocks; **induction** is the “sibling” of cases — details in Chapter 7

cases on ∨ and ∧: the goals after splitting

```
example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h
  . apply Or.inr
    assumption
  . apply Or.inl
    assumption
```

After cases h

```
case inl
h✝ : p
⊢ q ∨ p
case inr
h✝ : q
⊢ q ∨ p
```

cases h splits h : p ∨ q along the two constructors Or.inl, Or.inr ⇒ two goals inl, inr, both concluding q ∨ p. The new hypothesis gets the inaccessible name h✝ — assumption still finds it. In essence: Or.elim, step by step.

```
example (p q : Prop) : p ∧ q → q ∧ p := by
  intro h
  cases h with
  | intro hp hq ⇒ constructor; exact hq; exact hp
```

At | intro hp hq =>

```
case intro
hp : p
hq : q
⊢ q ∧ p
```

And has only one constructor, And.intro ⇒ one goal **intro**, with hp, hq in place of h. Then **constructor** (applying And.intro) produces left ⊢ q and right ⊢ p.

cases on ∃ and Nat: witnesses and metavariables

```
example (p q : Nat → Prop) :
    (∃ x, p x) → ∃ x, p x ∨ q x := by
  intro h
  cases h with
  | intro x px ⇒
    constructor; apply Or.inl; exact px
```

After constructor (context: x : Nat, px : p x)

```
case h
⊢ p ?w ∨ q ?w
case w
⊢ Nat
```

exact px unifies p ?w with p x ⇒ assigns ?w := x, and goal w closes by itself. Write **exists x** instead of constructor to state the witness up front.

```
open Nat
example (P : Nat → Prop)
    (h₀ : P 0) (h₁ : ∀ n, P (succ n))
    (m : Nat) : P m := by
  cases m with
  | zero    ⇒ exact h₀
  | succ m' ⇒ exact h₁ m'
```

```
case zero
⊢ P 0
```

```
case succ
m' : Nat
⊢ P (m' + 1)
```

cases works on data too. Lean displays succ m' as m' + 1; h_1 m' : P (succ m') still matches because the two forms are definitionally equal.

CHAPTER 5 · PROOF STRATEGY

Read the shape, pick the tactic: a strategy table

Goal of the form	Try
`⊢ p → q, ⊢ ∀ x, P x`	intro h, intro x
`⊢ ¬p`	intro h — since ¬p is just p → False
`⊢ p ∧ q, ⊢ p ↔ q`	constructor — splits into two goals
`⊢ p ∨ q`	apply Or.inl / apply Or.inr — pick a side
`⊢ ∃ x, P x`	exists t, or exact ⟨t, h⟩
`⊢ a = b`	rfl, rw [h], simp
`⊢ T, with h : T`	assumption, exact h
`⊢ anything, absurd context`	contradiction

Hypothesis of the form	Try
`h : p ∧ q`	cases h, or intro ⟨hp, hq⟩ from the start
`h : p ∨ q`	cases h — two branches inl, inr
`h : ∃ x, P x`	cases h with | intro x hx
`h : a = b`	rw [h], rw [h] at h', simp [h]
`h : p → q, ⊢ q`	apply h — new goal ⊢ p
`h : ∀ x, P x`	have h' := h t, exact h t
`h : False`	contradiction
`n : Nat, xs : List α`	cases n, induction n (Chapter 7)

> *The basic loop: (1) read the shape of the goal; (2) "open" it with intro, constructor; (3) "open" hypotheses with cases; (4) when stuck, find a hypothesis with a matching conclusion to apply; (5) for equalities: rw, simp.*

CHAPTER 5 · §5.4–5.5

Structuring proofs and combining tactics

```
example (p q r : Prop) (hp : p) (hq : q) (hr : r) : p ∧ ((p ∧ q) ∧ r) ∧ (q ∧ r ∧ p) := by
  repeat (any_goals constructor)
  all_goals assumption
```

Structure (§5.4)

- **show T:** states (or definitionally restates) the goal
- **have h : T := e:** adds a fact; omit the name ⇒ this
- **let a := …:** a local definition that can be unfolded later
- Blocks { … } and ; instead of indentation; ‹q› refers to a hypothesis by type
- Mix freely: apply/exact take terms, by appears inside terms

Combinator (§5.5)	Meaning
`t₁; t₂`	Run in sequence
`t₁ <;> t₂`	t_2 on every goal t_1 produces
`first | t₁ | t₂`	Try each in turn until one succeeds
`try t`	Always succeeds (first | t | skip)
`all_goals / any_goals`	Every goal / at least one
`focus t`	Only the current goal

have, show and let in tactic mode

```
example (p q r : Prop) :
    p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := by
  intro ⟨hp, hqr⟩
  cases hqr with
  | inl hq ⇒
    have := And.intro hp hq
    apply Or.inl; exact this
  | inr hr ⇒
    have := And.intro hp hr
    apply Or.inr; exact this
```

have h : T := e adds a fact to the context; omit the name ⇒ this; omit the type too ⇒ Lean infers it. **show T** states the goal; T need only be definitionally equal to it (n + 1 and Nat.succ n). **let** keeps the value a := 3 * 2, so exists a closes a + 2 = 8 by rfl; with have, the value is “forgotten”.

Branch inl, after have := And.intro hp hq

```
hp : p
hq : q
this : p ∧ q
⊢ p ∧ q ∨ p ∧ r
```

```
example (n : Nat) : n + 1 = Nat.succ n := by
  show Nat.succ n = Nat.succ n
  rfl

example : ∃ x, x + 2 = 8 := by
  let a : Nat := 3 * 2
  exists a
```

Lean prints the goal without parentheses because ∧ binds tighter than ∨.

Tactic combinators step by step

One line for p ∧ q ∧ r (recall: ∧ associates to the right)

```
example (p q r : Prop) (hp : p) (hq : q)
    (hr : r) : p ∧ q ∧ r := by
  constructor <;> (try constructor) <;> assumption
```

> *t_1 <;> t_2 runs t_2 on every goal produced by t_1 — a "parallel" version of ;.*

After constructor

```
⊢ p        ⊢ q ∧ r
```

After <;> (try constructor): fails on p, but try still succeeds

```
⊢ p        ⊢ q        ⊢ r
```

After <;> assumption: all three closed by hp, hq, hr

first | t_1 | t_2: with hq : q, the branch apply Or.inl; assumption fails (no hypothesis for ⊢ p) ⇒ backtrack, try apply Or.inr; assumption — success.

try t = first | t | skip: never fails. Consequence: repeat (try t) loops forever, while repeat t stops when t fails.

all_goals t fails if t fails on any goal; **any_goals t** need only succeed on one ⇒ any_goals constructor can replace all_goals (try constructor).

CHAPTER 5 · WORKED EXAMPLE

One proof, two styles (1): term mode

Proposition: if some x has p x ∨ q x, then some x has p x, or some x has q x. Idea: that same x is the witness for the true side.

Writing the term directly (illustration)

```
example (p q : Nat → Prop) :
  (∃ x, p x ∨ q x) →
    (∃ x, p x) ∨ (∃ x, q x) :=
  fun ⟨x, hx⟩ ⇒
    match hx with
    | Or.inl hp ⇒ Or.inl ⟨x, hp⟩
    | Or.inr hq ⇒ Or.inr ⟨x, hq⟩
```

Fragment	Meaning
`fun ⟨x, hx⟩ ⇒`	Takes the proof of ∃ and destructures it at once: x : Nat, hx : p x ∨ q x
`match hx with`	Two cases, one per constructor of ∨
`⟨x, hp⟩`	Exists.intro x hp : ∃ x, p x — Lean knows the expected type, so it understands ⟨…⟩
`Or.inl ⟨x, hp⟩`	Puts that proof on the left side of ∨

> *To write this term, we must keep the type of every hole in our head. The next slide rewrites the same proof with tactics, so that Lean displays those holes.*

CHAPTER 5 · WORKED EXAMPLE

One proof, two styles (2): tactic mode

```
example (p q : Nat → Prop) :
  (∃ x, p x ∨ q x) →
    (∃ x, p x) ∨ (∃ x, q x) := by
  intro ⟨x, hx⟩
  cases hx with
  | inl hp ⇒ exact Or.inl ⟨x, hp⟩
  | inr hq ⇒ apply Or.inr; exact ⟨x, hq⟩
```

Tactic	Part of the term it builds
`intro ⟨x, hx⟩`	`fun ⟨x, hx⟩ ⇒ ?_`
`cases hx with …`	`match hx with …`
`exact Or.inl ⟨x, hp⟩`	`Or.inl ⟨x, hp⟩`
`apply Or.inr`	`Or.inr ?_`
`exact ⟨x, hq⟩`	fills `?_` with ⟨x, hq⟩

After intro ⟨x, hx⟩

```
x : Nat
hx : p x ∨ q x
⊢ (∃ x, p x) ∨ ∃ x, q x
```

Branch inl (before exact)

```
x : Nat
hp : p x
⊢ (∃ x, p x) ∨ ∃ x, q x
```

Branch inr, after apply Or.inr

```
x : Nat
hq : q x
⊢ ∃ x, q x
```

The context p q : Nat → Prop is present in every box (omitted). Lean prints (∃ x, p x) ∨ ∃ x, q x: the right-hand parentheses are unneeded because ∃ extends to the end of the expression. Both versions give the same proof; tactics differ only in that Lean **shows us** each open ?_.

Rewriting with rw

- **rw [t]**: replaces the left side of the equation t with its right side; t can be a hypothesis, a lemma or a compound term (h hq)
- **rw [t_1, ..., t_n]**: a chain of rewrites; **rw [← t]**: the reverse direction (type \l)
- Rewrites the *first* matching position; give arguments (Nat.add_comm b) to pick the position
- **rw [t] at h**: rewrites a hypothesis
- After rewriting, rw tries to close the goal with rfl
- Not only for propositions: with h : n = 0, t : Tuple α n becomes t : Tuple α 0

Example — controlling direction and position

```
example (a b c : Nat) :
    a + b + c = a + c + b := by
  rw [Nat.add_assoc, Nat.add_comm b,
     ← Nat.add_assoc]
```

Goal after each step

```
⊢ a + (b + c) = a + c + b
⊢ a + (c + b) = a + c + b
⊢ a + c + b = a + c + b — closed by rfl
```

rw step by step: direction and hypotheses

Shared context: variable (k a b : Nat) (f : Nat → Nat)

```
example (h₁ : f 0 = 0) (h₂ : k = 0) : f k = 0 := by
  rw [h₂] -- replace k with 0
  rw [h₁] -- replace f 0 with 0
```

```
⊢ f k = 0
⊢ f 0 = 0   after rw [h₂]
⊢ 0 = 0     after rw [h₁], rfl closes
```

```
example (h₁ : a = b) (h₂ : f a = 0) : f b = 0 := by
  rw [←h₁, h₂]
```

```
⊢ f b = 0
⊢ f a = 0   ←h₁: b becomes a
⊢ 0 = 0     h₂, then rfl closes
```

```
example (f : Nat → Nat) (a : Nat) (h : a + 0 = 0) :
    f a = f 0 := by
  rw [Nat.add_zero] at h
  rw [h]
```

```
h : a + 0 = 0
⊢ f a = f 0
h : a = 0   after … at h
⊢ f 0 = f 0 after rw [h], rfl closes
```

Explicit arguments pick the position: for a + (b + c) = a + (c + b), Nat.add_comm _ b matches only ?x + b, so only c + b is swapped.

When rw fails: pattern not found, ill-typed motive

Rewriting in the wrong direction (illustration)

```
example (f : Nat → Nat) (a b : Nat)
    (h₁ : a = b) (h₂ : f a = 0) :
    f b = 0 := by
  rw [h₁]
```

```
Tactic `rewrite` failed: Did not find
an occurrence of the pattern
  a
in the target expression
  f b = 0
```

Read: rw [h_1] looks for the left side a to replace with b, but the goal contains only b. **Fix:** reverse the direction — rw [← h_1, h_2].

Rewriting a variable another type depends on (illustration)

```
example (n : Nat) (h : n = 0) (v : Fin n)
    (P : (k : Nat) → Fin k → Prop) :
    P n v := by
  rw [h]
```

```
Tactic `rewrite` failed:
motive is not type correct:
  fun _a ⟹ P _a v
Error: Application type mismatch: …
v has type Fin n … expected Fin _a
```

Read: rw replaces every n with a variable _a to build the **motive** — the "template" of the goal (Chapter 7); but v still has type Fin n, so P _a v is ill-typed. **Fix:** subst h — replaces n := 0 everywhere, including in the type of v.

CHAPTER 5 · §5.7–5.8

The simplifier simp and the split tactic

Variant	Meaning
`simp`	Repeated rewriting with @[simp] lemmas
`simp [h, ←h', f]`	Adds hypotheses, lemmas, definitions to unfold
`simp [*]`	Uses every hypothesis in the context
`simp at h / at *`	Simplifies a hypothesis / everything
`simp only [...]`	Uses only the given list
`simp [-name]`	Removes a lemma
`+contextual, +arith`	Uses hypotheses per branch; arithmetic

```
example (x y z : Nat) :
    (x + 0) * (0 + y * 1 + z * 0)
    = x * y := by
  simp
```

Ordered rewriting: permutation (commutativity) lemmas apply only when the result is “smaller” in an internal order — no loops, a normal form.

split (§5.8): splits if-then-else and match into cases; split at h; commonly split <;> first | contradiction | rfl.

simp by example: rule lists, [*] and at *

```
def f (m n : Nat) : Nat :=
  m + n + m

example {m n : Nat} (h : n = 1) (h' : 0 = m)
        : (f m n) = n := by
  simp [h, ←h', f]
```

```
⊢ f m n = n  ⇒  m + n + m = n
⇒  0 + 1 + 0 = 1  ⇒  True
```

```
example (p q : Prop) (hp : p) : p ∧ q ↔ q := by
  simp [*]
```

```
p ∧ q ↔ q ⇒ True ∧ q ↔ q ⇒ q ↔ q ⇒ True
```

```
example (u w x x' y y' z : Nat) (p : Nat → Prop)
        (h₁ : x + 0 = x') (h₂ : y + 0 = y')
        : x + y + 0 = x' + y' := by
  simp at *
  simp [*]
```

*After simp at **

```
h₁ : x = x'
h₂ : y = y'
⊢ x + y = x' + y'
```

simp at * simplifies all hypotheses and the goal; **simp [*]** uses every hypothesis as a rule: x becomes x', y becomes y', then rfl closes it. The rule list accepts hypotheses (h), reversed ones (←h') and names of definitions to unfold (f).

Ordered rewriting, @[simp] and simp only

```
attribute [local simp] Nat.mul_comm Nat.mul_assoc Nat.mul_left_comm
attribute [local simp] Nat.add_assoc Nat.add_comm Nat.add_left_comm

example (w x y z : Nat) (p : Nat → Prop)
        : x * y + z * w * x = x * w * z + y * x := by
  simp
```

- Commutativity x * y = y * x could loop forever; simp recognizes permutation rules and applies them only when the term decreases in an internal order (*ordered rewriting*).
- comm + assoc + left_comm: parentheses move right, terms are sorted canonically ⇒ two sides equal modulo AC reach the same form.
- **@[simp]** adds reverse_mk_symm : (mk_symm xs).reverse = mk_symm xs to the default rule set; **[-name]** removes a rule; **only** uses just the given list.

The hypothesis h : p (xs ++ mk_symm ys).reverse becomes

Command	h after running
`simp at h`	`p (mk_symm ys ++ xs.reverse)`
`simp [-reverse_mk_symm] at h`	`p ((mk_symm ys).reverse ++ xs.reverse)`
`simp only [List.reverse_append] at h`	`p ((mk_symm ys).reverse ++ xs.reverse)`

split and contradiction by example

```
def f (x y z : Nat) : Nat :=
  match x, y, z with
  | 5, _, _ ⇒ y
  | _, 5, _ ⇒ y
  | _, _, 5 ⇒ y
  | _, _, _ ⇒ 1
```

```
example (p q : Prop) : p ∧ ¬ p → q := by
  intro h
  cases h
  contradiction
```

contradiction closes any goal when the context contains False, a pair P and ¬P, or an absurd hypothesis such as a ≠ a. Here, after cases h we have left✝ : p and right✝ : ¬p.

```
example (x y z : Nat) :
  x ≠ 5 → y ≠ 5 → z ≠ 5 → z = w →
  f x y w = 1 := by
  intros
  simp [f]
  split
  . contradiction
  . contradiction
  . contradiction
  . rfl
```

split creates one goal per match branch. First three: x, y or w matches 5, so x ≠ 5, y ≠ 5, or z ≠ 5 with z = w is contradictory ⇒ contradiction. Last: ⊢ 1 = 1, closed by rfl. Shorter: split <;> first | contradiction | rfl.

CHAPTER 5 · §5.9–5.10

Extensible tactics

Example — defining a new tactic "triv"

```
syntax "triv" : tactic
macro_rules
  | `(tactic| triv) ⟹ `(tactic| assumption)
example (h : p) : p := by
  triv
```

- **syntax** declares new tactic syntax; **macro_rules** defines how it expands
- Add macro_rules later: Lean tries them in turn until one succeeds
- Expansions can be recursive: apply And.intro <;> triv

Further problems (§5.10)

1. Rewrite the proofs from Chapters 3 and 4 with tactics, using rw and simp where appropriate
2. Write a one-line proof using tactic combinators for the following goal, with hp : p

`(p ∨ q ∨ r) ∧ (q ∨ p ∨ r) ∧ (q ∨ r ∨ p)`

Hint: combine repeat, constructor and first with apply Or.inl, apply Or.inr, assumption.

Extending triv with several macro_rules

```
macro_rules
  | `(tactic| triv) ⇒ `(tactic| rfl)

example (x : α) : x = x := by
  triv

example (x : α) (h : p) : x = x ∧ p := by
  apply And.intro <;> triv

macro_rules | `(tactic| triv) ⇒
    `(tactic| apply And.intro <;> triv)

example (x : α) (h : p) : x = x ∧ p := by
  triv
```

Each expansion of triv on ⊢ x = x ∧ p

```
assumption           fails
rfl                  fails
apply And.intro <;> triv succeeds:
  ⊢ x = x  only rfl succeeds
  ⊢ p      only assumption succeeds
```

- **syntax** "triv" : tactic adds a new tactic keyword; **macro_rules** says what it expands to.
- Each macro_rules adds an alternative; the interpreter tries the expansions until one succeeds.
- The third rule is recursive: triv calls triv again on smaller subgoals.
- Lean's tactics are written in Lean itself; macros are the lightest way to extend them.

CHAPTER 5 · QUICK REFERENCE

Reference table of the main tactics

Tactic	Purpose
`by`	Switch to tactic mode
`apply e`	Match the conclusion, create subgoals
`exact e`	Close the goal with exactly e
`intro, intros`	Move hypotheses into the context
`assumption`	Use a matching hypothesis
`rfl`	Definitional equality
`revert, generalize`	Move back / generalize
`cases`	Case analysis by constructor
`constructor`	Apply a matching constructor
`exists x`	Witness for ∃

Tactic	Purpose
`contradiction`	Inconsistent hypotheses
`show, have, let`	Goal / fact / definition
`case, ·, { }`	Focus on a subgoal
`; and <;>`	Sequence / on every goal
`first, try, repeat`	Choice / tolerate failure / loop
`all_goals, any_goals`	Every goal / at least one
`rw [t, ← t'] at h`	Rewrite with equations
`simp …`	Simplify
`split`	Split if / match
`syntax, macro_rules`	Define new tactics

CHAPTER 5 · CHAPTER SUMMARY

Tactics: the key ideas to take away

1. **by** opens tactic mode anywhere a term is needed. Tactics build a proof term incrementally; the kernel checks only the final term.
2. **The state** is a list of goals; each goal has a context (above) and a conclusion (after ⊢). Tactics act on the first goal.
3. **Open the goal:** intro, constructor, exists. **Open hypotheses:** cases. **Match the conclusion:** apply, exact.
4. **Structure:** have, show, let; case and ·; combinators ;, <;>, first, try, repeat, all_goals.
5. **Equality:** rw rewrites directionally at the first match; simp rewrites repeatedly with the @[simp] set; split splits if and match.
6. **Extension:** syntax and macro_rules define new tactics in Lean itself.

Key syntax	Meaning
`:= by tac₁; tac₂`	A tactic block in place of a term
`· tac / case tag ⇒ tac`	Focus on one subgoal
`cases h with | c x ⇒ …`	Split by constructor
`have h : T := e`	Forward reasoning
`apply e`	Backward reasoning
`rw [h, ← h'] at h''`	Rewrite in a hypothesis
`simp [*] at *`	Simplify everywhere
`t₁ <;> t₂`	t_2 on every subgoal

CHAPTER 5 · QUESTIONS TO REFLECT ON

Four questions to check understanding

Question 1

Can a tactic proof be “stronger” or “less safe” than a proof written as a term? Why?

Question 2

How do apply h and exact h differ? With h : p → q → r and goal ⊢ r, which goals does apply h leave?

Question 3

Why can repeat (try t) run forever, while repeat t always stops (when t itself does not loop)?

Question 4

simp can use the commutativity rule a + b = b + a without looping. What about repeating rw [Nat.add_comm]? Which mechanism differs?

Suggested answers to the four questions

Question 1 — No

A tactic is just a program that produces a proof term; #print shows that term. The kernel checks the final term like any other, so a faulty tactic can only **fail to produce** a term, never produce a wrong one. Exceptions are sorry or user-added axioms — they always show up in #print axioms (e.g. sorryAx).

Question 2 — exact must close the goal entirely

exact h requires the type of h to **equal** the goal. apply h matches the **conclusion** r of h with the goal, and each missing premise becomes a new goal: ⊢ p and ⊢ q remain, in that order.

Question 3 — try never fails

repeat t stops when t fails. try t = first | t | skip always succeeds (even when it does nothing), so the stopping condition never occurs.

Question 4 — ordered rewriting

rw performs exactly one rewrite per command, in the direction we specify; repeating it swaps a + b ↔ b + a forever. simp recognizes permutation rules and applies them only when the result is “smaller” in an internal order, so each expression has a unique normal form.

PART II · CHAPTER

6

Interacting with Lean

Files, namespaces, notation and options

SECTIONS

6.1 Messages

6.2 Importing Files

6.3 More on Sections

6.4 More on Namespaces

6.5 Attributes

6.6 More on Implicit Arguments

6.7 Notation

6.8 Coercions

6.9 Displaying Information

6.10 Setting Options

6.11 Using the Library

6.12 Auto Bound Implicit Arguments

6.13 Implicit Lambdas

6.14 Sugar for Simple Functions

6.15 Named Arguments

A Lean file is a sequence of commands, read top-down

Lean processes commands in order. Each def, theorem, notation… **adds** an entry to the **environment**; a command below sees only what exists above it.

Full names and short names (illustration)

```
namespace Shapes
def double (n : Nat) : Nat := n + n
#eval double 3        -- 6
end Shapes

#eval Shapes.double 3 -- 6
open Shapes in
#eval double 4        -- 8
#check double         -- error: Unknown identifier
```

Concept	Picture it as
`import`	Brings a whole other file (and everything it imports) into the environment
`namespace`	A name's "surname": double in Shapes has full name Shapes.double
`open`	Lets us use first names (dropping the surname) within a scope
`section`	A temporary "bracket": variable, open, local inside expire at end
`variable`	A "shared assumption" for the whole block, added to any definition using it
`@[simp]`	A label on a theorem so a tool (here simp) uses it automatically

CHAPTER 6 · §6.1–6.3

Messages, imports and sections

6.1 Messages

Errors, warnings (e.g. sorry), information (#check, #eval). **#guard_msgs in** checks messages against the doc comment above; #guard_msgs (error) checks only errors.

6.2 Imports

Init is loaded automatically; import Bar.Baz.Blah at the top of the file. Imports are transitive, but open namespaces do not carry over to other files.

6.3 Sections

A variable is added only to definitions that actually use it. Sections also limit the scope of local attributes and notation.

Example — double takes only x, not y

```
section
variable (x y : Nat)

def double := x + x

theorem t1 :
    double (x + y) = double x + double y := by
  simp [double]
end
```

Three kinds of messages and #guard_msgs

Asserting in advance the type error Lean will report

```
/--
error: Type mismatch
  "Not a number"
has type
  String
but is expected to have type
  Nat
-/
#guard_msgs in
def x : Nat := "Not a number"
```

Kind	When it appears
Error	Code cannot be processed: a missing), adding a number to a function…
Warning	Code is meaningful but suspicious, e.g. using sorry
Information	Output of #check, #eval — not a problem

- **#guard_msgs in C** runs C and checks all messages against the doc comment /-- … -/ above. Match ⇒ messages are “swallowed”, errors included; mismatch ⇒ error.
- **#guard_msgs(error) in #eval (sorry : Nat)** checks only the error “Aborting evaluation…”; the warning declaration uses `sorry` still shows. Without (error), the doc comment needs both, split by a --- line.
- Use: testing docs and internal libraries — a changed message becomes an error.

variable in a section: only variables actually used are added

The rest of the section example (double, t1 seen above)

```
section
variable (x y : Nat)

def double := x + x

#check double y
#check double (2 * x)

theorem t2 : double (x * y) = double x * y := by
  simp [double, Nat.add_mul]

end
```

Name	Actual signature (after Lean adds variables)
`double`	`double (x : Nat) : Nat`
`t1`	`t1 (x y : Nat) : double (x + y) = double x + double y`
`t2`	`t2 (x y : Nat) : double (x * y) = double x * y`

- Lean checks which variables the type and body use: double uses only x, so it takes only x; y is not added.
- Variables are not applied automatically: inside the section, double is already a one-argument function, so double y and double (2 * x) both have type Nat.
- After end, x and y disappear; attribute [local simp] declared in the section also expires.

CHAPTER 6 · §6.4–6.5

Namespaces and attributes

Variants of open and export

```
open Nat (succ zero gcd)
open Nat hiding succ gcd
open Nat renaming mul → times, add → plus
#eval plus (times 2 2) 3 -- 7

export Nat (succ add sub)
```

Open only the listed names · open all but some names · open under new names · create aliases that propagate.

- Full names must be unique; short names may clash — Lean uses types to disambiguate; **_root_** denotes the root namespace
- **protected:** no short name is created on open (e.g. Nat.sub)
- **Attributes:** @[simp] at declaration, or attribute [simp] name later
- By default attributes, notation and instances apply wherever the file is imported; **local** limits them to the section/namespace/file
- **set_option** always applies only within the current section or file

Clashing aliases, _root_ and protected

```
def String.add (a b : String) : String :=
  a ++ b
def Bool.add (a b : Bool) : Bool :=
  a ≠ b
def add (α β : Type) : Type := Sum α β

open Bool
open String
```

```
protected def Foo.bar : Nat := 1
open Foo
/-- error: Unknown identifier `bar` -/
#guard_msgs in
#check bar
```

Command	Lean prints
`#check add "hello" "world"`	`"hello".add "world" : String`
`#check add true false`	`true.add false : Bool`
`#check add Nat Nat`	`_root_.add Nat Nat : Type`
`#check _root_.add`	`_root_.add (α β : Type) : Type`

- Full names must be unique, aliases need not be: add points to three functions, and Lean uses the argument types to choose; #check add alone is ambiguous.
- "hello".add "world" is the dot-notation display of String.add "hello" "world"; _root_ denotes the root namespace.
- **protected:** open Foo creates no alias bar; typically used for Nat.rec, Nat.recOn.

attribute local: the scope of [simp]

```
def isPrefix (l₁ : List α) (l₂ : List α) : Prop :=
  ∃ t, l₁ ++ t = l₂
```

```
section
theorem List.isPrefix_self (as : List α) :
    isPrefix as as :=
  ⟨[], by simp⟩
attribute [local simp] List.isPrefix_self
example : isPrefix [1, 2, 3] [1, 2, 3] := by
  simp
end
```

```
/-- error: `simp` made no progress -/
#guard_msgs in
example : isPrefix [1, 2, 3] [1, 2, 3] := by
  simp
```

- Inside the section, simp uses isPrefix_self to rewrite the goal to True.
- After end, the rule expires ⇒ simp made no progress.
- Without local (@[simp], attribute [simp]): the rule follows every import and cannot be removed.
- Likewise: attribute [local instance]; set_option is always local.

CHAPTER 6 · IN PRACTICE

Organizing a file in a real project

MyProject/Lists.lean (illustration) — part 1

```
-- import MyProject.Basic (always first)
/-! # Lists: reversal -/
universe u
namespace MyList
variable {α : Type u}
/-- Reverses a list. -/
def rev : List α → List α
  | []      ⇒ []
  | a :: as ⇒ rev as ++ [a]
```

- Order: import → module doc /-! … -/ → universe, namespace, variable → definitions (with docstrings /-- … -/).
- Other files use MyProject/Lists.lean via import MyProject.Lists.

Part 2 — lemmas, a section, closing the namespace

```
@[simp] theorem rev_nil :
    rev ([] : List α) = [] := rfl
@[simp] theorem rev_cons (a : α) (as : List α) :
    rev (a :: as) = rev as ++ [a] := rfl
section Lemmas
theorem rev_singleton (a : α) :
    rev [a] = [a] := by simp
end Lemmas
end MyList
```

- The defining equations get @[simp] right after the definition ⇒ simp proves rev_singleton by itself.
- Names follow content: rev_nil, rev_cons; full name MyList.rev_singleton. For narrow scope: attribute [local simp], open … in.

CHAPTER 6 · BACKGROUND

Elaboration: from what we write to the term the kernel checks

Source code
i + m, (· + 1), {α}

→

Parser
notation into function calls

→

Elaborator
fills in implicit arguments, instances, coercions, auto-bound

→

Kernel
checks the complete term

With m : Nat, i : Int, set_option pp.explicit true and pp.notation false, #check shows

```
-- written: i + m
@HAdd.hAdd Int Int Int (@instHAdd Int Int.instAdd) i (@Nat.cast Int instNatCastInt m)
-- written: fun (x : Nat) ⟹ x = x
fun x ⟹ @Eq Nat x x : Nat → Prop
-- written: def compose (g : β → γ) (f : α → β) (x : α) : γ := g (f x)
compose.{u_1, u_2, u_3} {β : Sort u_1} {γ : Sort u_2} {α : Sort u_3} …
```

> *Notation, implicit arguments, coercions (↑m is Nat.cast m), auto-bound, (· + 1) all exist only at the elaboration level — the kernel knows nothing of them. Facing a puzzling error: turn on pp.explicit to see the real term.*

CHAPTER 6 · §6.6–6.7

Kinds of arguments and user-defined notation

Syntax	Name	Behaviour
`(x : α)`	Explicit	Supplied by the user
`{x : α}`	Implicit	Filled in eagerly: t means @t _; @t turns this off
`⦃x : α⦄, {{x : α}}`	Strict implicit	Filled in only when an explicit argument follows
`[inst : C α]`	Instance implicit	Found by type class resolution (Chapter 10)

```
infixl:65   " + " ⇒ HAdd.hAdd -- left-assoc
infix:50    " = " ⇒ Eq        -- non-assoc
infixr:80   " ^ " ⇒ HPow.hPow -- right-assoc
prefix:100  "-"   ⇒ Neg.neg
postfix:max "⁻¹"  ⇒ Inv.inv
```

- All are shorthands for the general **notation** command
- max = 1024; infixl:65 gives the right operand precedence 66 ⇒ left-associative
- Remaining ambiguity is resolved during elaboration

Implicit { } and strict implicit {{ }}

Declaration	f 7 is read as	f 7 3 is read as
`def f (x : Nat) {y : Nat} (z : Nat)`	`@f 7 _`	`@f 7 _ 3`
`def f (x : Nat) {{y : Nat}} (z : Nat)`	`f 7`	`@f 7 _ 3`

The strict implicit version; the two comment lines summarize th3 in both versions

```
def symmetric {α : Type u} (r : α → α → Prop) : Prop :=
  ∀ {{a b : α}}, r a b → r b a
def Euclidean {α : Type u} (r : α → α → Prop) : Prop :=
  ∀ {{a b c : α}}, r a b → r a c → r b c

-- with { }:  th2 (th1 reflr @euclr) @euclr
-- with {{ }}: th2 (th1 reflr euclr) euclr
```

th3 combines th1 (reflexive + Euclidean ⇒ symmetric) with th2 (symmetric + Euclidean ⇒ transitive).

With { }, writing just euclr makes Lean insert three holes for a b c at once, giving r ?a ?b → r ?a ?c → r ?b ?c rather than Euclidean r ⇒ we must write @euclr.

With {{ }}, a bare euclr is kept as is; holes are inserted only when an explicit argument follows, as in euclr h (reflr _).

Notation: expansion and precedence

infixl, infix, infixr, prefix, postfix expand to notation

```
notation:65 lhs:65 " + " rhs:66 ⇒ HAdd.hAdd lhs rhs
notation:50 lhs:51 " = " rhs:51 ⇒ Eq lhs rhs
notation:80 lhs:81 " ^ " rhs:80 ⇒ HPow.hPow lhs rhs
notation:100 "-" arg:100 ⇒ Neg.neg arg
 -- `max` is a shorthand for precedence 1024:
notation:1024 arg:1024 "⁻¹" ⇒ Inv.inv arg
```

```
notation:65 lhs:65 " ~ " rhs:65 ⇒ wobble lhs rhs

notation:65 a " + " b:66 " + " c:66 ⇒ a + b - c
#eval 1 + 2 + 3  -- 0
```

- **notation:p** — p is the precedence of the whole notation; a slot **lhs:q** accepts only expressions of precedence ≥ q.
- a + b + c: rhs:66 rejects b + c (level 65) ⇒ only (a + b) + c — left-associative.
- a ^ b ^ c: rhs:80 accepts b ^ c ⇒ a ^ (b ^ c) — right-associative.
- =: both sides 51 ⇒ a = b = c does not parse — non-associative.
- ~: both sides 65 cannot decide ⇒ the “longest parse” rule: a ~ (b ~ c).
- 1 + 2 + 3 matches the new (longer) ternary notation ⇒ 1 + 2 - 3, which is 0 on Nat. If several notations tie for longest, elaboration picks the unique well-typed one.

CHAPTER 6 · §6.8–6.15

Everyday language conveniences

Section	Syntax	Meaning
6.8 Coercion	`i + ↑m + ↑n : Int`	Lean inserts the Nat → Int coercion, shown as ↑
6.9 Display	`#check @foo, #print, #eval`	Type (with implicit arguments), definition, value
6.10 Options	`set_option pp.all true`	pp.explicit, pp.universes, pp.notation — for debugging
6.11 Library	`Prod.fst, Or.elim`	camelCase for terms, UpperCamelCase for types and propositions
6.12 Auto bound	`def compose (g : β → γ) …`	Unbound names become implicit arguments
6.13 Implicit λ	`fun {α} ⟹ …`	Inserted when the expected type awaits implicit arguments
6.14 Short functions	`(· + 1), (· + ·)`	fun x => x + 1; several · become several arguments
6.15 Named args	`xs.foldl (init := 0) (·+·)`	Pass by name; .. fills the rest with _

CHAPTER 6 · §6.8, §6.10, §6.12

Coercions, display options, auto bound implicits

```
variable (m n : Nat)
variable (i j : Int)

#check i + m      -- i + ↑m : Int
#check i + m + j  -- i + ↑m + j : Int
#check i + m + n  -- i + ↑m + ↑n : Int
```

i + m + n is (i + m) + n, so m and n are coerced separately (Int.ofNat).

```
def compose (g : β → γ) (f : α → β) (x : α) : γ :=
  g (f x)
#check @compose
```

```
@compose : {β : Sort u_1} → {γ : Sort u_2} → {α : Sort u_3} → (β → γ) → (α → β) → α → γ
```

```
set_option pp.explicit true
set_option pp.universes true
set_option pp.notation false
#check 2 + 2 = 4
```

Shows all implicit arguments, universe parameters, and function form (HAdd.hAdd, Eq) instead of notation. pp.all true turns everything on — useful for debugging.

> β, γ, α are undeclared ⇒ implicit arguments, types generalized to Sort. set_option autoImplicit false disables this (error Unknown identifier `β`) — many projects do, so typos never become new variables.

CHAPTER 6 · §6.13–6.15

Short functions with ·, named arguments and ..

With def f (x y z : Nat) := x + y + z

Expression	Lean prints
`#check (· + 1)`	`fun x ⟹ x + 1 : Nat → Nat`
`#check (2 - ·)`	`fun x ⟹ 2 - x : Nat → Nat`
`#eval [1, 2, 3, 4, 5].foldl (· * ·) 1`	`120`
`#check (f · 1 ·)`	`fun x1 x2 ⟹ f x1 1 x2 : Nat → Nat → Nat`
`#eval [(1, 2), (3, 4), (5, 6)].map (·.1)`	`[1, 3, 5]`

The nearest parentheses containing · become a fun; several · become arguments from left to right; nested parentheses create a new function.

```
def sum (xs : List Nat) :=
  xs.foldl (init := 0) (·+·)

example {a b : Nat} {p : Nat → Nat → Nat → Prop}
    (h₁ : p a b b) (h₂ : b = a) :
    p a a b :=
  Eq.subst (motive := fun x ⟹ p a x b) h₂ h₁
```

- **(init := 0)**: passes by parameter name; #eval sum [1, 2, 3, 4] gives 10.
- **(motive := ...)** supplies an implicit argument Lean cannot infer on its own.
- **..** fills the remaining explicit arguments with _: Nat.add_assoc ..
- Implicit lambda: Lean wraps fun {α} => ... when the expected type awaits implicit arguments; disable with @fun.

Unknown names, expired rules, “silent” typos

Message	Cause → fix
`Unknown identifier `gcd``	The name lives in a namespace not yet opened → write Nat.gcd, or open Nat (gcd), or open Nat in
`Unknown identifier `bar``	Foo is open but Foo.bar is protected → always write the full name Foo.bar
`` `simp` made no progress ``	The rule was tagged with attribute [local simp] in a section that has ended → tag it again, or simp [rule_name]
`Unknown identifier `β` … autoImplicit option is set to false`	The project disables auto-bound → declare {β : Type} explicitly or use variable

A trap with no error message: a misspelled name (illustration)

```
theorem T : Lenght = 3 := sorry
#check @T
-- @T : ∀ {Lenght : Nat}, Lenght = 3
```

Lean does not report “unknown Lenght”: auto-bound implicits turn it into a **new implicit argument.** The proposition now says “every number equals 3” — far from the intent, and it surfaces only when the proof fails.

> *Hence many projects set autoImplicit false: a misspelled name becomes an immediate Unknown identifier error.*

CHAPTER 6 · CHAPTER SUMMARY

Working with Lean: the key ideas

1. **Messages:** errors, warnings, information; #guard_msgs turns them into tests.
2. **import** is transitive; **open** applies only within the current file and scope.
3. **section** limits scope; a variable is added only to definitions that use it; local attributes and notation expire at end.
4. **namespace:** full names must be unique, short names may clash — Lean uses types to choose; protected, _root_, export.
5. **Arguments:** ( ), { }, ⦃ ⦄, [ ]; @ disables implicit filling; pass by name (x := e).
6. **Elaboration conveniences:** notation with precedence, coercion ↑, auto-bound implicits, short functions with · — the kernel sees only the expanded term.

Key syntax	Meaning
`open Nat (succ) hiding …`	Selective open
`open X in cmd`	Open for exactly one command
`attribute [local simp] t`	Local simp rule
`infixl:65 " + " ⇒ f`	Left-associative notation
`set_option pp.all true`	See the real term
`#guard_msgs in cmd`	Assert messages
`f (init := 0) ..`	Named arguments; .. fills _
`autoImplicit false`	Disable auto-bound

CHAPTER 6 · QUESTIONS TO REFLECT ON

Four questions on scope, arguments and notation

Question 1

open does not travel with import, while @[simp] (non-local) follows every importing file. Why is this design sensible?

Question 2

With variable (x y : Nat), double := x + x takes only x. If Lean added every variable to every definition, what type would double y have?

Question 3

When is strict implicit ⦃a b⦄ more necessary than ordinary implicit {a b}? Recall the Euclidean example.

Question 4

Why is a + b + c read as (a + b) + c, but a ^ b ^ c as a ^ (b ^ c)? Which number in the notation declaration decides this?

Suggested answers to the four questions

Question 1 — naming is the reader's choice

open only changes how names are **referred to**; if it spread through imports, every file would receive a pile of unwanted short names prone to clashes. @[simp] is a **fact** about a theorem, deliberately shared by the library — but for that reason it cannot be removed, so a rule suited to one file should be local.

Question 2 — a superfluous argument

double would be double (x y : Nat) : Nat, so double y would have type Nat → Nat instead of Nat. Every use would have to pass a meaningless y; theorems about double would carry y too. Lean avoids this by adding only variables that occur in the type or body.

Question 3 — when a hypothesis stands alone

{a b c} is filled in as soon as the name appears: euclr becomes r ?a ?b → ... rather than Euclidean r, so we must write @euclr. ⦃a b c⦄ is filled only when an explicit argument follows, so a bare euclr keeps its type.

Question 4 — the precedence of the slots

+ expands to notation:65 lhs:65 " + " rhs:66: the right slot requires level ≥ 66, so it cannot hold b + c ⇒ left-associative. ^ has lhs:81, rhs:80: the right side accepts b ^ c (level 80) ⇒ right-associative.

PART III · CHAPTER

7

Inductive Types

Building data from constructors

SECTIONS

7.1 Enumerated Types

7.2 Constructors with Arguments

7.3 Inductively Defined Propositions

7.4 Defining the Natural Numbers

7.5 Other Recursive Data Types

7.6 Tactics for Inductive Types

7.7 Inductive Families

7.8 Axiomatic Details

7.9 Mutual and Nested Inductive Types

7.10 Further problems

Inductive types as LEGO bricks, or as a grammar



`succ (succ zero)`

2: three stacked bricks



`cons 1 (cons 2 nil)`

the list [1, 2]: a chain of bricks

```
-- read as a grammar (illustration, not Lean code)
Nat    ::= zero | succ Nat
List α ::= nil  | cons α (List α)
```

- Each **constructor** is a kind of brick. A brick may have “sockets” that take other bricks of the same type: succ takes a Nat, cons takes a List.
- Every value is a **finite** structure built from these bricks. There are no other values, and two different assemblies give two different values.
- Seen as a grammar: constructors are production rules, values are the “sentences” they generate.
- Key consequence: to define a function or prove a property for **all** values, it suffices to say what to do with **each kind of brick**.

Enumerated types and the recursor

- Inductive type = a list of **constructors**; every element is generated by them alone (subject to “positivity”)
- **Introduction** = constructors; **elimination** = the recursor, with induction as a special case
- Lean generates **Weekday.rec** with motive : Weekday → Sort u and one value per constructor
- match compiles via Weekday.casesOn; **deriving Repr** lets #eval display values
- Bool is also an enumerated type; and, or, not are defined by match

Example — days of the week (abridged)

```
inductive Weekday where
  | sunday | monday | tuesday
  | wednesday | thursday
  | friday | saturday
  deriving Repr

-- next, previous defined by match
theorem next_previous (d : Weekday) :
    next (previous d) = d := by
  cases d <;> rfl
```

The full Weekday and numberOfDay defined by match

Declaration: seven constructors, each one an element

```
inductive Weekday where
  | sunday : Weekday
  | monday : Weekday
  | tuesday : Weekday
  | wednesday : Weekday
  | thursday : Weekday
  | friday : Weekday
  | saturday : Weekday
```

- Constructors live in the **Weekday** namespace: Weekday.sunday; after open Weekday, just sunday
- ": Weekday" may be omitted; several constructors may share one line
- Seven distinct elements, and **no others**

A function Weekday → Nat: one value per constructor

```
open Weekday

def numberOfDay (d : Weekday) : Nat :=
  match d with
  | sunday    ⇒ 1
  | monday    ⇒ 2
  | tuesday   ⇒ 3
  | wednesday ⇒ 4
  | thursday  ⇒ 5
  | friday    ⇒ 6
  | saturday  ⇒ 7

#eval numberOfDay Weekday.sunday  -- 1
#eval numberOfDay Weekday.monday  -- 2
#eval numberOfDay Weekday.tuesday -- 3
```

Behind match: Weekday.casesOn and the recursor Weekday.rec

The recursor is generated together with the type

```
Weekday.rec.{u} {motive : Weekday → Sort u}
  (sunday : motive Weekday.sunday)
  (monday : motive Weekday.monday)
  (tuesday : motive Weekday.tuesday)
  (wednesday : motive Weekday.wednesday)
  (thursday : motive Weekday.thursday)
  (friday : motive Weekday.friday)
  (saturday : motive Weekday.saturday)
  (t : Weekday) :
  motive t
```

> *Computation rule: Weekday.rec v_1 … v_7 tuesday ⇝ v_3. This is how #eval and rfl "compute" values.*

Command	What you observe
`set_option pp.all true`	Shows all hidden detail: universes, implicit arguments
`#print numberOfDay`	The body calls the auxiliary numberOfDay.match_1
`#print numberOfDay.match_1`	Auxiliary generated by match, built from Weekday.casesOn
`#print Weekday.casesOn`	Built from Weekday.rec, with t moved before the cases
`#check @Weekday.rec`	The recursor's type, as on the left

> *In the logic, match reduces to the recursor, so it is always sound; compiled code (#eval) translates match as any functional language does.*

CHAPTER 7 · §7.2–7.3

Constructors with arguments and inductive propositions

- **Prod:** one constructor with two arguments — a “conjunction” type; **Sum:** inl, inr — a “disjunction” type
- Defining functions by cases with casesOn and (motive := fun _ => Nat)
- **structure:** a one-constructor type with generated projections (Prod, Color, Semigroup)
- Option (partial functions), Inhabited (nonempty types), Sigma
- Inductive types in Prop usually eliminate only into Prop — proofs carry no data

In Type	In Prop	Constructors
`Empty`	`False`	(none)
`Unit`	`True`	intro
`Prod α β`	`And a b`	intro
`Sum α β`	`Or a b`	inl, inr
`Sigma (Σ)`	`Exists (∃)`	intro

> **Subtype** {x : α // p x}: a hybrid of ∃ and Σ — in fact a structure with two fields, val and property.

Prod and Sum: defining functions with casesOn and motive

The type of Prod.casesOn (omitting {α} {β})

```
Prod.casesOn :
  {motive : α × β → Sort u_1} →
  (t : α × β) →
  ((fst : α) → (snd : β) → motive (fst, snd)) →
  motive t
```

One constructor, two arguments ⇒ one case taking both a and b

```
def prod_example (p : Bool × Nat) : Nat :=
  Prod.casesOn (motive := fun _ ⟹ Nat) p
    (fun b n ⟹ cond b (2 * n) (2 * n + 1))

#eval prod_example (true, 3)  -- 6
#eval prod_example (false, 3) -- 7
```

Two constructors ⇒ two cases, one function each

```
def sum_example (s : Sum Nat Nat) : Nat :=
  Sum.casesOn (motive := fun _ ⟹ Nat) s
    (fun n ⟹ 2 * n)
    (fun n ⟹ 2 * n + 1)

#eval sum_example (Sum.inl 3) -- 6
#eval sum_example (Sum.inr 3) -- 7
```

- **motive** := fun _ => Nat: the result type, a function since it may depend on the input value; here the constant Nat
- **cond b t_1 t_2**: returns t_1 if b = true, otherwise t_2
- Prod is a **conjunction** (you get a *and* b); Sum is a **disjunction** (inl a *or* inr b)

structure: a one-constructor type with projections

An RGB colour record — the default constructor is mk

```
structure Color where
  red : Nat
  green : Nat
  blue : Nat
deriving Repr

def yellow := Color.mk 255 255 0

#eval Color.red yellow  -- 255
```

One structure command generates: type Color, constructor Color.mk, eliminators rec and recOn, and projections Color.red, Color.green, Color.blue.

Prod rewritten as a structure, naming the constructor

```
structure Prod (α : Type u) (β : Type v) where
  mk ::
  fst : α
  snd : β
```

A semigroup: a type, an operation and a proof (Chapter 9)

```
structure Semigroup where
  carrier : Type u
  mul : carrier → carrier → carrier
  mul_assoc : ∀ a b c,
    mul (mul a b) c = mul a (mul b c)
```

Sigma, Option and Inhabited

Three library types (redefined in namespace Hidden)

```
inductive Sigma {α : Type u} (β : α → Type v) where
  | mk : (a : α) → β a → Sigma β
inductive Option (α : Type u) where
  | none : Option α
  | some : α → Option α
inductive Inhabited (α : Type u) where
  | mk : α → Inhabited α
```

Further problems

Compose two partial functions; show that Bool, Nat, products and function types into inhabited types are inhabited.

Option — partial functions. Type theory has no partial functions: every f : α → β has a value at every input. With f : α → Option β, f a = none means "undefined" and f a = some b means the value is b.

Inhabited — evidence of an element. An element of Inhabited α is just a "witness" that α is nonempty. In Chapter 10 it becomes a type class: Lean infers that compound types built from inhabited types are inhabited.

Sigma — dependent pairs. The type of the second component, β a, depends on the first component a; seen in Chapter 2 with the notation Σ a : α, β a.

The logical connectives are inductive types in Prop

```
inductive False : Prop

inductive True : Prop where
  | intro : True

inductive And (a b : Prop) : Prop where
  | intro : a → b → And a b

inductive Or (a b : Prop) : Prop where
  | inl : a → Or a b
  | inr : b → Or a b
```

```
inductive Exists {α : Sort u} (p : α → Prop) :
    Prop where
  | intro (w : α) (h : p w) : Exists p

structure Subtype {α : Sort u} (p : α → Prop) where
  val : α
  property : p val
```

> *{x : α // p x} is Subtype: like ∃, but val is data that can be extracted.*

Type	Chapter 3's introduction and elimination rules, revisited
`False`	No constructors ⇒ no way to prove it; the recursor needs no cases ⇒ it implies anything
`And, Or`	And.intro is exactly ⟨ha, hb⟩; Or has two constructors ⇒ eliminating Or is proof by cases
`Exists`	∃ x : α, p is notation for Exists (fun x : α => p); eliminates only into Prop — proofs carry no data

CHAPTER 7 · BACKGROUND

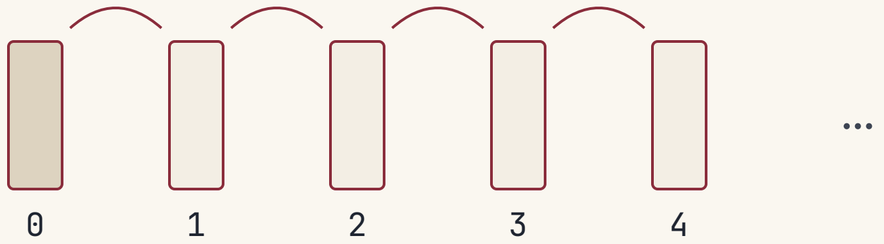
Mathematical induction: the domino effect

To prove a property P n for **every** natural number n, two things suffice:

Step	To prove
Base case	P 0 — the first domino falls
Inductive step	for all n, P n → P (n + 1) — each falling domino knocks over the next
Conclusion	∀ n, P n — every domino falls

The hypothesis P n in the inductive step is called the **induction hypothesis** (usually named ih).

> *In Lean this principle is not a separate axiom: it is the recursor Nat.rec, used with a proposition as motive.*



Why is it enough? Given base : P 0, step : ∀ n, P n → P (n + 1) and n = 3

```
base                  : P 0
step 0 base           : P 1
step 1 (step 0 base)  : P 2
step 2 (step 1 (…))   : P 3
```

Each n is built from zero by **finitely** many succ, so applying step that many times to base reaches P n. This is also exactly how Nat.rec computes.

Defining the natural numbers, and the shape of the recursor

```
inductive Nat where
  | zero : Nat
  | succ : Nat → Nat

-- type of the recursor Lean generates:
Nat.rec.{u} :
  {motive : Nat → Sort u} →
  (zero : motive Nat.zero) →
  (succ : (n : Nat) → motive n → motive (Nat.succ n)) →
  (t : Nat) → motive t
```

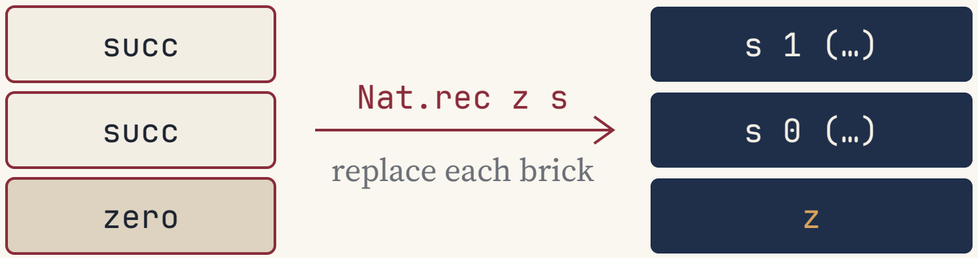
> **motive** — the target type, which may depend on t

> **minor premises** — the zero and succ cases (using n and the value of motive n)

> **major premise** — the input t : Nat; Nat.recOn puts it first

Addition is defined by recursion on the second argument; **induction = recursion with a Prop motive:** zero_add, add_assoc, add_comm are proved with Nat.recOn and calc, then shortened with simp.

The recursor: the one universal way to “consume” data



succ (succ zero) ⟼ s 1 (s 0 z)

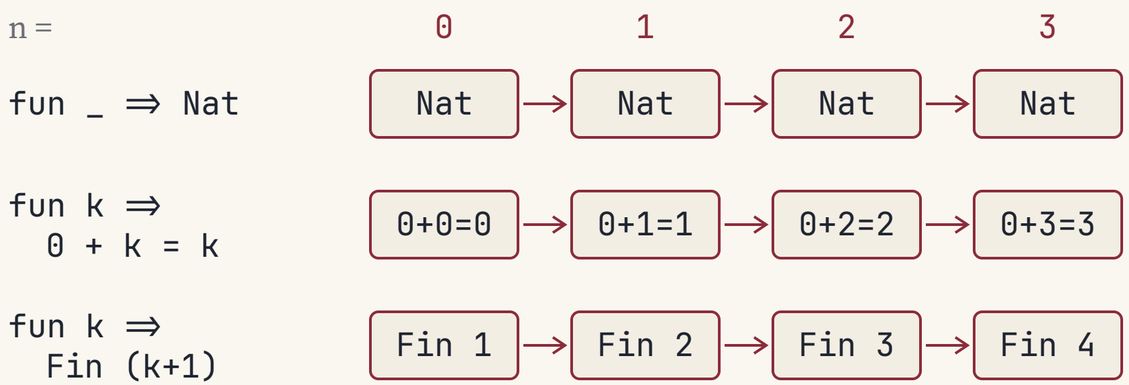
double written directly with the recursor (illustration)

```
noncomputable def double (n : Nat) : Nat :=
  Nat.rec (motive := fun _ ⇒ Nat)
    0 (fun _ r ⇒ r + 2) n
example : double 3 = 6 := rfl
```

- Every type T comes with an automatically generated T.rec. We give it **one formula per constructor:** z replaces zero, s replaces succ.
- For a brick with a socket (succ n), the formula s receives both n and **the result already computed for the sub-part** — that is where recursion happens, and it is also the induction hypothesis.
- For double: z = 0, s _ r = r + 2, so 3 ⟼ ((0 + 2) + 2) + 2 = 6.
- match, cases, induction, structural recursion, even WellFounded.fix (via Acc.rec): every way of using an inductive value is ultimately translated to the recursor.

> *The kernel need trust only one principle per type; every convenience above is just friendlier notation.*

The motive, slowly: each n gets a typed “answer slot”



Arrow = formula s, from slot n to slot n+1. Row 3 written with Nat.rec (illustration):

```
noncomputable def top (n : Nat) : Fin (n + 1) :=
  Nat.rec (motive := fun k ⇒ Fin (k + 1))
    0 (fun _ r ⇒ r.succ) n
example : top 3 = 3 := rfl
```

- **motive : Nat → Sort u** maps **values** to **types**: it answers “what type must the answer at n have?”. Each row of the figure is a motive.
- z : motive 0 fills the first slot; s : (n : Nat) → motive n → motive (n+1) turns slot n into slot n+1. Result: Nat.rec z s t : motive t.
- Row 1: one type throughout — plain recursion. Row 2: each slot a proposition — induction. Row 3: type varies with n — dependent types.
- In row 3, r : Fin (k+1) and r.succ : Fin (k+2) — the next slot’s type; top 3 is the largest element of Fin 4.
- induction reads the motive off the goal; if Lean cannot infer it, pass (motive := …).

Nat.rec argument by argument: addition via the recursor

Argument	When defining add m n (m fixed)
`motive`	fun _ => Nat — the result is always a natural number
`zero`	the value at 0: just m
`succ`	takes n and r = add m n (computed), returns succ r
`t`	the input n we recurse on

Computation rules (ι-reduction)

```
Nat.rec z s Nat.zero      ⤳ z
Nat.rec z s (Nat.succ n) ⤳
    s n (Nat.rec z s n)
```

Addition written directly with Nat.rec (illustration)

```
noncomputable def add' (m n : Nat) : Nat :=
  Nat.rec (motive := fun _ ⟹ Nat) m
    (fun _ r ⟹ Nat.succ r) n

example (m n : Nat) :
    add' m (Nat.succ n) = Nat.succ (add' m n) :=
  rfl
```

Computing add (succ (succ zero)) (succ zero), i.e. 2 + 1

```
⤳ succ (add (succ (succ zero)) zero)
⤳ succ (succ (succ zero))
```

noncomputable: older Lean versions generated no code for direct recursor uses; in Lean 4.33 the marker is optional but still valid.

Why do add_zero and add_succ need only rfl?

In namespace Hidden, after inductive Nat … deriving Repr

```
namespace Nat
def add (m n : Nat) : Nat :=
  match n with
  | Nat.zero   ⇒ m
  | Nat.succ n ⇒ Nat.succ (add m n)
instance : Add Nat where
  add := add
theorem add_zero (m : Nat) : m + zero = m := rfl
theorem add_succ (m n : Nat) :
    m + succ n = succ (m + n) := rfl
end Nat
```

rfl makes Lean reduce both sides to the same form

```
m + succ n
⇝ add m (succ n)      -- unfold the Add instance
⇝ succ (add m n)      -- succ branch of the match
≡ succ (m + n)        -- right-hand side
```

> *By contrast, 0 + n = n does **not** hold by rfl: match n with is "stuck" because n is a variable, not a constructor. Induction is needed.*

- add recurses on the **second** argument; the first is just carried along
- instance : Add Nat attaches the + notation to add (mechanism in Chapter 10)

Line by line: zero_add with Nat.recOn

Induction = recursion whose motive is a proposition

```
open Nat

theorem zero_add (n : Nat) : 0 + n = n :=
  Nat.recOn (motive := fun x ⟹ 0 + x = x)
   n
   (show 0 + 0 = 0 from rfl)
   (fun (n : Nat) (ih : 0 + n = n) ⟹
    show 0 + (n + 1) = n + 1 from
    calc 0 + (n + 1)
      _ = (0 + n) + 1 := rfl
      _ = n + 1       := by rw [ih])
```

Line	Meaning
`motive := …`	A proposition per x; the final goal is motive n
`show 0 + 0 = 0`	Base case motive 0: 0 + 0 ⇝ 0 by definition ⇒ rfl
`fun n ih ⟹`	Inductive step: from ih : motive n, prove motive (n + 1)
`(0 + n) + 1 := rfl`	succ branch of addition — holds by definition
`by rw [ih]`	Rewrite 0 + n to n by the induction hypothesis

> *Short version: rfl for the base case, (fun n ih => by simp [ih]) for the step.*

add_assoc: choosing the induction variable and tracing each calc step

m + n + k means (m + n) + k — induction on k

```
open Nat
theorem add_assoc (m n k : Nat) :
    m + n + k = m + (n + k) :=
  Nat.recOn (motive := fun k ⇒ m + n + k = m + (n + k)) k
    (show m + n + 0 = m + (n + 0) from rfl)
    (fun k (ih : m + n + k = m + (n + k)) ⇒
      show m + n + (k + 1) = m + (n + (k + 1)) from
      calc m + n + (k + 1)
        _ = (m + n + k) + 1   := rfl
        _ = (m + (n + k)) + 1 := by rw [ih]
        _ = m + ((n + k) + 1) := rfl
        _ = m + (n + (k + 1)) := rfl)
```

The inductive step, with hypothesis

```
ih : m + n + k = m + (n + k)
  m + n + (k + 1)
= (m + n + k) + 1   add_succ
= (m + (n + k)) + 1 rw [ih]
= m + ((n + k) + 1) add_succ ←
= m + (n + (k + 1)) add_succ ←
```

> *Addition recurses on its second argument, so induction on k makes every remaining step hold by definition.*

For add_comm, the final step succ (n + m) = succ n + m needs an auxiliary lemma succ_add, proved by induction on m.

CHAPTER 7 · §7.5–7.6

Other recursive types and tactics for inductive types

- **List α:** nil and cons (h : α) (t : List α); append, nil_append hold by rfl
- **BinaryTree**, and the countably branching tree **CBTree** with sup : (Nat → CBTree) → CBTree

```
theorem zero_add (n : Nat) : 0 + n = n := by
  induction n with
  | zero ⟹ rfl
  | succ n ih ⟹ rw [Nat.add_succ, ih]
```

Tactic	Effect on inductive types
`cases`	One goal per constructor; automatically reverts and reintroduces dependent hypotheses (h : n ≠ 0 becomes 0 ≠ 0)
`induction`	Like cases but also produces induction hypotheses; supports custom principles: induction x, y using …
`injection`	Constructors are injective: from succ (succ m) = succ (succ n) derive m = n
`contradiction`	Detects two distinct constructors claimed to be equal, e.g. 7 = 4

List α: append and its definitional equations

A list: empty, or a head element attached to a tail

```
inductive List (α : Type u) where
  | nil  : List α
  | cons (h : α) (t : List α) : List α

namespace List

def append (as bs : List α) : List α :=
  match as with
  | nil       ⇒ bs
  | cons a as ⇒ cons a (append as bs)
```

Recursion on the **first** list: peel off each element of as and reattach it to the front of the result.

Two defining equations — rfl suffices

```
theorem nil_append (as : List α) :
    append nil as = as :=
  rfl

theorem cons_append (a : α) (as bs : List α) :
    append (cons a as) bs = cons a (append as bs) :=
  rfl
```

> *append as nil = as is **not** rfl: match as gets stuck when as is a variable. Use induction as, then rw [cons_append, ih].*

Further problems: append_nil, append_assoc; define length and prove length (append as bs) = length as + length bs.

The goal state step by step: cases on Nat

```
example (p : Nat → Prop)
    (hz : p 0) (hs : ∀ n, p (Nat.succ n)) :
    ∀ n, p n := by
  intro n
  cases n
  . exact hz    -- zero branch
  . apply hs    -- succ branch
```

> *n✝ is inaccessible; name it with cases n with | zero => … | succ m => … or case succ m => ….*

cases reverts and reintroduces hypotheses on n: for h : n ≠ 0, branch zero has h : 0 ≠ 0 (absurd rfl h), branch succ m has h : m + 1 ≠ 0.

First branch (before exact hz)

```
case zero
p : Nat → Prop
hz : p 0
hs : ∀ (n : Nat), p n.succ
⊢ p 0
```

Second branch (before apply hs)

```
case succ
p : Nat → Prop
hz : p 0
hs : ∀ (n : Nat), p n.succ
n✝ : Nat
⊢ p (n✝ + 1)
```

induction: where does the induction hypothesis appear?

```
theorem zero_add (n : Nat) : 0 + n = n := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒ rw [Nat.add_succ, ih]
```

- Same syntax as cases, but the succ branch also gets **ih** — exactly the motive n argument of Nat.rec
- Equivalently: induction n, then case zero => … and case succ n ih => …
- Even shorter: induction n <;> simp [*, add_zero, add_succ] — the * passes ih to simp
- Custom principles, several variables: induction x, y using Nat.mod.inductionOn

zero branch — closed by rfl (0 + 0 ⇝ 0)

```
⊢ 0 + 0 = 0
```

succ branch — initially

```
n : Nat
ih : 0 + n = n
⊢ 0 + (n + 1) = n + 1
```

After rw [Nat.add_succ]: 0 + succ n becomes succ (0 + n)

```
⊢ (0 + n).succ = n + 1
```

After rw [ih]: rw tries rfl and closes the goal

```
⊢ n.succ = n + 1  ✓
```

One mechanism, many notations: from recursor to tactic

Notation	Level	Translated to	Induction hypothesis?
`Nat.rec, Nat.recOn`	term	itself — explicit motive	Yes: the motive n argument in succ
`Nat.casesOn`	term	Nat.rec, dropping recursive results	No
`match n with`	term	casesOn, via the auxiliary f.match_1	No (unless it recurses)
`| n+1 ⇒ … f n`	term	brecOn (Chapter 8) or WellFounded.fix	The call f n plays the role of ih
`cases n`	tactic	casesOn, motive read from the goal	No
`induction n`	tactic	rec, motive read from the goal	Yes: the variable ih in succ

cases ≈ casesOn ≈ match: asks only "which brick built this value?". Enough when the sub-parts do not matter.

induction ≈ rec ≈ recursion: can also use the result for the sub-parts. Needed when n+1's property relies on n's.

The same zero_add seen three times: Nat.recOn (§7.4), induction (§7.6), recursive function (§8.3) — all reduce to Nat.rec.

cases on expressions; injection and contradiction

Expression occurring in the goal: generalize, then split (after open Nat)

```
example (p : Nat → Prop) (hz : p 0)
    (hs : ∀ n, p (succ n)) (m k : Nat)
    : p (m + 3 * k) := by
  cases m + 3 * k
  exact hz    -- goal is p 0
  apply hs    -- goal is a : Nat ⊢ p (succ a)
```

Expression not in the goal: cases splits an ∨ hypothesis

```
example (p : Prop) (m n : Nat)
    (h₁ : m < n → p) (h₂ : m ≥ n → p) : p := by
  cases Nat.lt_or_ge m n
  case inl hlt ⟹ exact h₁ hlt
  case inr hge ⟹ exact h₂ hge
```

```
open Nat
example (m n k : Nat)
    (h : succ (succ m) = succ (succ n))
    : n + k = m + k := by
  injection h with h'
  injection h' with h''
  rw [h'']
```

Hypotheses added at each step

```
h' : m + 1 = n + 1   -- succ is injective
h'' : m = n
⊢ n + k = n + k      -- rw closes it by rfl
```

Two distinct constructors cannot be equal: given h : succ m = 0 or 7 = 4, injection or contradiction closes the goal.

cases and induction: missing branches, lost variable names

Forgetting the succ branch (illustration)

```
theorem t (n : Nat) : 0 + n = n := by
  induction n with
  | zero ⟹ rfl
```

```
error: Alternative `succ` has not been
provided
```

- **Reading:** the with | ... syntax requires exactly one branch per constructor; the missing branch is named explicitly.
- **Fix:** add | succ n ih => rw [Nat.add_succ, ih]. For types with many constructors, check against the constructor list in the inductive declaration.

Reusing the old name after cases (illustration)

```
example (n : Nat) (h : n ≠ 0) : n ≥ 1 := by
  cases n
  · contradiction
  · exact Nat.succ_le_succ (Nat.zero_le n)
```

```
error: Unknown identifier `n`
```

- **Reading:** cases n replaces n by zero or succ n✝; the new variable has the hidden name n✝, and the old n no longer exists in the branch.
- **Fix:** name it when splitting — cases n with | zero => ... | succ m => ... m — or use rename_i m at the start of the branch.

An induction hypothesis too weak: when generalizing is needed

Reversing a list with an accumulator acc (illustration)

```
def revAux : List α → List α → List α
  | [],      acc ⇒ acc
  | a :: as, acc ⇒ revAux as (a :: acc)

theorem revAux_eq (as acc : List α) :
    revAux as acc = as.reverse ++ acc := by
  induction as with
  | nil ⇒ rfl
  | cons a as ih ⇒ simp [revAux, ih]
```

Fix: induction as generalizing acc with … — keeping both branches. Then ih : ∀ (acc : List α), revAux as acc = as.reverse ++ acc, and simp [revAux, ih] closes the goal.

```
error: unsolved goals
case cons  -- α, a, as omitted
acc : List α
ih : revAux as acc = as.reverse ++ acc
⊢ revAux as (a :: acc) =
    as.reverse ++ a :: acc
```

- **Reading:** ih speaks only about **the** original acc, whereas the goal needs it for a :: acc. They do not match, so simp ignores ih (Lean also warns that ih is unused).
- **Why:** acc was already in the context before induction ran, so the motive treats it as a constant. generalizing acc moves acc into the motive: “for **every** acc”.

CHAPTER 7 · §7.7–7.8

Inductive families and axiomatic details

```
inductive Vect (α : Type u) : Nat → Type u where
  | nil  : Vect α 0
  | cons : α → {n : Nat} → Vect α n → Vect α (n + 1)

inductive Eq {α : Sort u} (a : α) : α → Prop where
  | refl : Eq a a
```

- Inductive families: inductive foo : … → Sort u — the arguments are **indices**
- Eq is a family indexed by x with one constructor refl; every axiom of equality follows from refl and Eq.rec
- Lean does not support induction-recursion or induction-induction

Parameters vs indices: parameters stay fixed throughout the definition; indices vary by constructor

Strict positivity: the type being defined may occur only in the result of an arrow

Universes: a type outside Prop must live in a universe ≥ that of every constructor argument

Singleton elimination: one constructor, arguments Props or indices ⇒ eliminates into any Sort

Eq.rec: a motive depending on both point and proof

The eliminator of Eq (with universes u v)

```
#check (@Eq.rec : {α : Sort u} → {a : α} →
    {motive : (x : α) → a = x → Sort v} →
    motive a rfl →
    {b : α} → (h : a = b) → motive b h)
```

- Eq a x is a **family** indexed by x; only Eq a a is inhabited (refl)
- **motive x h:** the target type, depending on the point x and the proof h : a = x
- Only **one** case to handle: motive a rfl — x is just a
- Eq lives in Prop, yet the motive targets any Sort v: allowed by **singleton elimination** (§7.8)

Substitution follows directly from Eq.rec

```
theorem subst {α : Type u} {a b : α} {p : α → Prop}
    (h₁ : Eq a b) (h₂ : p a) : p b :=
  Eq.rec (motive := fun x _ ⟹ p x) h₂ h₁
```

With motive := fun x _ => p x

```
motive a rfl ≡ p a   -- supplied by h₂
motive b h₁  ≡ p b   -- the required result
```

```
-- match on h also reduces to Eq.rec
theorem symm (h : Eq a b) : Eq b a :=
  match h with
  | rfl ⟹ rfl
```

Why strict positivity, universe and elimination constraints?

(illustration) A declaration Lean rejects

```
inductive Bad where
  | mk : (Bad → Nat) → Bad
-- error: non positive occurrence
```

- **Strict positivity:** the type being defined may appear only in the *result* of an arrow in a constructor argument
- Bad sits to the *left* of the arrow: some f : Bad → Nat could be applied to mk f itself — self-reference that breaks "building from the bottom up", leading to non-terminating functions and inconsistency
- Valid: sup : (Nat → CBTree) → CBTree — CBTree only in the result

Universes. For a constructor c : (a : α) → (b : β[a]) → C a p[a,b], if C is not in Prop then each argument β_k : Sort v must satisfy u ≥ v. Hence Prod α β with α : Type u, β : Type v lives in Type (max u v).

Type in Prop	Can eliminate into
`Eq, And, True`	Any Sort — one constructor, arguments Props or indices (singleton elimination)
`Or`	Prop only — two constructors
`Exists`	Prop only — the argument w : α is data

> *Casting along h : a = b creates no new data; it only reinterprets existing data.*

Rejected inductive declarations: positivity and universes

The type appears left of an arrow (illustration)

```
inductive Bad where
  | mk : (Bad → Nat) → Bad
```

```
error: (kernel) arg #1 of 'Bad.mk'
has a non positive occurrence of
the datatypes being declared
```

- **Reading:** “(kernel)” — rejected by the kernel itself; “arg #1” — mk’s first argument; “non positive” — Bad is left of an arrow.
- **Fix:** redesign. Bad may appear only in the **result:** | mk : (Nat → Bad) → Bad is valid (like CBTree).

A constructor holding a type larger than itself (illustration)

```
inductive Box : Type where
  | mk : Type → Box
```

```
error: Invalid universe level in
constructor `Box.mk`: Parameter has type
  Type
at universe level 2 which is not less
than or equal to the inductive type's
resulting universe level 1
```

- **Reading:** mk’s parameter has type Type : Sort 2, but Box : Type = Sort 1. The universe rule (§7.8) needs the type’s universe ≥ each argument’s.
- **Fix:** raise the result — inductive Box : Type 1 where | mk : Type → Box.

CHAPTER 7 · §7.9–7.10

Mutual and nested inductive types

```
mutual
  inductive Even : Nat → Prop where
    | even_zero : Even 0
    | even_succ : (n : Nat) → Odd n → Even (n + 1)
  inductive Odd : Nat → Prop where
    | odd_succ : (n : Nat) → Even n → Odd (n + 1)
end
```

```
inductive Tree (α : Type u) where
  | mk : α → List (Tree α) → Tree α
```

- Lean “compiles” them to primitive inductive types using auxiliary types
- Nested: Tree occurs inside List; the kernel builds the isomorphism TreeList α ≅ List (Tree α)

Further problems (§7.10)

On Nat: multiplication, pred, truncated subtraction, exponentiation · On List: length, reverse; prove reverse (reverse xs) = xs · A Term type with an evaluation function · Propositional formulas with evaluation, complexity, substitution

Comparison: mutual trees (TreeList) and nested trees (List)

Mutual: two types defined together, referring to each other

```
mutual
    inductive Tree (α : Type u) where
      | node : α → TreeList α → Tree α
    inductive TreeList (α : Type u) where
      | nil  : TreeList α
      | cons : Tree α → TreeList α → TreeList α
end
```

TreeList α is isomorphic to List (Tree α), but cannot use the List library; converting back and forth along the isomorphism is tedious.

Nested: the type we really want

```
inductive Tree (α : Type u) where
  | mk : α → List (Tree α) → Tree α
```

- Tree occurs *inside* List — not strictly positive in the narrow sense
- The kernel builds the isomorphism TreeList α ≅ List (Tree α) and defines the constructor through it
- Using it effectively needs the mutual recursion of Chapter 8 (numConsts on Term)

(illustration) Building a proof of Even 2 from the mutual constructors

```
example : Even 2 :=
  Even.even_succ 1 (Odd.odd_succ 0 Even.even_zero)
```

CHAPTER 7 · CHAPTER SUMMARY

Inductive types: the key ideas

1. **inductive** = a list of constructors; every element is built finitely from them; distinct constructors give distinct values; constructors are injective.
2. **Recursor** T.rec: one formula per constructor; the **motive** gives the result type, possibly value-dependent.
3. **Induction** = recursion with a Prop motive.
4. **Logic as inductive types**: False, True, And, Or, Exists; Prop types usually eliminate only into Prop.
5. **Inductive families**: fixed parameters, indices vary by constructor; Eq is a family with one constructor refl.
6. **Constraints**: strict positivity, universes; mutual and nested types compile to primitive ones.

Key syntax	Meaning
`inductive T where | c : … → T`	Declare a type
`structure S where f : α`	One constructor + projections
`T.rec (motive := …)`	Recursor
`match x with | c a ⇒ …`	Case analysis
`cases x with | c a ⇒ …`	Split, no ih
`induction x generalizing y`	Induction, general ih
`injection h with h'`	Constructors are injective
`deriving Repr`	Lets #eval display values

CHAPTER 7 · QUESTIONS TO REFLECT ON

Four questions on constructors, recursors and motives

Question 1

Why does n + 0 = n hold by rfl, while 0 + n = n must be proved by induction?

Question 2

Or eliminates only into Prop, And into any Sort. If we could extract a Bool from h : p ∨ q (inl ↦ true, inr ↦ false), what would go wrong?

Question 3

If Lean allowed | mk : (Bad → False) → Bad, how could we prove False?

Question 4

A motive is a function Nat → Sort u, not a fixed type. Give three situations where the motive genuinely depends on n.

Suggested answers to the four reflection questions

Question 1 — addition recurses on the second argument

n + 0 matches the zero branch of the match at once ⇒ reduces to n. In 0 + n, the match must inspect n, but n is a variable, neither zero nor succ — computation gets “stuck”. We must split n by constructor, i.e. use induction.

Question 2 — it contradicts proof irrelevance

In Lean, all proofs of the same proposition are equal. When p and q both hold, Or.inl hp and Or.inr hq are two proofs of p ∨ q, hence equal, which gives true = false. And has only one constructor, so it reveals no “choice”.

Question 3 — self-application

Using Bad’s recursor, define g : Bad → False by g (mk f) := f (mk f). Then g (mk g) : False — no other axiom needed. That is why the kernel demands strict positivity.

Question 4 — three examples

(a) Induction: motive n := 0 + n = n — a different proposition for each n. (b) Dependent types: motive n := Fin (n + 1) or Vect α n. (c) Eq.rec: motive x h depends on both the point x and the proof h.

PART III · CHAPTER

8

Induction and Recursion

Pattern matching, recursion, termination

SECTIONS

8.1 Pattern Matching

8.2 Wildcards and Overlapping Patterns

8.3 Structural Recursion and Induction

8.4 Local recursive declarations

8.5 Well-Founded Recursion and Induction

8.6 Functional Induction

8.7 Mutual Recursion

8.8 Dependent Pattern Matching

8.9 Inaccessible Patterns

8.10 Match Expressions

8.11 Further problems

Recursion and induction: two sides of one coin

```
-- recursion: compute a value
def add : Nat → Nat → Nat
  | m, zero   ⇒ m
  | m, succ n ⇒ succ (add m n)
```

```
-- induction: build a proof
theorem zero_add : ∀ n, add zero n = n
  | zero   ⇒ rfl
  | succ n ⇒ congrArg succ (zero_add n)
```

Recursion (defining a function)	Induction (a proof)	Common ground
Result: a value of type C n	Result: a proof of P n	Motive n := C n or P n
zero branch: the value at 0	Base case: prove P 0	One branch per constructor
succ n branch calls add m n	succ n branch uses zero_add n	Call on a subterm = induction hypothesis
Call on any smaller value (x - y)	Use P y for every smaller y	Well-founded recursion / induction (§8.5)
Must terminate	No circular reasoning	The same check

> *Curry–Howard: zero_add does not merely look like a recursive function — it is one, taking n and returning a proof for n.*

Pattern matching and overlapping patterns

Example — nested patterns with numeric notation

```
def sub2 : Nat → Nat
  | 0     ⇒ 0
  | 1     ⇒ 0
  | x + 2 ⇒ x

-- after elaboration:
-- fun x ⇒ match x with … (sub2.match_1)
```

- Definition by cases on constructors; the equations hold **definitionally**
- 0 and x + 1 can serve as patterns thanks to the **[match_pattern]** attribute
- Works for every inductive type: products, Option, even propositions (And.intro h_1 h_2, Or.inl hp); multiple arguments are separated by commas
- _ is a wildcard, not an implicit argument
- **Overlap:** the first matching equation is used; missing cases ⇒ return a default or an Option; the compiler lists the missing cases

sub1 and isZero: definitionally true equations

Every natural number is zero or succ x — one line per case

```
open Nat

def sub1 : Nat → Nat
  | zero   ⇒ zero
  | succ x ⇒ x

def isZero : Nat → Bool
  | zero   ⇒ true
  | succ x ⇒ false
```

> *The equation compiler translates these lines into casesOn/recursors. It is not part of the trusted core: the kernel rechecks the result.*

Every equation closes by rfl

```
example : sub1 0 = 0 := rfl
example (x : Nat) : sub1 (succ x) = x := rfl

example : isZero 0 = true := rfl
example (x : Nat) : isZero (succ x) = false := rfl

example : sub1 7 = 6 := rfl
example (x : Nat) : isZero (x + 3) = false := rfl
```

- x + 3 reduces to succ (x + 2): the succ constructor is exposed, so rfl works
- Writing | 0 => … and | x + 1 => … also works: 0 and + carry **[match_pattern]**, so Lean normalizes until it sees zero, succ

Matching on pairs, Option and even propositions

Data: products and Option

```
def swap : α × β → β × α
  | (a, b) ⇒ (b, a)

def foo : Nat × Nat → Nat
  | (m, n) ⇒ m + n

def bar : Option Nat → Nat
  | some n ⇒ n + 1
  | none   ⇒ 0
```

- (a, b) is notation for Prod.mk a b — a constructor pattern; Prod has one constructor, hence one line
- Option has two constructors ⇒ two lines; drop the none line and Lean reports a missing case

Proof by cases and splitting logical hypotheses

```
def not : Bool → Bool
  | true  ⇒ false
  | false ⇒ true

theorem not_not : ∀ (b : Bool), not (not b) = b
  | true  ⇒ show not (not true) = true from rfl
  | false ⇒ show not (not false) = false from rfl

example (p q : Prop) : p ∧ q → q ∧ p
  | And.intro h₁ h₂ ⇒ And.intro h₂ h₁

example (p q : Prop) : p ∨ q → q ∨ p
  | Or.inl hp ⇒ Or.inr hp
  | Or.inr hq ⇒ Or.inl hq
```

Curry–Howard: each line of not_not is a proof for one value of b.

Nested patterns and the automatic case-split tree

Patterns 0, 1, x+2 and equations that hold by rfl

```
def sub2 : Nat → Nat
  | 0   ⇒ 0
  | 1   ⇒ 0
  | x+2 ⇒ x

example : sub2 0 = 0 := rfl
example : sub2 1 = 0 := rfl
example : sub2 (x+2) = x := rfl

example : sub2 5 = 3 := rfl
```

#print sub2 reveals the auxiliary sub2.match_1: the definition is really fun x => match x with … ; if any case is missing, Lean reports an error.

Case-split tree of sub2

```
x = zero                ⇒ 0
x = succ x'
    x' = zero           ⇒ 0   (x = 1)
    x' = succ x'' ⇒ x''   (x = x''+2)
```

Several arguments, separated by commas, split in turn

```
def foo : Nat → Nat → Nat
  | 0,     n     ⇒ 0
  | m + 1, 0     ⇒ 1
  | m + 1, n + 1 ⇒ 2
```

Parameters before the “:” (such as {α : Type u} in tail1) take no part in the case split.

Overlapping patterns, wildcards and missing cases

Overlapping patterns: 0, 0 matches all three lines

```
def foo : Nat → Nat → Nat
  | 0, n ⇒ 0
  | m, 0 ⇒ 1
  | m, n ⇒ 2

example : foo 0       0       = 0 := rfl
example : foo 0       (n + 1) = 0 := rfl
example : foo (m + 1) 0       = 1 := rfl
example : foo (m + 1) (n + 1) = 2 := rfl
```

> *Rule: the **first** matching equation is used. Unused variables can be replaced by _: | 0, _ => 0 | _, 0 => 1 | _, _ => 2.*

Simulating "incomplete patterns": default or Option

```
def f1 : Nat → Nat → Nat
  | 0, _  ⇒ 1
  | _, 0  ⇒ 2
  | _, _  ⇒ default  -- the "incomplete" case

def f2 : Nat → Nat → Option Nat
  | 0, _  ⇒ some 1
  | _, 0  ⇒ some 2
  | _, _  ⇒ none     -- the "incomplete" case
```

- _ here is a wildcard pattern, *not* an implicit argument
- default comes from Inhabited Nat; f2 (a+1) (b+1) = none
- Omitting a case ⇒ an error listing exactly the missing cases; on Char, a match may compile to if-then-else

“Missing cases”: a non-exhaustive match

One argument (illustration)

```
def isZero : Nat → Bool
  | 0 ⇒ true
```

```
error: Missing cases:
(Nat.succ _)
```

Each line after “Missing cases:” is an **uncovered pattern** in raw constructors: (Nat.succ _) means “any n + 1”. Fix: add | _ + 1 => false.

> *Why is it required? Functions in type theory are total: they must have a value at every input.*

Three arguments: bar without the line | 0, [], true => 7

```
def bar : Nat → List Nat → Bool → Nat
  | 0,   _,      false ⇒ 0
  | 0,   b :: _, _     ⇒ b
  | a+1, [],     false ⇒ a
  | a+1, [],     true  ⇒ a + 1
  | a+1, b :: _, _     ⇒ a + b
```

```
error: Missing cases:
Nat.zero, [], _
```

- Columns are arguments; _ means “anything”. Lean may report a **wider** region than needed: line 1 covers (0, [], false), so only (0, [], true) is missing.
- Fix: add the missing line, or a final | _, _, _ => ... line if a default value is really intended.

CHAPTER 8 · §8.3–8.4

Structural recursion and local recursive declarations

Example — a proof by induction written as a recursive function

```
open Nat
-- add is defined just before
theorem zero_add : ∀ n, add zero n = n
  | zero   ⇒ rfl
  | succ n ⇒ congrArg succ (zero_add n)
```

The call zero_add n is on n — a subterm of succ n — so it is accepted.

- **Structural recursion:** every recursive call is on a subterm of the pattern
- A proof by induction is exactly a structurally recursive function
- Implemented via course-of-values recursion: **below**, **brecOn**
- #eval fib 50 runs compiled code (exponential time); #reduce fib 50 uses the brecOn definition in the kernel (fast)
- **let rec** and **where:** local recursive functions; Lean generates auxiliary declarations such as replicate.loop

Structural recursion: add, mul and the proof of zero_add

```
open Nat
def add : Nat → Nat → Nat
  | m, zero   ⇒ m
  | m, succ n ⇒ succ (add m n)

theorem add_zero (m : Nat) : add m zero = m := rfl
theorem add_succ (m n : Nat) :
    add m (succ n) = succ (add m n) := rfl

theorem zero_add : ∀ n, add zero n = n
  | zero   ⇒ rfl
  | succ n ⇒ congrArg succ (zero_add n)

def mul : Nat → Nat → Nat
  | n, zero   ⇒ zero
  | n, succ m ⇒ add (mul n m) n
```

The succ n branch of zero_add

```
⊢ add zero (succ n) = succ n
≡ succ (add zero n) = succ n  -- by definition
zero_add n : add zero n = n
congrArg succ (zero_add n) :
    succ (add zero n) = succ n  ✓
```

> *The call zero_add n on the subterm n is precisely the induction hypothesis.*

A simp version using Lean's generated equation lemmas

```
theorem zero_add : ∀ n, add zero n = n
  | zero   ⇒ by simp [add]
  | succ n ⇒ by simp [add, zero_add]
```

Comparing fib and fibFast: where and let rec

Two recursive calls, both on subterms of n+2

```
def fib : Nat → Nat
  | 0   ⇒ 1
  | 1   ⇒ 1
  | n+2 ⇒ fib (n+1) + fib n

example : fib 0 = 1 := rfl
example : fib 1 = 1 := rfl
example : fib (n + 2) = fib (n + 1) + fib n := rfl
example : fib 7 = 21 := rfl
```

> *Correct, and definitionally so, but it runs in **exponential** time: fib n is recomputed many times.*

loop n returns a pair of two consecutive values

```
def fibFast (n : Nat) : Nat :=
  (loop n).2
where
  loop : Nat → Nat × Nat
    | 0   ⇒ (0, 1)
    | n+1 ⇒ let p := loop n; (p.2, p.1 + p.2)

#eval fibFast 100 -- 573147844013817084101
```

- loop 0 = (0, 1), loop 1 = (1, 1), loop 2 = (1, 2), loop 3 = (2, 3)… — one addition per step: linear time
- Writing **let rec loop …** inside the body gives the same result; both generate the auxiliary declaration **fibFast.loop**

below and brecOn: why is #reduce fib 50 fast?

Course-of-values recursion: constants generated with each inductive type

```
variable (C : Nat → Type u)

#check (@Nat.below C : Nat → Type u)

#reduce @Nat.below C (3 : Nat)

#check (@Nat.brecOn C : (n : Nat) →
  ((n : Nat) → @Nat.below C n → C n) → C n)
```

- **Nat.below C 3:** a data structure storing elements of C 0, C 1, C 2
- **brecOn:** defines the value at n from *all* earlier values, supplied as Nat.below C n

Intuition: fib via brecOn is a table lookup

```
n = 0 : table []          ⇒ fib 0 = 1
n = 1 : table [1]         ⇒ fib 1 = 1
n = 2 : table [1, 1]      ⇒ 1 + 1 = 2
n = 3 : table [1, 1, 2] ⇒ 2 + 1 = 3
n = 4 : look up 3, 2    ⇒ 3 + 2 = 5
```

```
-- Slow:
-- #eval fib 50
-- Fast:
#reduce fib 50
```

#eval runs compiled code with direct recursion (exponential); #reduce reduces the definition sent to the kernel, which is built on brecOn.

Well-founded recursion: termination_by and decreasing_by

Example — the Ackermann function, lexicographic order on Nat × Nat

```
def ack : Nat → Nat → Nat
  | 0,   y   ⇒ y+1
  | x+1, 0   ⇒ ack x 1
  | x+1, y+1 ⇒ ack x (ack (x+1) y)
termination_by x y ⇒ (x, y)
```

> **Warning:** decreasing_by sorry makes Lean “trust” termination — even False becomes provable; #print axioms shows sorryAx.

- Foundations: **Acc r x** (x is accessible), **WellFounded r**, **WellFounded.fix** — hand-written uses are hard to read and do not reduce by rfl
- In practice: Lean tries structural recursion first, then well-founded; **decreasing_tactic** proves the calls decrease, using hypotheses such as have : x - y < x
- **termination_by**: maps the arguments to a type with a WellFoundedRelation
- **decreasing_by**: supply your own tactic for the decrease proof
- Default: (· < ·) on Nat; sizeOfWFRel for types with SizeOf

Non-termination = circular reasoning: a “proof” of False

Making Lean “trust” termination with decreasing_by sorry

```
def unsound (x : Nat) : False :=
  unsound (x + 1)
decreasing_by sorry

#check unsound 0
-- `unsound 0` is a proof of `False`

#print axioms unsound
-- 'unsound' depends on axioms: [sorryAx]
```

> *In ordinary programming, an infinite loop just hangs the program. In logic, it collapses the whole system: False implies every proposition.*

- **Curry–Howard:** unsound has type (x : Nat) → False, i.e. it is a “proof” of ∀ x, False. The equation unsound x = unsound (x + 1) never reaches a value — it only says “False holds because False holds”.
- **Hence** Lean accepts a recursive definition only when it can verify termination: structural recursion, or well-founded recursion with a proof that each call descends along a relation with no infinite descending chain.
- **sorry** skips that check — and leaves sorryAx behind, so #print axioms always detects it.
- **An escape hatch for programming:** partial def accepts a possibly non-terminating function but treats it as an opaque constant: unfold fails, so its equation cannot be used to derive falsehoods.

Well-founded relations: every descending staircase hits the ground

Relation	Trying to descend forever	Well-founded?
< on Nat	`7 > 4 > 1 > 0`	Yes — forced to stop at 0
Lexicographic on Nat × Nat	`(2,0) > (1,99) > (1,5) > (0,9) > (0,0)`	Yes — the first component decreases only finitely often
List length	`[a,b,c] > [b,c] > [c] > []`	Yes
< on Int	`0 > -1 > -2 > -3 > …`	No
< on positive fractions	`1 > 1/2 > 1/4 > 1/8 > …`	No

- A relation $\prec$ is **well-founded** if there is no infinite descending chain $x_0 \succ x_1 \succ x_2 \succ \ldots$
- **Recursion**: if every recursive call uses an argument smaller under $\prec$, the chain of calls is a descending chain — so it must stop.
- **Induction**: to prove P x, we may assume P y for **every** y $\prec$ x (not just y = x - 1).
- For ack, $\prec$ is the lexicographic order: (x, y) decreases even if y shoots up, as long as x decreases.
- Next slide: Acc r x formalizes “every downward path from x hits the ground”; WellFounded r says this for every x.

Logical foundations: Acc, WellFounded and WellFounded.fix

```
variable (α : Sort u)
variable (r : α → α → Prop)

#check (Acc r : α → Prop)
#check (WellFounded r : Prop)
```

Acc r x — an inductive predicate, equivalent to ∀ y, r y x → Acc r y. Read r y x as y ≺ x: x is accessible if every element before it is accessible.

WellFounded r — every element of α is accessible. Then a value can be assigned to each x by first assigning values to all its predecessors.

The well-founded recursion principle

```
noncomputable
def f {α : Sort u}
    (r : α → α → Prop)
    (h : WellFounded r)
    (C : α → Sort v)
    (F : (x : α) → ((y : α) → r y x → C y) → C x) :
    (x : α) → C x :=
WellFounded.fix h F
```

- C is the motive; F is the recipe: knowing the values at every y ≺ x, it builds C x
- With C in Prop, fix is exactly the well-founded **induction** principle
- noncomputable: the code generator does not yet support WellFounded.fix

Division written by hand with WellFounded.fix

```
open Nat

theorem div_lemma {x y : Nat} : 0 < y ∧ y ≤ x → x - y < x :=
  fun h ⟹ sub_lt (Nat.lt_of_lt_of_le h.left h.right) h.left

def div.F (x : Nat) (f : (x₁ : Nat) → x₁ < x → Nat → Nat) (y : Nat) : Nat :=
  if h : 0 < y ∧ y ≤ x then
    f (x - y) (div_lemma h) y + 1
  else
    zero

noncomputable def div := WellFounded.fix (measure id).wf div.F
```

f is the recursive “promise”: division by y is already available for *every* $x_1 < x$.

div_lemma h is evidence that x - y < x — the reason the call f (x - y) is legitimate.

(measure id).wf: < on Nat is well-founded; fix assembles div.F into div.

Common error: div 8 2 = 4 cannot be proved by rfl

```
example : div 8 2 = 4 := by rfl
-- error: Tactic `rfl` failed: The left-hand side
--   div 8 2
-- is not definitionally equal to
-- the right-hand side
--   4
```

Only propositionally true — unfold layer by layer

```
example : div 8 2 = 4 := by
  simp [div, WellFounded.fix, WellFounded.fixF,
        Acc.rec_eq_recC, div.F, Acc.recC]
```

Here div is the WellFounded.fix version from the previous slide: its defining equation holds only propositionally.

In practice: Lean builds the fix itself

```
def div (x y : Nat) : Nat :=
  if h : 0 < y ∧ y ≤ x then
    have : x - y < x :=
      Nat.sub_lt (Nat.lt_of_lt_of_le h.1 h.2) h.1
    div (x - y) y + 1
  else
    0
```

- Lean tries structural recursion first; if that fails, it switches to well-founded
- **decreasing_tactic** uses assumption: have : x - y < x is a hint for it
- Lean generates equation lemmas; unfold with conv => lhs; unfold div, then simp [h]

“failed to prove termination”: reading the three parts of the message

```
error: fail to show termination for
  loop
with errors
failed to infer structural recursion:
Cannot use parameter n:
  failed to eliminate recursive application
    loop (n + 1)
failed to prove termination, possible solutions:
  - Use `have`-expressions to prove …
  - Use `termination_by` to specify …
  - Use `decreasing_by` to specify …
n : Nat
⊢ n + 1 < n
```

The source (illustration)

```
def loop (n : Nat) : Nat := loop (n + 1)
```

1. **Structural recursion** (faded text): Lean tries each parameter; n + 1 is not a subterm of n.
2. **Well-founded recursion**: tries the default measure, then decreasing_tactic; fails — with three suggested fixes.
3. **The remaining goal** (the ⊢ line) — the part most worth reading. Here n + 1 < n is **false**: the function really does not terminate; the definition must change.

If the goal is **true** but Lean cannot see it: add a have proving it (like div on the previous slide), choose another measure with termination_by, or prove it yourself with decreasing_by.

When structural recursion does not apply

Forcing Lean to use structural recursion on n (illustration)

```
def log2 (n : Nat) : Nat :=
  if n < 2 then 0 else log2 (n / 2) + 1
termination_by structural n
```

```
error: failed to infer structural recursion:
Cannot use parameter n:
  failed to eliminate recursive application
    log2 (n / 2)
```

Reading: “Cannot use parameter n” — n cannot serve as the structural parameter, because the call log2 (n / 2) is not on a **subterm** of n. Older Lean versions print the equivalent “structural recursion cannot be used”.

Call (in branch)	Subterm?
`fib n, fib (n+1) (| n+2)`	Yes — n+2 = succ (succ n)
`append as bs (| cons a as)`	Yes — the tail of the list
`log2 (n / 2)`	No — smaller, but not a “piece” of n
`div (x - y) y`	No — needs a proof of x - y < x

Fix: remove the line termination_by structural n. Lean then switches to well-founded recursion on n and proves n / 2 < n itself from the else branch’s condition ¬(n < 2); #eval log2 1024 gives 10.

> *“Smaller” in the arithmetic sense is not enough for structural recursion; it accepts only pieces extracted by pattern matching.*

ack: termination_by and decreasing_by for each call

termination_by picks the measure; decreasing_by proves it decreases

```
def ack : Nat → Nat → Nat
  | 0,   y   ⇒ y+1
  | x+1, 0   ⇒ ack x 1
  | x+1, y+1 ⇒ ack x (ack (x+1) y)
termination_by x y ⇒ (x, y)
decreasing_by
  -- unfold well-founded recursion's auxiliaries
  all_goals simp_wf
  · apply Prod.Lex.left; simp +arith
  · apply Prod.Lex.right; simp +arith
  · apply Prod.Lex.left; simp +arith
```

The WellFoundedRelation instance for Nat × Nat is the **lexicographic** order. For ack, Lean finds it alone, so both clauses can be dropped.

Call: old (x, y) ↦ new	Why it decreases
`ack x 1` (x+1, 0) ↦ (x, 1)	x < x+1: first component decreases — Prod.Lex.left
`ack (x+1) y` (x+1, y+1) ↦ (x+1, y)	first unchanged, y < y+1 — Prod.Lex.right
`ack x (…)` (x+1, y+1) ↦ (x, ack (x+1) y)	x < x+1 — Prod.Lex.left; the second may be arbitrarily large

> *(a, b) < (a', b') lexicographically when a < a', or a = a' and b < b'. No infinite descending chain ⇒ ack terminates.*

Evaluating ack 1 1 step by step, with a decreasing measure

```
def ack : Nat → Nat → Nat
  | 0,   y   ⇒ y+1              -- (1)
  | x+1, 0   ⇒ ack x 1          -- (2)
  | x+1, y+1 ⇒ ack x (ack (x+1) y) -- (3)
termination_by x y ⇒ (x, y)
```

- ack from §8.5, lines numbered. Each step: take the innermost call, find the first matching line, substitute its right-hand side.
- #eval ack 1 1 gives 3, #eval ack 2 2 gives 7.
- But example : ack 1 1 = 3 := by rfl **fails**: the kernel does not unfold well-founded definitions on its own. by simp [ack] works — simp uses Lean's generated equations (1)–(3).

Expression	Next step	New call: why smaller than (1, 1)...
`ack 1 1`	line (3), x = 0, y = 0	ack 1 0: (1, 0) < (1, 1), first equal, 0 < 1; ack 0 ...: 0 < 1 in first
`ack 0 (ack 1 0)`	evaluate ack 1 0 by line (2), x = 0	ack 0 1: (0, 1) < (1, 0), since 0 < 1 in first
`ack 0 (ack 0 1)`	evaluate ack 0 1 by line (1): 2	no new call
`ack 0 2`	line (1): 2 + 1	no new call
`3`	result	—

CHAPTER 8 · §8.6–8.7

Functional induction and mutual recursion

§8.6 — one line for every branch of ack

```
theorem ack_gt_zero : ack n m > 0 := by
  fun_induction ack <;> simp [*, ack]
```

§8.7 — two functions calling each other

```
mutual
  def even : Nat → Bool
    | 0   ⇒ true
    | n+1 ⇒ odd n
  def odd : Nat → Bool
    | 0   ⇒ false
    | n+1 ⇒ even n
end
```

- **Functional induction:** an induction principle that follows the recursive structure of the *function*, not of the data type
- **fun_induction f:** one goal per branch, with an induction hypothesis for each recursive call — nested calls included
- **fun_cases f:** like cases; adds hypotheses ruling out the branches not taken
- **mutual … end:** compiles to one function on a sum type with a well-founded measure; reason with simp, unfold
- Nested types (Term containing List Term): mutual definitions and proofs

Goal states across the branches of fun_induction ack

Each branch corresponds to one line of the ack definition

```
theorem ack_gt_zero : ack n m > 0 := by
  fun_induction ack with
  | case1 y ⇒
    simp
  | case2 x ih ⇒
    exact ih
  | case3 x y ih1 ih2 ⇒
    simp [ack, *]
```

- The ack … side of the goal is already reduced to the right-hand side of the matching line
- One induction hypothesis per recursive call — even nested ones (ih1)
- Induction on n or m alone does not give exactly these hypotheses

```
case case1  — line | 0, y => y+1
y : Nat
⊢ y + 1 > 0

case case2  — line | x+1, 0 => ack x 1
x : Nat
ih : ack x 1 > 0
⊢ ack x 1 > 0

case case3  — line | x+1, y+1 => …
x : Nat
y : Nat
ih1 : ack (x + 1) y > 0
ih2 : ack x (ack (x + 1) y) > 0
⊢ ack x (ack (x + 1) y) > 0
```

fun_cases: hypotheses ruling out skipped branches

Five-way disjunction, with overlapping patterns

```
def f : Bool → Bool → Bool → Bool → Bool → Bool
  | true, _, _, _ , _ ⇒ true
  | _, true, _, _ , _ ⇒ true
  | _, _, true, _ , _ ⇒ true
  | _, _, _, true, _  ⇒ true
  | _, _, _, _, x  ⇒ x

theorem f_or : f b1 b2 b3 b4 b5 =
    (b1 || b2 || b3 || b4 || b5) := by
  fun_cases f <;> simp_all
```

The last branch holds only if b1…b4 are all not true — exactly the information fun_cases adds; simp_all uses it to close every branch.

After fun_cases f (2 of 5 branches; Bool variables on one line)

```
case case2
b1 b3 b4 b5 : Bool
x✝ : b1 = true → False
⊢ true = (b1 || true || b3 || b4 || b5)

case case5
b1 b2 b3 b4 b5 : Bool
x✝³ : b1 = true → False
x✝² : b2 = true → False
x✝¹ : b3 = true → False
x✝ : b4 = true → False
⊢ b5 = (b1 || b2 || b3 || b4 || b5)
```

Reasoning about even/odd and the mutual predicates Even/Odd

With the functions even, odd defined in a mutual … end block

```
example : even (a + 1) = odd a := by
  simp [even]
example : odd (a + 1) = even a := by
  simp [odd]
theorem even_eq_not_odd :
    ∀ a, even a = not (odd a) := by
  intro a; induction a
  . simp [even, odd]
  . simp [even, odd, *]
```

Internally, even and odd compile to **one** function on a sum type (an input of even or of odd) with a well-founded measure; the standard tools are simp or unfold.

With the mutually inductive Even, Odd (§7.9): inverting constructors

```
open Even Odd
theorem not_odd_zero : ¬ Odd 0 :=
  fun h ⟹ nomatch h
theorem even_of_odd_succ :
    ∀ n, Odd (n + 1) → Even n
  | _, odd_succ n h ⟹ h
theorem odd_of_even_succ :
    ∀ n, Even (n + 1) → Odd n
  | _, even_succ n h ⟹ h
```

nomatch h: no constructor produces Odd 0 (odd_succ only gives Odd (n + 1)), so no cases remain.

Dependent pattern matching, inaccessible patterns, match

§8.9 — .(f a) is forced by the type, not matched on

```
def inverse {f : α → β} : (b : β) → ImageOf f b → α
  | .(f a), imf a ⟹ a
```

§8.10 — match anywhere inside an expression

```
def isNotZero (m : Nat) : Bool :=
  match m with
  | 0     ⟹ false
  | m + 1 ⟹ true
```

- **§8.8** For indexed families (Vect α n), case splits constrain the indices; writing them by hand with casesOn is laborious (tailAux, Nat.noConfusion)
- The equation compiler generates head, tail, map, zip automatically, dropping impossible cases such as head nil
- **§8.9** .(_) tells the compiler to infer the index instead of splitting on it
- **§8.10** All pattern matching is a match underneath; fun (m, n) => and let (m, n) := p work the same way
- **§8.11** Further problems: +, *, ^ and their properties; reverse; implementing WellFounded.fix; vector append; an Expr evaluator and constant folding

Why is tail hard to write with Vect.casesOn?

An auxiliary function carrying evidence about the index (in namespace Vect)

```
def tailAux (v : Vect α m) : m = n + 1 → Vect α n :=
  Vect.casesOn (motive := fun x _ ⟹ x = n + 1 → Vect α n) v
    (fun h : 0 = n + 1 ⟹ Nat.noConfusion h)
    (fun (a : α) (m : Nat) (as : Vect α m) ⟹
     fun (h : m + 1 = n + 1) ⟹
       Nat.noConfusion h (fun h1 : m = n ⟹ h1 ▸ as))
def tail (v : Vect α (n+1)) : Vect α n :=
  tailAux v rfl
```

> *The problem: v : Vect α (n + 1) cannot be nil, yet Vect.casesOn still demands a value for the nil branch.*

> **motive** returns a *function* demanding evidence x = n + 1: a way to remember how the index relates to n.

> **nil branch:** the index is 0, with h : 0 = n + 1 — absurd; Nat.noConfusion h eliminates this branch.

> **cons branch:** h : m + 1 = n + 1; noConfusion gives h1 : m = n; h1 ▸ as casts as : Vect α m to Vect α n.

> **tail:** calls tailAux with rfl : n + 1 = n + 1.

The equation compiler does the work: head, tail, eta, zip

```
def head : {n : Nat} → Vect α (n+1) → α
  | n, cons a as ⇒ a
def tail : {n : Nat} → Vect α (n+1) → Vect α n
  | n, cons a as ⇒ as
theorem eta : ∀ {n : Nat} (v : Vect α (n+1)),
    cons (head v) (tail v) = v
  | n, cons a as ⇒ rfl
```

No nil line needed: nil has type Vect α 0, which does not match the index n+1; the compiler removes the impossible case.

{n : Nat} after the ":" because n varies from line to line and cannot be fixed.

eta holds by rfl once v is split into cons a as.

```
def zip : {n : Nat} → Vect α n → Vect β n → Vect (α × β) n
  | 0,   nil,       nil       ⇒ nil
  | n+1, cons a as, cons b bs ⇒ cons (a, b) (zip as bs)
```

Two vectors of the same length n: no line pairing nil with cons. map is similar; #print zipWith shows how complex the generated code is.

Inaccessible patterns .(t): forced by the type, not matched

```
inductive ImageOf {α β : Type u} (f : α → β) :
    β → Type u where
  | imf : (a : α) → ImageOf f (f a)
open ImageOf
def inverse {f : α → β} : (b : β) → ImageOf f b → α
  | .(f a), imf a ⇒ a
def inverse' {f : α → β} : (b : β) → ImageOf f b → α
  | _, imf a ⇒ a
```

> ImageOf f b is evidence that b is in the image of f; imf a proves that f a is in the image.

> Matching imf a forces b to be f a — but f a is neither a variable nor a constructor, so marking it .(f a) is **mandatory**.

> When Lean can infer it, write _ instead of .(...).

```
def add [Add α] : {n : Nat} → Vect α n → Vect α n → Vect α n
  | .(_), nil,       nil       ⇒ nil
  | .(_), cons a as, cons b bs ⇒ cons (a + b) (add as bs)
```

.(_) tells Lean not to split on n but to infer it from nil/cons. Most concise: def add [Add α] : Vect α n → ... with an auto-bound implicit.

match inside an expression, and four ways to write one function

match on two arbitrary expressions, inside an addition

```
def foo (n : Nat) (b c : Bool) :=
  5 + match n - 5, b && c with
      | 0,     true  ⇒ 0
      | m + 1, true  ⇒ m + 7
      | 0,     false ⇒ 5
      | m + 1, false ⇒ m + 3

example : foo 7 true false = 9 := rfl
```

Evaluating foo 7 true false

```
n - 5 = 2 = 1 + 1       ⇒ m = 1
b && c = false
line | m + 1, false    ⇒ 1 + 3 = 4
5 + 4 = 9
```

All pattern matching in Lean reduces to match

```
def bar₁ : Nat × Nat → Nat
  | (m, n) ⇒ m + n

def bar₂ (p : Nat × Nat) : Nat :=
  match p with
  | (m, n) ⇒ m + n

def bar₃ : Nat × Nat → Nat :=
  fun (m, n) ⇒ m + n

def bar₄ (p : Nat × Nat) : Nat :=
  let (m, n) := p; m + n
```

All four definitions give the same result. The same goes for propositions: fun ⟨x, px⟩ ⟨y, qy⟩ => ⟨x, y, px, qy⟩ destructures two ∃ hypotheses.

CHAPTER 8 · CHAPTER SUMMARY

Induction and recursion: the key ideas

1. **Pattern matching**: the equation compiler translates the lines into casesOn and recursors; the first matching line is used; patterns must be exhaustive.
2. **Structural recursion**: calls on subterms; implemented via brecOn; equations hold definitionally. A proof by induction is a recursive function.
3. **Well-founded recursion**: termination_by sets the measure, decreasing_by proves descent; based on Acc, WellFounded.fix; equations are propositional only.
4. **Termination** is a condition for logical soundness; partial is for programming.
5. **fun_induction, fun_cases**: induction along the function's call tree.
6. **Dependent**: matching on inductive families removes impossible cases; .(t) marks patterns forced by the type.

Key syntax	Meaning
`| 0 ⇒ … | n+1 ⇒ …`	Definition by patterns
`let rec, where`	Local recursive function
`termination_by x y ⇒ (x, y)`	Choose the measure
`decreasing_by tac`	Prove the decrease yourself
`fun_induction f`	Functional induction
`mutual … end`	Mutual recursion
`| .(f a), imf a ⇒ a`	Inaccessible pattern
`nomatch h`	No cases remain

CHAPTER 8 · QUESTIONS TO REFLECT ON

Four questions on pattern matching, termination and induction

Question 1

Why does fib (n + 2) = fib (n + 1) + fib n hold by rfl, while ack 1 1 = 3 (or div 8 2 = 4) does not?

Question 2

In ack x (ack (x+1) y), the outer second argument is unbounded — why does the measure decrease? And with termination_by x y => (y, x)?

Question 3

head : Vect α (n+1) → α needs no nil line. Why can a function head : List α → α not drop that line?

Question 4

fun_induction ack gives the third branch two hypotheses, ih1 and ih2. Why does induction on a single variable x not directly yield ih2?

Suggested answers to the four questions

Question 1 — two different implementations

fib is structurally recursive, compiled via brecOn: the kernel reduces it once the argument exposes a constructor. ack and div use WellFounded.fix: reducing them means unfolding the Acc proof, so Lean does not unfold them automatically; their equations become theorems, used via simp [ack] or unfold.

Question 2 — the first component decides

With (x, y), from (x+1, y+1) to (x, …): x < x+1 suffices; the second component is irrelevant. With (y, x), ack x 1 in line (x+1, 0) goes from (0, x+1) up to (1, x): Lean reports failed to prove termination, goal ⊢ 1 < 0.

Question 3 — the information is in the type

nil has type Vect α 0 and cannot match the index n+1, so that case is ruled out (via noConfusion). List α records no length: [] is a valid input and a total function must return something — use Option α or a default.

Question 4 — induction must follow the call tree

ih2 is about ack x (ack (x+1) y): the second argument is an arbitrary value, and ih1 needs induction on y with x+1 fixed. Induction on x must nest an induction on y by hand (with generalizing). fun_induction builds exactly that principle: one ih per recursive call.

PART III · CHAPTER

9

Structures and Records

Records, projections and inheritance

SECTIONS

9.1 Declaring Structures

9.2 Objects

9.3 Inheritance

Structure = non-recursive inductive type with one constructor; the structure command generates projections, supports inheritance and handy value syntax.

A structure is the familiar “record” from programming

Language	Same idea: “a Person has a name and an age”
C	`struct Person { char *name; int age; };`
TypeScript	`interface Person { name: string; age: number }`
Python	`@dataclass class Person: name: str; age: int`
Lean	`structure Person where (name : String) (age : Nat)`

- A structure is a **box with named compartments** (fields); each compartment has a type
- Create = fill every compartment; read one = projection leo.name
- Values are **immutable**: “updating” a field actually creates a new copy

A first example (illustration)

```
structure Person where
  name : String
  age  : Nat

def leo : Person := { name := "Leo", age := 30 }
#eval leo.name  -- "Leo"
#eval { leo with age := 31 }
-- { name := "Leo", age := 31 }
```

> *Where Lean goes far beyond ordinary records: a field can be a **proof** about other fields, e.g. den ≠ 0 — a condition checked at compile time, not at run time.*

Declaring a structure, and dot notation

```
structure Point (α : Type u) where
  mk ::
  x : α
  y : α

def Point.add (p q : Point Nat) :=
  mk (p.x + q.x) (p.y + q.y)

def p : Point Nat := Point.mk 1 2
def q : Point Nat := Point.mk 3 4

#eval p.add q
-- { x := 4, y := 6 }
```

Lean generates

- The type Point and the eliminator Point.rec
- The constructor Point.mk (named mk by default)
- The projections Point.x, Point.y
- A namespace with the same name, Point

- **Dot notation:** p.x means Point.x p
- p.foo x y z inserts p as the *first* argument of type Point: p.smul 3 is Point.smul 3 p
- Likewise xs.map f is List.map f xs; open Point to drop the prefix

What Lean generates from one structure command

The declaration and Lean's #check messages

```
structure Point (α : Type u) where
  mk ::
  x : α
  y : α

#check @Point.mk
-- @Point.mk : {α : Type u_1} → α → α → Point α
#check @Point.x
-- @Point.x : {α : Type u_1} → Point α → α
#check @Point.y
-- @Point.y : {α : Type u_1} → Point α → α
```

General form: structure ‹name› ‹parameters› ‹parent structures› where ‹constructor› :: ‹fields› — most parts are optional.

Generated name	Role
`Point.mk`	The single constructor; without "mk ::" the default name is still mk
`Point.rec`	Eliminator: a function on Point is determined by its value on mk x y — like the recursor in Chapter 7
`Point.x`, `Point.y`	Projections; take a Point α and return the corresponding field
`namespace Point`	Holds all the names above; open Point to write mk, x, y

> *#eval Point.x (Point.mk 10 20) gives 10. Why does example (a b : α) : x (mk a b) = a := rfl hold? A projection applied to a constructor reduces directly, so both sides are definitionally equal.*

General dot notation: p.smul 3 and xs.map f

```
-- Point as before, plus deriving Repr
def Point.smul (n : Nat) (p : Point Nat) :=
  Point.mk (n * p.x) (n * p.y)

def p : Point Nat := Point.mk 1 2

#eval p.smul 3  -- { x := 3, y := 6 }

example {p : Point Nat} :
    p.smul 3 = Point.smul 3 p := rfl
```

> *Rule: for p : Point, p.foo x y z looks up Point.foo, then inserts p as the first explicit argument of type Point; the remaining arguments fill in order.*

Inside the body of def Point.add, the namespace Point is temporarily open, so mk can stand for Point.mk.

Written	Lean reads	Why
`p.x`	`Point.x p`	p is the only explicit argument
`p.add q`	`Point.add p q`	First argument is a Point
`p.smul 3`	`Point.smul 3 p`	n : Nat comes before p
`xs.map f`	`List.map f xs`	xs is the second argument

```
def xs : List Nat := [1, 2, 3]
def f : Nat → Nat := fun x ⟹ x * x
#eval xs.map f  -- [1, 4, 9]
```

deriving Repr tells #eval how to print a Point; recent Lean 4 derives Repr when missing, but stating it is still good practice.

Object syntax and implicit fields

```
structure Point (α : Type u) where
  x : α
  y : α

#check { x := 10, y := 20 : Point Nat }
-- { x := 10, y := 20 } : Point Nat
#check { y := 20, x := 10 : Point _ }
-- { x := 10, y := 20 } : Point Nat
#check ({ x := 10, y := 20 } : Point Nat)
-- { x := 10, y := 20 } : Point Nat

example : Point Nat :=
  { y := 20, x := 10 }
```

```
structure MyStruct where
    {α : Type u}
    {β : Type v}
    a : α
    b : β

#check { a := 10, b := true : MyStruct }
```

- Fields may come in any order; Lean always prints them in declaration order
- Point _: α is inferred to be Nat from the value 10
- Omit “: Point Nat” when the expected type is known, as in the example
- Fields in {...} become implicit parameters of MyStruct.mk; here Lean infers α := Nat, β := Bool
- A field neither assigned nor inferable ⇒ error: the placeholder cannot be synthesized

Updating a record from one or more sources

```
def p : Point Nat :=
  { x := 1, y := 2 }

#eval { p with y := 3 }  -- { x := 1, y := 3 }
#eval { p with x := 4 }  -- { x := 4, y := 2 }

structure Point3 (α : Type u) where
  x : α
  y : α
  z : α

def q : Point3 Nat :=
  { x := 5, y := 5, z := 5 }

def r : Point3 Nat :=
  { p, q with x := 6 }
```

Field of r	Taken from	Value
x	Assigned directly after with	6
y	p — the first source with a field y	2
z	p has no z ⇒ move on to q	5

```
example : r.x = 6 := rfl
example : r.y = 2 := rfl
example : r.z = 5 := rfl
```

> *{ s with … } creates a new record; it does not modify s. Sources are searched in order for each missing field; if none supplies it, Lean reports an error.*

Four common errors when creating objects

For a structure Point with two fields x y : Nat (illustration) — messages exactly as Lean 4.33 prints them

Offending code	Lean reports	Meaning and fix
`({ x := 1 } : Point)`	`Fields missing: `y``	Field y is missing: supply y := ..., or declare a default y : Nat := 0
`{ x := 1, y := 2, z := 3 }`	`` `z` is not a field of structure `Point` ``	A misspelled or extra field name
`(⟨1⟩ : Point)`	`Insufficient number of fields for `⟨...⟩` constructor: Constructor `Point.mk` has 2 explicit field, but only 1 was provided`	⟨...⟩ fills fields in declaration order and needs every explicit field
`def d := { x := 1, y := 2 }`	`invalid {...} notation, expected type is not known`	Lean does not guess the structure from field names: add the type, def d : Point := ...

> *General approach: find the field or constructor name quoted in `...` and compare it with the structure declaration. In recent Lean 4 versions (tested with 4.33), #eval derives Repr when needed, so forgetting deriving Repr no longer causes an error.*

CHAPTER 9 · §9.2–9.3

Objects, record updates and inheritance

- **Creating objects:** { x := 1, y := 2 : Point Nat }; omit the type ascription when known; fields in any order
- Fields marked implicit {...}; omitted fields are inferred when possible
- **Updating:** { p with y := 3 } copies the remaining fields from p; multiple sources { p, q with x := 6 }
- **Inheritance:** extends adds new fields; multiple inheritance is supported; a new field can be a proof about inherited fields

Example — multiple inheritance with a proof field

```
structure RedGreenPoint (α : Type u)
    extends Point α, RGBValue where
  no_blue : blue = 0

def rgp : RedGreenPoint Nat :=
  { p with red := 200, green := 40, blue := 0,
     no_blue := rfl }

example : rgp.x   = 10 := rfl
example : rgp.red = 200 := rfl
```

Here p is a previously declared three-dimensional Point with x := 10.

Inheritance: extends adds fields to a parent structure

```
structure Point (α : Type u) where
  x : α
  y : α

inductive Color where
  | red | green | blue

structure ColorPoint (α : Type u)
    extends Point α where
  c : Color
```

Trying it out (illustration)

```
def cp : ColorPoint Nat :=
  { x := 1, y := 2, c := Color.red }
#eval cp.x         -- 1
#eval cp.toPoint.y -- 2
```

- **extends Point α:** ColorPoint has the inherited fields x, y and a new field c; internally, the parent part is kept as a field toPoint : Point α
- Lean generates the projection ColorPoint.toPoint; cp.x is shorthand for cp.toPoint.x
- When creating an object, list all fields flat, as if on one level
- **Multiple parents:** extends Point α, RGBValue merges both
- A new field can be a *proof* about inherited fields: no_blue : blue = 0
- A parent-structure object can serve as a source: { p with red := 200, ... } takes x, y, z from p; no_blue := rfl works since blue := 0 turns the goal into 0 = 0

CHAPTER 9 · BACKGROUND

Algebraic structures: a set + operations + laws

Semigroup: a set S, an operation ·, and associativity (a · b) · c = a · (b · c)

Monoid: a semigroup with an identity element e such that e · a = a and a · e = a

- Modern mathematics calls such a tuple (set, operations, laws) an **algebraic structure**
- More laws give richer structures: Monoid + inverses = Group; Monoid + commutativity = CommMonoid
- The payoff: prove **once** for every monoid, reuse for every concrete example — like generic programming
- In Lean: operations are data fields, laws are proof fields — exactly a structure

Tuple	What is it?
`(Nat, +, 0)`	Monoid
`(Nat, *, 1)`	Monoid
`(String, ++, "")`	Monoid (not commutative: "ab" ≠ "ba")
`(Int, +, 0)`	Group: has inverses −a
`(positives, +)`	Semigroup, no identity
`(Nat, -)`	Not a semigroup: (5 − 3) − 1 = 1 but 5 − (3 − 1) = 2

> *One set can carry several structures: Nat is a monoid under + and also a monoid under *. So a structure is data attached to a set, not a property of the set alone.*

A small hierarchy: Semigroup → Monoid via extends

Declarations (illustration; the ' avoids clashing with library names)

```
structure Semigroup' (α : Type) where
  mul : α → α → α
  mul_assoc : ∀ a b c,
    mul (mul a b) c = mul a (mul b c)

structure Monoid' (α : Type)
    extends Semigroup' α where
  one : α
  one_mul : ∀ a, mul one a = a
  mul_one : ∀ a, mul a one = a
```

mul_assoc, one_mul, mul_one are **proof** fields: a Monoid' cannot be built from an operation that breaks the laws.

A concrete instance: (Nat, +, 0)

```
def natAdd : Monoid' Nat where
  mul := Nat.add
  mul_assoc := Nat.add_assoc
  one := 0
  one_mul := Nat.zero_add
  mul_one := Nat.add_zero

#eval natAdd.mul 3 4  -- 7
#check natAdd.toSemigroup'
-- natAdd.toSemigroup' : Semigroup' Nat
```

- List all fields flat, including the parent's; Lean packs the parent part into toSemigroup'
- Nat.add_assoc has exactly the type mul_assoc requires, so it serves directly as the proof

One theorem for every monoid: the identity is unique

Using Monoid' from the previous slide (illustration)

```
theorem Monoid'.one_unique {α : Type}
    (M : Monoid' α) (e : α)
    (h : ∀ a, M.mul e a = a) :
    e = M.one := by
  have h1 := h M.one
  rw [M.mul_one] at h1
  exact h1
```

- Statement: if e is also a left identity, then e is exactly M.one
- M.mul_one : ∀ a, M.mul a M.one = a — a law taken from a field of M
- Applies at once: natAdd.one_unique for Nat with +; one proof, every monoid

Initially

```
α : Type
M : Monoid' α
e : α
h : ∀ (a : α), M.mul e a = a
⊢ e = M.one
```

After have h1 := h M.one — specialize to a := M.one

```
h1 : M.mul e M.one = M.one
```

After rw [M.mul_one] at h1 — M.mul e M.one becomes e

```
h1 : e = M.one
⊢ e = M.one  — exact h1 closes the goal
```

CHAPTER 9 · CHAPTER SUMMARY

Structures and Records on one page

- Structure = inductive type, **non-recursive, one constructor**; Lean generates constructor, recursor, projections, namespace
- Dot notation p.f looks up S.f and inserts p as the first argument of type S
- Immutable values: { p with … } creates a new record, possibly drawing fields from several sources
- extends merges the fields of one or more parents; each parent part lives in a field toParent
- Fields can be proofs ⇒ a structure can describe an **algebraic structure:** data + laws

Syntax	Meaning
`structure S where f : T`	Declaration
`S.mk a b, ⟨a, b⟩`	Constructor, in field order
`{ f := a, g := b }`	Create an object, any order
`p.f`	Projection, dot notation
`{ p, q with f := a }`	Update from sources
`extends A, B`	Inheritance; projections toA, toB
`f : T := v`	Field with a default value
`deriving Repr`	Generates printing for #eval

CHAPTER 9 · QUESTIONS TO REFLECT ON

Four questions to check your understanding

1. A structure is just a special case of inductive. If we wrote Point as inductive Point (α : Type u) | mk : α → α → Point α, what would we lose?

2. Does { p with y := 3 } “modify” p? How does this help in reasoning about programs?

3. How does a field no_blue : blue = 0 differ from an assert(blue == 0) in an ordinary programming language?

4. Monoid' extends Semigroup'. How can a theorem proved for every Semigroup' be used for natAdd : Monoid' Nat?

> *Try to answer before the next slide; questions 1 and 4 connect this chapter with Chapters 7 and 10.*

Suggested answers to the four questions

1. The type and recursor are the same, so everything is still definable. But we must write the projections x, y ourselves with match, and we lose the syntax { x := … }, { p with … }, extends and default values.

2. No: it creates a new value; p is unchanged. Nothing changes implicitly, so a name always denotes one value and rewriting with equalities (rw, rfl) is safe.

3. assert only detects violations *at run time*, and costs run time. no_blue is checked *at compile time*: a violating value cannot be built; the proof lives in Prop, so it is erased during code generation at no cost.

4. Apply the theorem to natAdd.toSemigroup' — the projection generated by extends. In Chapter 10, once Semigroup' and Monoid' are classes, Lean finds this projection itself; no need to write it.

PART IV · CHAPTER

10

Type Classes

Ad-hoc polymorphism and instance search

SECTIONS

10.1 Chaining Instances

10.2 ToString

10.3 Numerals

10.4 Output Parameters

10.5 Default Instances

10.6 Local Instances

10.7 Scoped Instances

10.8 Decidable Propositions

10.9 Managing Type Class Inference

10.10 Coercions using Type Classes

Type classes are like Java interfaces and Rust traits

Task	Java interface	Rust trait	Lean class
Declare a “capability”	`interface Show { String show(); }`	`trait Show { fn show(&self) → String; }`	`class ToString (α : Type u) where toString : α → String`
Implement for a type	In the class definition: class Point implements Show	Anywhere: impl Show for Point	Anywhere: instance : ToString Point where …
Generic function	`<T extends Show>`	`fn f<T: Show>`	`[ToString α]`
Who picks the implementation	Usually at run time, by object	The compiler, by type	The elaborator, by type, via search

Alike: separate “which capability is needed” from “how each type does it”; a function written once works for every type with an implementation.

Difference 1: a class can also hold **laws** as proof fields, e.g. associativity — an interface only promises function signatures.

Difference 2: instances can be **conditional**: if α is printable, so is List α — Lean assembles instances into a tree.

CHAPTER 10 · INTRODUCTION

From structure to class: ad-hoc polymorphism

1. Pass the implementation by hand: structure Add α with a field add, call double { add := Nat.add } 10 — correct but verbose
2. Turn structure into **class**: the argument becomes *instance implicit* — @Add.add : {α : Type} → [self : Add α] → α → α → α
3. Declare an **instance**; Lean finds it by type class resolution; instances can depend on others: [Add α] ⇒ Add (Array α)
4. **Inhabited** supplies a default element default — e.g. the head of an empty list

Example — class, instance and the argument [Add α]

```
class Add (α : Type) where
  add : α → α → α

instance : Add Nat where
  add := Nat.add

def double [Add α] (x : α) : α :=
  Add.add x x
```

Compared: passing implementations by hand vs. classes

Step 1 — structure: the implementation is an explicit argument

```
structure Add (α : Type) where
  add : α → α → α

#check @Add.add
-- @Add.add : {α : Type} → Add α → α → α → α

def double (s : Add α) (x : α) : α :=
  s.add x x

#eval double { add := Nat.add } 10 -- 20
#eval double { add := Nat.mul } 10 -- 100
#eval double { add := Int.add } 10 -- 20
```

Step 2 — class: the implementation is instance implicit

```
class Add (α : Type) where
  add : α → α → α

#check @Add.add
-- @Add.add : {α : Type} → [self : Add α] →
--   α → α → α

def double [Add α] (x : α) : α :=
  Add.add x x
#eval double 10 -- 20
#eval double (7 : Float) -- 14.000000
```

> *The only difference: (s : Add α) becomes [self : Add α], so @double : {α : Type} → [Add α] → α → α; Lean finds the instances Add Nat, Add Float itself — like add :: Add a => a -> a -> a in Haskell.*

CHAPTER 10 · LINK TO CHAPTER 9

The hierarchy Semigroup' → Monoid' written with classes

As on the Chapter 9 slide, with structure changed to class (illustration)

```
class Semigroup' (α : Type) where
  mul : α → α → α
  mul_assoc : ∀ a b c,
    mul (mul a b) c = mul a (mul b c)

class Monoid' (α : Type)
    extends Semigroup' α where
  one : α
  one_mul : ∀ a, mul one a = a
  mul_one : ∀ a, mul a one = a
```

- def natAdd becomes an instance (no name needed); the explicit M : Monoid' α becomes [Monoid' α]
- extends on a class also registers Monoid'.toSemigroup' as an instance: Monoid' α gives Semigroup' α

```
instance : Monoid' Nat where
  mul := Nat.add
  mul_assoc := Nat.add_assoc
  one := 0
  one_mul := Nat.zero_add
  mul_one := Nat.add_zero

def mconcat {α : Type} [Monoid' α] :
    List α → α
  | [] ⇒ Monoid'.one
  | x :: xs ⇒ Semigroup'.mul x (mconcat xs)

#eval mconcat [1, 2, 3, 4]  -- 10
```

Semigroup'.mul needs Semigroup' Nat: Lean finds it via Monoid'.toSemigroup' and the instance above. Mathlib builds Monoid, Group, Ring… exactly this way.

Instance resolution = querying a database of facts

Kind	Entry in the "instance database"
Fact	Monoid' Nat (instance on the previous slide)
Fact	Inhabited Nat, Inhabited Bool, ToString String
Rule	Monoid' α ⇒ Semigroup' α (from extends)
Rule	Inhabited α and Inhabited β ⇒ Inhabited (α × β)
Rule	ToString α ⇒ ToString (List α)

- Each instance is an entry; […] in an instance is a **condition** of the rule
- Query: "is there a Semigroup' Nat?" — Lean finds a rule whose conclusion matches, then queries its conditions
- The answer is not just yes/no but **the instance term itself**, inserted at […]

Ask Lean with #synth (using the previous slide's declarations)

```
#synth Semigroup' Nat
-- instMonoid'Nat.toSemigroup'
```

Abridged from set_option trace.Meta.synthInstance true

```
✅ Semigroup' Nat
  apply @Monoid'.toSemigroup'
  ✅ new goal Monoid' Nat
    apply instMonoid'Nat
result instMonoid'Nat.toSemigroup'
```

instMonoid'Nat is the name Lean gives the anonymous instance. Three possible outcomes: found; not found (failed to synthesize); or the query lacks information (stuck) — the next two slides.

“failed to synthesize”: a complete query with no answer

Only the instance Monoid' Nat exists (illustration)

```
#eval mconcat ["a", "b"]
```

Lean reports

```
error: failed to synthesize instance
  of type class
  Monoid' String

Hint: Type class instance resolution
failures can be inspected with the
`set_option trace.Meta.synthInstance true`
command.
```

The indented line (in yellow) is the failed query; it contains no ?m, so Lean really searched and found nothing.

Cause	Fix
No instance for that type yet	Declare instance : Monoid' String where …
Instance exists but is not active	a local instance ended with its section; a scoped instance needs open scoped or open namespace
Type hidden behind a def, like Set α	instance : … := inferInstanceAs …
Forgot to import a library	Add the import that holds the instance
Operation on a custom type: p + p with p : Point	Lean reports HAdd Point Point ?m — needs an instance Add Point

> *Not sure why? Turn on the trace as suggested, or ask #synth Monoid' String directly to test each subquery.*

“instance problem is stuck”: the query still has a hole ?m

```
example := mconcat []
```

Lean reports (excerpt)

```
error: typeclass instance problem
  is stuck
  Monoid' ?m.6

Note: Lean will not try to resolve
this typeclass instance problem
because the type argument to
`Monoid'` is a metavariable. …
```

?m.6 is a **metavariable**: a hole Lean does not know yet; the number after the dot is just a counter that varies with context and version.

- [] can be a List of any type, so mconcat’s α is undetermined
- Lean **refuses to guess**: picking an arbitrary instance would fix ?m wrongly and cause obscure errors elsewhere
- Unlike “failed”: here Lean *never searched*, as the query lacks information

```
-- three ways to supply the type, all print 0
#eval mconcat ([] : List Nat)
#eval (mconcat [] : Nat)
#eval @mconcat Nat _ []
```

> *Class designers have two more ways to get unstuck: outParam and @[default_instance] — §10.4–10.5.*

CHAPTER 10 · §10.1–10.3

Chaining instances, ToString and polymorphic numerals

§10.1 — a product is inhabited if both components are

```
instance [Inhabited α] [Inhabited β] :
    Inhabited (α × β) where
  default := (default, default)
#eval (default : Nat × Bool) -- (0, true)
```

§10.3 — numerals for a user-defined type

```
structure Rational where
  num : Int
  den : Nat
  inv : den ≠ 0
instance : OfNat Rational n where
  ofNat := { num := n, den := 1, inv := by decide }
```

- Instance search is recursive, with backtracking — Prolog-style
- **inferInstance** resolves when the expected type is an instance
- **ToString:** declare it for your type ⇒ compound types (pairs, lists) print too
- Polymorphic numerals: (2 : Nat) is @OfNat.ofNat Nat 2 _; the “raw” nat_lit 2 is not polymorphic
- OfNat takes the numeral as a parameter ⇒ per-numeral instances

Instance resolution: a Prolog-style search

```
instance [Inhabited α] [Inhabited β] :
    Inhabited (α × β) where
  default := (default, default)

instance [Inhabited β] : Inhabited (α → β) where
  default := fun _ ⟹ default

#eval (default : Nat × Bool) -- (0, true)

def bar : Inhabited (Nat → Nat) := inferInstance
```

> *Read an instance as a Prolog rule: "Inhabited (α × β) if Inhabited α and Inhabited β". […] are conditions; after ":" comes the conclusion.*

inferInstance : {α : Sort u} → [i : α] → α just returns the instance found — used when the expected type is itself a class.

```
Goal 1: Inhabited (Nat × Bool)
Inhabited (Nat × Bool)
  ← product rule, α := Nat, β := Bool
  ├ Inhabited Nat  ← default := 0 ✓
  └ Inhabited Bool ← default := true ✓
  ⇒ default = (0, true)
Goal 2: Inhabited (Nat → Nat)
Inhabited (Nat → Nat)
  ← function rule, α := Nat, β := Nat
  └ Inhabited Nat  ← default := 0 ✓
  ⇒ default = fun _ ⟹ 0
```

Each condition […] becomes a subgoal, solved recursively. If a branch fails, Lean backtracks and tries another instance.

ToString: one instance, every compound type prints

```
structure Person where
  name : String
  age  : Nat

instance : ToString Person where
  toString p := p.name ++ "@" ++ toString p.age

#eval toString { name := "Leo", age := 542 : Person } -- "Leo@542"
#eval toString ({ name := "Daniel", age := 18 : Person }, "hello") -- "(Daniel@18, hello)"
```

Resolution for the second #eval

```
ToString (Person × String)
  ├ ToString Person ← instance above ✓
  └ ToString String ← built in ✓
```

- toString p.age uses the built-in instance ToString Nat
- The pair prints thanks to the library instance for α × β, with conditions [ToString α] [ToString β]
- Result: lists, pairs, Option… containing Person all print with no extra code

Polymorphic numerals: how a number is elaborated

```
structure Rational where
  num : Int
  den : Nat
  inv : den ≠ 0

instance : OfNat Rational n where
  ofNat := { num := n, den := 1, inv := by decide }

instance : ToString Rational where
  toString r := s!"{r.num}/{r.den}"

#eval (2 : Rational) -- 2/1
#check (2 : Rational) -- 2 : Rational
#check (2 : Nat)      -- 2 : Nat
```

Written	Lean elaborates to
`(2 : Nat)`	`@OfNat.ofNat Nat 2 (@instOfNatNat 2)`
`(2 : Rational)`	`@OfNat.ofNat Rational 2 (@instOfNatRational 2)`
`nat_lit 2`	“Raw” 2 : Nat — not polymorphic

- The 2 inside the terms above is a raw numeral; OfNat’s second parameter is that numeral, so “OfNat Rational n” covers every n
- inv := by decide: the goal 1 ≠ 0 is decidable, so decide solves it
- One-numeral instance: instance [s : Monoid α] : OfNat α (nat_lit 1) with ofNat := s.unit — then def getUnit [Monoid α] : α := 1 works

Controlling resolution: outputs, defaults, scope

Mechanism	Syntax	Meaning
Output parameter	`HMul α β (γ : outParam Type)`	Search can start with γ unknown; its value is read off the instance found
Semi-output	`semiOutParam`	Both selects the instance and can fill in an unknown value
Default instance	`@[default_instance]`	Used when the problem is "stuck"; OfNat Nat n is the default (priority 100)
Local instance	`local instance`	In effect until the end of the section, namespace or file
Disable instance	`attribute [-instance] addPoint`	Best used only for diagnosis
Scoped instance	`scoped instance, open scoped Point`	Active only inside the namespace or when it is opened

> *By default, resolution runs only once all class arguments are known — otherwise you get "typeclass instance problem is stuck". outParam and default instances are two ways to get unstuck.*

outParam: the result type is read off the instance

```
class HMul (α : Type u) (β : Type v)
    (γ : outParam (Type w)) where
  hMul : α → β → γ

export HMul (hMul)

instance : HMul Nat Nat Nat where
  hMul := Nat.mul

instance : HMul Int Int Int where
  hMul := Int.mul

instance [HMul α β γ] :
    HMul α (Array β) (Array γ) where
  hMul a bs := bs.map (fun b ⟹ hMul a b)
```

#eval	γ read off	Result
`hMul 4 3`	Nat	`12`
`hMul 4 #[2, 3, 4]`	Array Nat	`#[8, 12, 16]`
`hMul 2 #[#[2, 3], #[0, 4]]`	Array (Array Nat)	`#[#[4, 6], #[0, 8]]`

γ is an outParam: ignored during search, its value read off the instance found. The last line uses the array instance twice.

Compare: Inhabited has only input parameters

```
#eval (inferInstance : Inhabited (Nat × _))
error: typeclass instance problem is stuck,
  it is often due to metavariables
  Inhabited (Nat × ?m.2)
```

Getting unstuck with a default instance

Before — HMul as on the previous slide, with only an Int instance

```
instance : HMul Int Int Int where
  hMul := Int.mul

def xs : List Int := [1, 2, 3]

#eval fun y ⇒ xs.map (fun x ⇒ hMul x y)
```

```
error: typeclass instance problem is stuck
  HMul Int ?m.2 (?m.11 y)
```

y has no type yet ⇒ β is the metavariable ?m.2; β is an input parameter, so Lean refuses to start searching. Lean's hint: add a type ascription, e.g. (y : Int).

After — mark the instance as the default

```
@[default_instance]
instance : HMul Int Int Int where
  hMul := Int.mul

#check fun y ⇒ xs.map (fun x ⇒ hMul x y)
-- fun y ⇒ List.map (fun x ⇒ hMul x y) xs
--   : Int → List Int
```

The problem is stuck ⇒ Lean tries the default instance ⇒ β := Int, so y : Int.

> *Numerals likewise: OfNat Nat n is a default instance (priority 100), so 2 : Nat. Tag the instance OfNat Rational n with @[default_instance 200] and #check 2 prints 2 : Rational.*

Local instances and namespace-scoped instances

```
section
local instance : Add Point where
  add a b := { x := a.x + b.x, y := a.y + b.y }

def double (p : Point) :=
  p + p

end -- instance `Add Point` is not active anymore
def triple (p : Point) :=
  p + p + p
```

```
error: failed to synthesize instance of type class
  HAdd Point Point ?m.5
```

attribute [-instance] addPoint likewise temporarily disables a named instance — best used only for diagnosis.

```
namespace Point
scoped instance : Add Point where
  add a b := { x := a.x + b.x, y := a.y + b.y }

def double (p : Point) :=
  p + p

end Point

open scoped Point -- activate instance
#check fun (p : Point) ⇒ p + p + p
```

Where	Scoped instance
Outside namespace Point	Off — p + p + p fails as on the left
`open Point`	On, and also opens the name double
`open scoped Point`	Only the instance; double stays hidden

Decidable propositions and managing inference

Decidable (version in namespace Hidden) and decide

```
class inductive Decidable (p : Prop) where
  | isFalse (h : ¬p) : Decidable p
  | isTrue  (h : p)  : Decidable p

example : 10 < 5 ∨ 1 > 0 := by decide
```

> **§10.9 Debugging:** set_option trace.Meta.synthInstance true; limits via synthInstance.maxHeartbeats, synthInstance.maxSize

- Decidable is stronger than p ∨ ¬p: it can be used to **define data**; if c then t else e is ite, if h : c … is dite
- Decidable: =, < on Nat and Int; preserved by ∧, ∨, ¬
- open Classical: every proposition is decidable via **propDecidable** — noncomputable, priority := low
- **decide** runs the decision procedure; of_decide_eq_true turns decide p = true into a proof of p
- **Search order:** instances declared later are tried first; change it with instance (priority := default + 1)
- inferInstanceAs when the class is hidden behind a definition like Set α

Why if-then-else needs Decidable: ite and dite

Definitions (version in namespace Hidden)

```
def ite {α : Sort u}
    (c : Prop) [h : Decidable c]
    (t e : α) : α :=
  h.casesOn (motive := fun _ ⇒ α)
    (fun _ ⇒ e) (fun _ ⇒ t)

def dite {α : Sort u}
    (c : Prop) [h : Decidable c]
    (t : c → α) (e : Not c → α) : α :=
  Decidable.casesOn (motive := fun _ ⇒ α) h e t

def step (a b x : Nat) : Nat :=
  if x < a ∨ x > b then 0 else 1
```

- **Decidable p is data** (isTrue h or isFalse h), so case analysis on it yields a value of any type; p ∨ ¬p can only be eliminated into Prop
- **if c then t else e** is ite c t e: casesOn on h — the isFalse branch returns e, the isTrue branch t
- **if h : c then t else e** is dite c (fun h : c => t) (fun h : ¬c => e): the then branch has h : c, the else branch h : ¬c
- Decidability is preserved by connectives: instDecidableAnd : [Decidable p] → [Decidable q] → Decidable (p ∧ q); likewise for ∨, ¬
- In step, Lean assembles instances for < on Nat and for ∨ — see them with set_option pp.explicit true, then #print step

The decide tactic: proof by computation

```
example : 10 < 5 ∨ 1 > 0 := by
  decide

example : ¬(True ∧ False) := by
  decide

example : 10 * 20 = 200 := by
  decide

theorem ex : True ∧ 2 = 1 + 1 := by
  decide
```

#check @of_decide_eq_true and #check @decide

```
∀ {p : Prop} [inst : Decidable p],
  decide p = true → p
decide : (p : Prop) → [h : Decidable p] → Bool
```

What decide does, step by step

1. Find an instance Decidable p — for 10 < 5 ∨ 1 > 0: the instance for ∨ built from two instances for < on Nat
2. Evaluate decide p : Bool by definitional reduction; a closed, true proposition yields true
3. Then decide p = true holds by rfl, and of_decide_eq_true turns it into a proof of p

Common error: the proposition has free variables (x < y with x, y unknown) or the instance does not compute — decide fails as it cannot reduce to true

Curry–Howard: the proof is a Bool computation checked by the kernel, not a hand-written chain of reasoning

Which instance is tried first? Priorities

```
class Foo where
  a : Nat
  b : Nat

instance (priority := default + 1) i1 : Foo where
  a := 1
  b := 1

instance i2 : Foo where
  a := 2
  b := 2

example : Foo.a = 1 :=
  rfl
```

```
instance (priority := default + 2) i3 : Foo where
  a := 3
  b := 3

example : Foo.a = 3 :=
  rfl
```

- Without priorities, the instance declared **later** is tried first (across modules: the namespace opened later wins) — i2 would beat i1
- i1 has priority default + 1 ⇒ beats i2, so Foo.a is i1.a, which reduces to 1 and rfl holds
- i3 has default + 2 ⇒ from here on Foo.a = 3
- Conversely, Classical's propDecidable has priority := low: used only when every other Decidable instance fails, so if on a decidable proposition still yields runnable code

Coercions via type classes

Class	Used for	Example
`Coe α β`	Ordinary coercion	Coe Bool Prop, Coe (List α) (Set α)
`CoeDep`	Value-dependent coercion	Decidable proposition → Bool
`CoeT`	Transitive closure of Coe	Chains several coercion steps
`CoeSort`	View a structure as a type	(a b c : S) with S : Semigroup means elements of S.carrier
`CoeFun`	Apply a value as a function	f (a * b) with f : Morphism S1 S2 means f.mor (a * b)

```
instance : CoeSort Semigroup (Type u) where
  coe s := s.carrier
```

Three kinds of coercion: between type families, to a sort, to a function type. The ↑ notation forces a coercion at a specific position.

Coe: where Lean inserts coercions

Set α := α → Prop, with Set.empty, {a}, ∪ defined earlier

```
instance : Coe Bool Prop where
  coe b := b = true

#eval if true then 5 else 3
#eval if false then 5 else 3

def List.toSet : List α → Set α
  | []    ⟹ Set.empty
  | a::as ⟹ {a} ∪ as.toSet

instance : Coe (List α) (Set α) where
  coe a := a.toSet

def s : Set Nat := {1}
```

```
#check s ∪ [2, 3]
s ∪ [2, 3].toSet : Set Nat
#check let x := ↑[2, 3]; s ∪ x
let x := [2, 3].toSet; s ∪ x : Set Nat
#check let x := [2, 3]; s ∪ x
let x := [2, 3]; s ∪ x.toSet : Set Nat
```

- List Nat differs from the expected Set Nat and a Coe instance exists ⇒ Lean inserts coe; true in if becomes true = true
- ↑ forces the coercion right there: x is already a Set Nat; without ↑, x is a List Nat and the coercion happens at the use site
- CoeDep Prop p Bool needs [Decidable p]; CoeT chains Coe steps
- The Nat → Int coercion from earlier chapters is implemented with these same classes

CoeSort and CoeFun: structures used as types, as functions

```
structure Semigroup where
  carrier : Type u
  mul : carrier → carrier → carrier
  mul_assoc (a b c : carrier) :
    mul (mul a b) c = mul a (mul b c)

instance (S : Semigroup) : Mul S.carrier where
  mul a b := S.mul a b

instance : CoeSort Semigroup (Type u) where
  coe s := s.carrier

example (S : Semigroup) (a b c : S) :
    (a * b) * c = a * (b * c) :=
  Semigroup.mul_assoc _ a b c
```

```
structure Morphism (S1 S2 : Semigroup) where
  mor : S1 → S2
  resp_mul : ∀ a b : S1,
    mor (a * b) = (mor a) * (mor b)

instance (S1 S2 : Semigroup) :
    CoeFun (Morphism S1 S2) (fun _ ⟹ S1 → S2) where
  coe m := m.mor
```

- **CoeSort:** (a b c : S) means a b c : S.carrier; the instance Mul S.carrier lets us write a * b
- **CoeFun:** f (a * b) means f.mor (a * b); the parameter (fun _ => S1 → S2) is the target function type
- Hence f (a * a * a) = f a * f a * f a is proved by calc and rw [resp_mul f]

CHAPTER 10 · CHAPTER SUMMARY

Type Classes on one page

- Class = a structure whose argument of that type is **found automatically**; instance = an entry in the search database
- Resolution: recursive over conditions […], with backtracking, by priority; the result is a term
- Search needs known inputs; outParam and default instances unstick it
- Decidable p is data: if c then … and decide both rely on it
- Coercions (Coe, CoeSort, CoeFun) are type classes too

Syntax	Meaning
`class C (α : Type) where …`	Declare a class
`instance : C Nat where …`	Add a fact
`instance [C α] : C (List α)`	Add a conditional rule
`def f [C α] …`	Instance-implicit argument
`inferInstance, #synth C T`	Ask Lean for an instance
`outParam, @[default_instance]`	Control the search
`local / scoped instance`	Limit the scope
`(priority := default + 1)`	Change the trial order
`trace.Meta.synthInstance`	Debug the search

CHAPTER 10 · QUESTIONS TO REFLECT ON

Four questions about type classes

1. How do (s : Add α) and [Add α] differ? When should we keep a hand-passed structure instead of a class?

2. Why does Lean refuse to search when the query still contains ?m, instead of just trying instances one by one?

3. Why does if c then t else e require Decidable c, while Classical.em c : c ∨ ¬c is still not enough to run the if?

4. Two different instances match the same query: which does Lean pick, and how can that make rfl fail?

Suggested answers to the four questions

1. Only *who* supplies the argument differs: author or search. Keep a structure when a type has several equal implementations to pick explicitly — Nat is a monoid under + and *; use a class when each type has one “canonical” one.

2. Matching an instance *assigns* a value to ?m. Assigning via an arbitrarily chosen instance may contradict the author’s intent and cause errors elsewhere that are hard to trace. outParam marks where a hole is allowed.

3. Running an if must *compute* which branch to take. Classical.em is a proof in Prop, erased during code generation and containing no algorithm; Decidable c is isTrue/isFalse data the program can read.

4. Higher priority wins; at equal priority, the instance declared later is tried first. If two paths give terms that are not definitionally equal, two expressions that print identically may still not be provable equal by rfl.

PART IV · CHAPTER

11

The Conversion Tactic Mode

Targeted rewriting with conv

SECTIONS

11.1 Basic navigation and rewriting

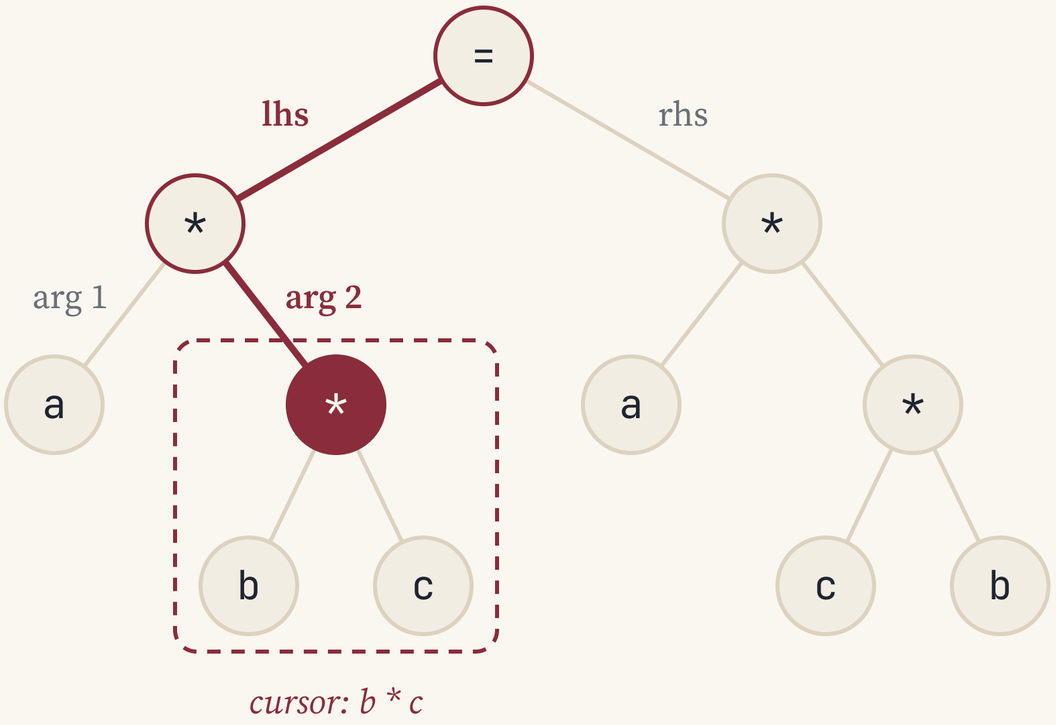
11.2 Pattern matching

11.3 Structuring conversion tactics

11.4 Other tactics inside conversion mode

conv goes inside a goal or hypothesis — even under fun binders and dependent arrows — to rewrite or simplify exactly the selected part.

conv is a cursor moving through the expression tree



*The goal a * (b * c) = a * (c * b) drawn as a tree*

- Each operation is a node, its arguments are child nodes; constants and variables are leaves
- conv => puts the cursor at the root: the whole goal
- lhs goes down the left branch; arg 2 goes down to the second argument — the cursor stops at b * c
- rw and simp can now change only the **subtree in the box**; the rest is left untouched
- At the end of the conv block, Lean reassembles the tree and proves the old goal from the new one

> *Compare: rw is like "Find and replace" on the whole text — it changes the first match; conv is like placing the mouse cursor on exactly the right spot before editing.*

CHAPTER 11 · BACKGROUND

The real tree Lean sees: explicit and implicit arguments

Printing every implicit argument to see it (illustration)

```
set_option pp.explicit true in
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  conv ⇒
    lhs
    trace_state
    arg 2
    rw [Nat.mul_comm]
```

trace_state prints (line breaks added to fit)

```
| @HMul.hMul Nat Nat Nat
    (@instHMul Nat instMulNat) a
    (@HMul.hMul Nat Nat Nat … b c)
```

- a * x is really HMul.hMul with **6 arguments**: three types, an instance (implicit) and a, x (explicit)
- arg i counts **only explicit arguments**: arg 1 is a, arg 2 is b * c; arg 3 is an error
- arg @i counts all arguments: arg @6 also reaches b * c
- = is a node too: @Eq Nat lhs rhs; lhs is just arg 1, rhs is arg 2
- fun x => … is a binder node: intro x enters the function body

conv command	Movement in the tree
`lhs, rhs, arg i`	Down to a child node
`intro x`	Into a binder
`enter [1, x, 2]`	A whole path in one go
`pattern p`	Jump to the first node matching p

Targeted navigation and rewriting

*Rewrite only b * c, leaving a * … alone*

```
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  conv ⇒
    lhs
    congr
    rfl
    rw [Nat.mul_comm]
```

Rewriting under a fun binder

```
example :
    (fun x : Nat ⇒ 0 + x) = (fun x ⇒ x) := by
  conv ⇒
    lhs
    intro x
    rw [Nat.zero_add]
```

Problem: rw [Nat.mul_comm] rewrites the first product it finds, giving b * c * a = …

lhs / rhs enter the two sides; **congr** splits by explicit argument; **rfl** skips a goal

conv at h rewrites a hypothesis; the right example also yields to funext or simp

The goal state through each conv step

```
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  conv ⇒
    lhs
    congr
    rfl
    rw [Nat.mul_comm]
```

Using just rw [Nat.mul_comm] from the start

```
a b c : Nat
⊢ b * c * a = a * (c * b)
```

rw commutes the *first* product it meets — the outermost one, a * (b * c).

```
After conv => (context a b c : Nat unchanged)
| a * (b * c) = a * (c * b)
After lhs — into the left-hand side
| a * (b * c)
After congr — one goal per argument of *
| a
| b * c
After rfl — a is kept, that goal is closed
| b * c
After rw [Nat.mul_comm]
| c * b
```

Leaving conv, Lean reassembles: the left side becomes a * (c * b), and the goal a * (c * b) = a * (c * b) is closed automatically by rfl.

Rewriting under a binder, and conv at h

rw [Nat.zero_add] fails: 0 + x sits under fun x =>

```
error: Tactic `rewrite` failed: Did not
  find an occurrence of the pattern
  0 + ?n
in the target expression
  (fun x ⇒ 0 + x) = fun x ⇒ x
⊢ (fun x ⇒ 0 + x) = fun x ⇒ x
```

```
example :
    (fun x : Nat ⇒ 0 + x) = (fun x ⇒ x) := by
  conv ⇒
    lhs
    intro x
    rw [Nat.zero_add]
```

```
After lhs
| fun x ⇒ 0 + x
After intro x — inside the binder
x : Nat
| 0 + x
After rw [Nat.zero_add]
| x
```

conv at h rewrites a hypothesis (illustration)

```
example (a b : Nat) (h : a * b = 0) :
    b * a = 0 := by
  conv at h ⇒ rw [Nat.mul_comm]
  exact h  -- now h : b * a = 0
```

CHAPTER 11 · §11.2–11.4

Patterns, structure and tactics inside conv

§11.2 — jump straight to the matching term

```
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  conv in _ * c ⇒ rw [Nat.mul_comm]
```

§11.3 — bullets for the goals from congr

```
example (a b c : Nat) :
    (0 + a) * (b * c) = a * (c * b) := by
  conv ⇒
    lhs
    congr
    . rw [Nat.zero_add]
    . rw [Nat.mul_comm]
```

Inside conv	Effect
`pattern p, conv in p`	Go to a subterm matching the pattern (_ allowed)
`arg i`	Enter the i-th explicit argument
`args`	Another name for congr
`enter [1, x, 2, y]`	Repeated arg and intro
`simp`	Simplify the current subterm
`done, trace_state`	Check no goals remain, print the state
`whnf`	Weak head normal form
`tactic ⇒ …, apply`	Return to ordinary tactic mode

CHAPTER 11 · §11.2 AND §11.4

pattern, arg and enter: straight to a subterm

```
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  conv in b * c ⇒
    rw [Nat.mul_comm]

-- is shorthand for
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  conv ⇒
    pattern b * c
    rw [Nat.mul_comm]
```

> *pattern p jumps to the first subterm matching p; _ may serve as a wildcard: conv in _ * c => rw [Nat.mul_comm].*

```
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  conv ⇒
    lhs
    arg 2
    rw [Nat.mul_comm]
```

After lhs

```
| a * (b * c)
```

*After arg 2 — the second explicit argument of **

```
| b * c
```

arg i counts only non-dependent explicit arguments; enter [1, x, 2, y] combines arg 1; intro x; arg 2; intro y.

Three frequent errors in conv

*Same goal a * (b * c) = a * (c * b) with a b c : Nat (illustration) — messages exactly as Lean 4.33 prints them*

Offending code	Lean reports	Meaning and fix
`conv in c * a ⇒ …`	`'pattern' conv tactic failed, pattern was not found` `  c * a`	No subterm matches c * a: check the argument order, or use _ as a wildcard
`conv ⇒ lhs; arg 3`	`invalid `arg` tactic, application has 2 explicit argument(s) but the index is out of bounds`	a * (b * c) has only 2 explicit arguments; counting starts at 1

`conv ⇒ lhs; rw [Nat.add_comm]`	`Tactic `rewrite` failed: Did not find an occurrence of the pattern`
	`  ?n + ?m`
	`in the target expression`
	`  a * (b * c)`
		The lemma does not match the term under the cursor (the line “| a * (b * c)” shows where it is): pick the right lemma or move the cursor

> *A silent trap: rw [Nat.mul_comm] twice in a row at the same position swaps and swaps back — the goal is unchanged. Use trace_state to see where the cursor is.*

tactic =>: back to tactic mode for a side goal

```
example (g : Nat → Nat → Nat)
        (h₁ : ∀ x, x ≠ 0 → g x x = 1)
        (h₂ : x ≠ 0)
        : g x x + x = 1 + x := by
  conv ⇒
    lhs
    arg 1
    rw [h₁]
    . skip
    . tactic ⇒
       exact h₂
```

- rw [h_1] also needs h_1's condition $x \neq 0$ ⇒ it creates an ordinary side goal (⊢) that conv cannot solve itself
- apply h_2 is shorthand for tactic => apply h_2

After lhs

```
| g x x + x
```

After arg 1

```
| g x x
```

After rw [h_1] — two goals

```
| 1
⊢ x ≠ 0
```

Inside tactic => (the second goal)

```
x : Nat
g : Nat → Nat → Nat
h₁ : ∀ (x : Nat), x ≠ 0 → g x x = 1
h₂ : x ≠ 0
⊢ x ≠ 0
```

CHAPTER 11 · COMPARISON

rw, rw with arguments, simp or conv: which when?

Other ways to solve the same chapter examples (illustration)

```
-- give the lemma its arguments
example (a b c : Nat) :
    a * (b * c) = a * (c * b) := by
  rw [Nat.mul_comm b c]

-- simp can go under binders
example :
    (fun x : Nat ⇒ 0 + x) = (fun x ⇒ x) := by
  simp
```

Nat.mul_comm b c : b * c = c * b matches exactly one place, so rw usually no longer hits “the wrong spot”.

Tool	Use when
`rw [h]`	The first match is exactly the spot to change
`rw [h x y]`	Several matches; supply arguments to pin down one
`simp`	You want to simplify everything, even under fun, and do not care about the shape of the result
`conv`	You need exact control of position; a lemma like mul_comm matches too many places; simp goes too far or loops

> *Rule of thumb: try rw or simp first; when it “edits the wrong spot”, switch to conv and use trace_state.*

CHAPTER 11 · CHAPTER SUMMARY

The Conversion Tactic Mode on one page

- The goal is a tree; conv puts a **cursor** on a subtree and edits only there
- The conv state is shown with |, not ⊢; on leaving conv, Lean reassembles and closes the goal by rfl if it can
- arg i counts only explicit arguments; intro x goes under a binder — where rw often fails
- Side goals produced (a lemma's conditions) are solved with tactic => …
- conv at h works on the hypothesis h

Syntax	Meaning
`conv ⇒ …, conv at h ⇒ …`	Enter conv mode
`conv in p ⇒ …`	Go straight to a term matching p
`lhs, rhs, arg i, arg @i`	Down to a child node
`intro x, enter [1, x]`	Into a binder; a whole path
`congr, rfl, skip`	Split by argument; skip
`rw […], simp, whnf`	Transform at the cursor
`tactic ⇒ …`	Solve a side goal
`trace_state, done`	Show the state; check completion

CHAPTER 11 · QUESTIONS TO REFLECT ON

Three questions about conv

1. Why does rw [Nat.zero_add] fail on the goal (fun x => 0 + x) = fun x => x, even though 0 + x is plainly visible in the goal?

2. On leaving a conv block, Lean needs a *proof* that the old goal is equivalent to the new one. What is that proof assembled from?

3. Why does arg count explicit arguments by default, and when do you need arg @i?

Suggested answers to the three questions

1. Inside fun, x is a bound variable, not a closed term that can be pulled out; rw builds its motive by abstracting the occurrences of the pattern, and cannot do so for a term containing a bound variable. intro x inside conv turns x into a local variable, and then 0 + x is an ordinary term.

2. From local equalities (each rw yields an equality for its subtree) and congruence lemmas: if x = y then f x = f y, plus funext when passing a binder. Chained along the cursor's path, they give old goal = new goal.

3. Implicit arguments are types and instances — almost never the place to rewrite. arg @i is for reaching them, e.g. when a dependent type or an instance must be transformed.

PART IV · CHAPTER

12

Axioms and Computation

propext, quotients, choice and computation

SECTIONS

12.1 Historical and Philosophical Context

12.2 Propositional Extensionality

12.3 Function Extensionality

12.4 Quotients

12.5 Choice

12.6 The Law of the Excluded Middle

What is an axiom? A claim accepted without proof

- **In mathematics:** an agreed starting point, like Euclid's "through two points there is exactly one line" — like the rules of chess: not proved, just stipulated
- **In Lean:** axiom name : T adds a constant of type T with **no body**. By Curry–Howard, it is a "proof" the kernel must trust and cannot check
- Every other theorem must be built as a term from definitions, inference rules and the axioms
- Ordinary mathematics in Lean needs only three axioms: propext, Quot.sound, Classical.choice

```
axiom propext {a b : Prop} : (a ↔ b) → a = b
-- only a type, no ":= …"
```

	def	theorem	axiom
Has a body (term)?	Yes	Yes	No
Kernel-checked?	Yes	Yes	Only its type
Reducible?	Yes	Yes, but rarely needed	No — the term gets stuck there
Generates code?	Yes	Erased (Prop)	No code

> *Axioms carry two costs: logical (if wrong, everything collapses) and computational (no reduction, no execution). The next slide covers the logical cost; the rest of the chapter covers the computational one.*

CHAPTER 12 · WHY

Why adding axioms is risky: one line can sink the whole system

Do not do this in a real project (illustration)

```
axiom bad : False

theorem one_eq_two : 1 = 2 :=
  False.elim bad

#print axioms one_eq_two
-- 'one_eq_two' depends on axioms: [bad]

theorem t : 1 = 1 := sorry
#print axioms t
-- 't' depends on axioms: [sorryAx]
```

- Lean does **not check** whether an axiom is consistent — it only checks that the axiom's type is well formed
- Once False is available, every proposition is provable (False.elim): the library loses all meaning
- Contradictions are rarely as blatant as bad: two harmless-looking axioms may contradict each other or type theory
- sorry is an axiom too (sorryAx); Lean warns "declaration uses `sorry`"

> *Good habit: use hypotheses (variable, an argument h : P) instead of axiom; run #print axioms on important theorems and accept only propext, Quot.sound, Classical.choice.*

CHAPTER 12 · INTRODUCTION

A computational core and three extensions

Core: the Calculus of Inductive Constructions with an impredicative, proof-irrelevant Prop. In the "purely computational" fragment, every closed term reduces to canonical form — a closed Nat term reduces to a numeral.

Extension	Form	Kernel reduction	Compiled code
Propositional extensionality `propext`	Axiom	May get stuck	Runs normally
Quotients `Quot` ⇒ funext	Primitive constants + Quot.sound	May get stuck	Runs normally
Choice `Classical.choice`	Axiom	No computational interpretation	Data-producing definitions must be noncomputable

> *Diaconescu's theorem: propext + funext + choice ⇒ the law of the excluded middle. Excluded middle remains compatible with compilation, as long as it is not used to produce data.*

Constructive or classical? Lean supports both

The constructive view

Every theorem should have computational meaning. Historically, mathematics was computational in character.

The classical view

Separate programming from reasoning about programs. Since the 19th century, abstract, non-algorithmic reasoning has been widely accepted.

How Lean reconciles them

The core library is developed constructively; classical reasoning is always available when needed.

- A proof-irrelevant Prop and irreducible theorems are a first step in separating the two concerns — proofs are erased in code extraction
- But equalities in Prop used as type casts can still block reduction; excluded middle blocks CIC reduction but not code generation
- Only the axiom of choice fully erases the line between the data-free and the data-carrying parts

CHAPTER 12 · §12.2–12.3

Propositional and function extensionality

§12.2 — propext and substituting equivalent propositions

```
axiom propext {a b : Prop} :
    (a ↔ b) → a = b

variable (a b c d e : Prop)
theorem thm₂ (p : Prop → Prop)
    (h : a ↔ b) (h₁ : p a) : p b :=
  propext h ▸ h₁
```

§12.3 — funext blocks kernel reduction

```
def f (x : Nat) := x
def g (x : Nat) := 0 + x
theorem f_eq_g : f = g :=
  funext fun x ⟹ (Nat.zero_add x).symm
def val : Nat :=
  Eq.recOn (motive := fun _ _ ⟹ Nat) f_eq_g 0
#reduce val -- does not reduce to 0
#eval val -- 0
```

propext is consistent with the set-theoretic model where each proposition is empty or a singleton

funext is a **theorem** proved from quotients; an extensional rather than intensional view

Together they give an extensional theory of sets: setext, inter_self, inter_comm

A consequence of propext: substitution in any context

```
variable (a b c d e : Prop)

theorem thm₁ (h : a ↔ b) :
    (c ∧ a ∧ d → e) ↔ (c ∧ b ∧ d → e) :=
  propext h ▸ Iff.refl _

theorem thm₂ (p : Prop → Prop) (h : a ↔ b)
    (h₁ : p a) : p b :=
  propext h ▸ h₁
```

1. propext h : a = b — turns an equivalence into an equality of two propositions
2. ▸ uses that equality to replace a by b in the type of h_1 : p a, yielding p b
3. thm_1: Iff.refl _ proves the left side ↔ itself; ▸ replaces a by b in the second copy

Why is thm_2 the essential use? thm_1 can be proved without propext, since ∧ and → both respect ↔. But p in thm_2 is an *arbitrary* function Prop → Prop, with no guarantee it respects ↔ — in fact thm_2 is equivalent to propext itself.

Combined with funext (mem, ∈ defined earlier)

```
def Set (α : Type u) := α → Prop

theorem setext {a b : Set α}
    (h : ∀ x, x ∈ a ↔ x ∈ b) : a = b :=
  funext (fun x ⟹ propext (h x))
```

Intuition: each a : Prop is either the empty set or a singleton {∗}.

Why #reduce val gets stuck while #eval val still gives 0

```
def f (x : Nat) := x
def g (x : Nat) := 0 + x

theorem f_eq_g : f = g :=
  funext fun x ⟹ (Nat.zero_add x).symm

def val : Nat :=
  Eq.recOn (motive := fun _ _ ⟹ Nat) f_eq_g 0

-- does not reduce to 0
#reduce val

-- evaluates to 0
#eval val
```

> *The same holds for propext: tteq : (True ∧ True) = True makes Eq.recOn … tteq 0 stuck. Observational and cubical type theories try to allow this reduction.*

Line	What happens
`f, g`	Two syntactically different functions, equal pointwise; 0 + x does not reduce when x is a variable
`f_eq_g`	Proved by funext — and funext rests on the axiom Quot.sound
`val`	“Casts” 0 : Nat along f = g; the motive is constantly Nat, so the cast is harmless in meaning
`#reduce`	The kernel can unfold Eq.recOn only if the proof reduces to Eq.refl or f, g are definitionally equal; neither holds ⇒ a closed Nat term fails to reach a numeral
`#eval`	The compiler erases proofs and casts ⇒ it runs to 0

CHAPTER 12 · BACKGROUND

Integers modulo 7 from scratch: sorting numbers into boxes

Today is Monday; in 7, 14 or 21 days it is Monday again. If we only care about “which weekday”, we need only the **remainder on division by 7.** Put each natural number into a box by its remainder:

Box 0	Box 1	Box 2	Box 3	Box 4	Box 5	Box 6
0, 7, 14, 21, …	1, 8, 15, 22, …	2, 9, 16, 23, …	3, 10, 17, 24, …	4, 11, 18, 25, …	5, 12, 19, 26, …	6, 13, 20, 27, …

- **Relation:** a ~ b when a % 7 = b % 7 (“same box”)
- ~ is an equivalence relation: a ~ a; a ~ b gives b ~ a; a ~ b and b ~ c give a ~ c — which is why the boxes do not overlap
- **The quotient type** = a type with exactly 7 elements, **one per box**; 3 and 10 become the same element
- A function on boxes must give the same result for every number in a box: “remainder mod 7” works, “is even” does not (0 is even, 7 odd, both in box 0)
- That is exactly the “respects the relation” condition that Quot.lift demands

From boxes to Lean: the type Mod7 built with Quot

mod7Rel is exactly the definition used later in §12.4; Mod7 and its functions are illustrations

```
def mod7Rel (x y : Nat) : Prop :=
  x % 7 = y % 7
def Mod7 : Type := Quot mod7Rel
def Mod7.mk (n : Nat) : Mod7 :=
  Quot.mk mod7Rel n

theorem Mod7.mk_seven :
    Mod7.mk 7 = Mod7.mk 0 := Quot.sound rfl

def Mod7.rem : Mod7 → Nat :=
  Quot.lift (· % 7) (fun _ _ h ⇒ h)

#eval Mod7.rem (Mod7.mk 100)  -- 2
```

Box idea	In Lean
Put the number n into its box	`Quot.mk mod7Rel n`
Same box means equal	`Quot.sound : r a b → mk a = mk b`
A function reads only the box label	`Quot.lift f (proof of respect)`
Proof for every box	`Quot.ind`

- Quot.sound rfl: needs mod7Rel 7 0, i.e. 7 % 7 = 0 % 7; both sides reduce to 0, so rfl suffices
- fun _ _ h => h: the respect condition is a % 7 = b % 7 — which is exactly h
- #print axioms: Mod7.mk_seven uses [Quot.sound]; Mod7.rem uses no axioms, and #eval runs

Quotient types: Quot, Setoid and Quotient

Constant	Meaning
`Quot r`	The quotient type by the relation r (need not be an equivalence)
`Quot.mk r a`	The class containing a
`Quot.ind`	A proof for every element from a proof for Quot.mk r a
`Quot.lift f h`	Lifts an f that respects r; Quot.lift f h (Quot.mk r a) reduces to f a
`Quot.sound`	**Axiom:** r a b → Quot.mk r a = Quot.mk r b

```
def mod7Rel (x y : Nat) : Prop :=
  x % 7 = y % 7
#check (Quot mod7Rel : Type)
```

- **Setoid α:** a relation r with Equivalence r, written ≈
- **Quotient s** with mk, ind, lift, sound, exact: quotient elements correspond exactly to equivalence classes
- Examples: the unordered pair UProd; funext follows from quotients

Quot.lift: lifting to the quotient, and the computation rule

```
def mod7Rel (x y : Nat) : Prop :=
  x % 7 = y % 7

-- the class of numbers equivalent to 4
#check (Quot.mk mod7Rel 4 : Quot mod7Rel)

def f (x : Nat) : Bool :=
  x % 7 = 0

theorem f_respects (a b : Nat) (h : mod7Rel a b) :
    f a = f b := by
  simp [mod7Rel, f] at *
  rw [h]
```

f_respects: two numbers congruent mod 7 give the same value of f — the condition for lifting. The function fun x => x cannot be lifted: 4 and 11 share a class but differ.

```
#check (Quot.lift f f_respects :
    Quot mod7Rel → Bool)

-- the computation principle
example (a : Nat) :
    Quot.lift f f_respects (Quot.mk mod7Rel a)
      = f a :=
  rfl
```

- Quot.lift f h (Quot.mk r a) reducing to f a is a **kernel reduction rule**, so rfl suffices
- Quot, Quot.mk, Quot.ind, Quot.lift alone are rather weak: taking Quot r := α and lift as f itself satisfies them ⇒ they count as part of the logical framework, not axioms
- Only Quot.sound : r a b → Quot.mk r a = Quot.mk r b makes the quotient “real” — and only it shows up in #print axioms

Setoid and Quotient: building the unordered pair type UProd

Standard library (the version in namespace Hidden)

```
class Setoid (α : Sort u) where
  r : α → α → Prop
  iseqv : Equivalence r

def Quotient {α : Sort u} (s : Setoid α) :=
  @Quot α Setoid.r
```

- Setoid = a type with an equivalence relation; a ≈ b is Setoid.r a b, with the instance found by type class resolution
- Quotient.mk, ind, lift, sound are just specializations of Quot; plus Setoid.refl, symm, trans
- Quotient.exact : Quotient.mk s a = Quotient.mk s b → a ≈ b; with sound ⇒ quotient elements match equivalence classes exactly

```
private def eqv (p₁ p₂ : α × α) : Prop :=
  (p₁.1 = p₂.1 ∧ p₁.2 = p₂.2) ∨
  (p₁.1 = p₂.2 ∧ p₁.2 = p₂.1)

infix:50 " ~ " ⇒ eqv

instance uprodSetoid (α : Type u) :
    Setoid (α × α) where
  r     := eqv
  iseqv := is_equivalence

def UProd (α : Type u) : Type u :=
  Quotient (uprodSetoid α)
```

eqv: two pairs are equal in the same order or in swapped order. is_equivalence bundles the three proofs eqv.refl, eqv.symm, eqv.trans (dependent pattern matching, then simp_all).

Working with UProd: Quot.sound and Quot.liftOn

```
namespace UProd

def mk {α : Type} (a₁ a₂ : α) : UProd α :=
  Quotient.mk' (a₁, a₂)

notation "{ " a₁ ", " a₂ " }" ⇒ mk a₁ a₂

theorem mk_eq_mk (a₁ a₂ : α) :
    {a₁, a₂} = {a₂, a₁} :=
  Quot.sound (Or.inr ⟨rfl, rfl⟩)
```

- Quot.sound needs $(a_1, a_2) \sim (a_2, a_1)$: the right disjunct Or.inr of eqv asks for $a_1 = a_1$ and $a_2 = a_2$ — rfl twice
- The notation $\{a_1, a_2\}$ is for illustration only: it shadows the brace syntax of records and sets

```
private def mem_fn (a : α) : α × α → Prop
  | (a₁, a₂) ⇒ a = a₁ ∨ a = a₂

def mem (a : α) (u : UProd α) : Prop :=
  Quot.liftOn u (fun p ⇒ mem_fn a p)
    (fun p₁ p₂ e ⇒ mem_respects a e)

theorem mem_mk_left (a b : α) : a ∈ {a, b} :=
  Or.inl rfl
```

- mem_respects : $p_1 \sim p_2 \to$ mem_fn a p_1 = mem_fn a p_2 (uses propext via the lemma mem_swap) — the liftOn-plus-respect-proof idiom used throughout the library
- mem_mk_left: liftOn on Quot.mk reduces to a = a ∨ a = b, so Or.inl rfl suffices

funext is a theorem: a proof sketch from quotients

```
variable {α : Sort u} {β : α → Sort v}
def extfun (α : Sort u) (β : α → Sort v) := Quot (fun (f g : (x : α) → β x) ⟹ ∀ x, f x = g x)

def extfun_app {α β} (f : extfun α β) (x : α) : β x :=
  Quot.lift (· x) (by intros; simp [*]) f

example (f₁ f₂ : (x : α) → β x) (h : ∀ x, f₁ x = f₂ x) :=
  calc f₁
    _ = extfun_app (.mk _ f₁) := rfl
    _ = extfun_app (.mk _ f₂) := by rw [Quot.sound]; trivial
    _ = f₂ := rfl
```

Construction: extfun = functions “modulo” pointwise equality; (· x) respects the relation, so Quot.lift gives extfun_app.

Steps 1 and 3 — rfl: extfun_app (.mk _ f) reduces to fun x => f x, i.e. f itself.

Step 2: Quot.sound h gives .mk _ f_1 = .mk _ f_2 — the only place an axiom is used.

Choice in plain words: “knowing one exists” is not “pointing one out”

Shoes and socks (Russell). Infinitely many pairs of shoes: choosing one from each pair is easy — “take the left one”. Infinitely many pairs of identical socks: no rule tells the two apart. The axiom of choice says a choice *still exists*, but not which one.

Everyday life. “Someone in this room was surely born on a Monday” (knowing one exists) differs from “That person is An” (pointing one out). No algorithm turns the first statement into the second.

In Lean	Meaning
`Nonempty α : Prop`	“Knowing one exists” — the evidence is erased at runtime
`Inhabited α`	“A specific element at hand” — data, not in Prop
`Classical.choice`	Nonempty α → α: turns “one exists” into “here is one”

- No term can implement choice, since the needed information has been erased — so it must be an axiom
- Consequence: every definition that *produces data* via choice cannot run and must be marked noncomputable
- Using choice inside proofs is harmless for code generation

The axiom of choice and the excluded middle

```
class inductive Nonempty (α : Sort u) : Prop where
  | intro (val : α) : Nonempty α

axiom choice {α : Sort u} : Nonempty α → α

noncomputable def indefiniteDescription {α : Sort u}
    (p : α → Prop) (h : ∃ x, p x) : {x // p x} :=
  choice ◁ let ⟨x, px⟩ := h; ⟨⟨x, px⟩⟩
```

Nonempty ≈ ∃ x : α, True; choice produces an element without saying how to find it ⇒ computation is blocked. Consequences: choose, choose_spec, epsilon (Hilbert).

Sketch: Diaconescu → em

1. Let U x := x = True ∨ p and V x := x = False ∨ p
2. Use choice to pick u satisfying U, v satisfying V
3. Case analysis: u ≠ v ∨ p
4. If p holds then U = V (funext, propext), so u = v
5. Combine: p ∨ ¬p — that is, Classical.em

Consequences: a = True ∨ a = False; every proposition is (classically) decidable.

From choice to choose, choose_spec and epsilon

```
example (α : Type u) :
    Nonempty α ↔ ∃ x : α, True :=
  Iff.intro (fun ⟨a⟩ ⟹ ⟨a, trivial⟩)
    (fun ⟨a, h⟩ ⟹ ⟨a⟩)

variable {α : Sort u} {p : α → Prop}

noncomputable def choose (h : ∃ x, p x) : α :=
  (indefiniteDescription p h).val

theorem choose_spec (h : ∃ x, p x) :
    p (choose h) :=
  (indefiniteDescription p h).property
```

Nonempty α : Prop eliminates only into Prop, so no element can be extracted; choice breaks that barrier and loses computational content.

Signatures in the Classical namespace

```
strongIndefiniteDescription {α : Sort u}
  (p : α → Prop) (h : Nonempty α) :
  {x // (∃ (y : α), p y) → p x}

epsilon {α : Sort u} [h : Nonempty α]
  (p : α → Prop) : α

epsilon_spec {α : Sort u} {p : α → Prop}
  (hex : ∃ (y : α), p y) :
  p (@epsilon _ hex.nonempty p)
```

- choose: the data part, choose_spec: the proof part of indefiniteDescription
- epsilon (Hilbert's ε): picks an element satisfying p if one exists, otherwise an arbitrary element

Diaconescu (1): build U, V, then choose u, v

```
open Classical
theorem em (p : Prop) : p ∨ ¬p := by
  let U (x : Prop) : Prop := x = True ∨ p
  let V (x : Prop) : Prop := x = False ∨ p
  have exU : ∃ x, U x := ⟨True, Or.inl rfl⟩
  have exV : ∃ x, V x := ⟨False, Or.inl rfl⟩
  let u : Prop := choose exU
  let v : Prop := choose exV
  have u_def : U u := choose_spec exU
  have v_def : V v := choose_spec exV
```

If p holds: every x : Prop belongs to both U and V

If p fails: U holds only of True, V only of False

After the first four let/have lines

```
p : Prop
U : Prop → Prop := fun x ⇒ x = True ∨ p
V : Prop → Prop := fun x ⇒ x = False ∨ p
exU : ∃ x, U x
exV : ∃ x, V x
⊢ p ∨ ¬p
```

After choosing — the context also has

```
u : Prop := choose exU
v : Prop := choose exV
u_def : U u
v_def : V v
⊢ p ∨ ¬p
```

Diaconescu (2): u ≠ v ∨ p through four cases

```
have not_uv_or_p : u ≠ v ∨ p := by
  match u_def, v_def with
  | Or.inr h, _ ⇒ exact Or.inr h
  | _, Or.inr h ⇒ exact Or.inr h
  | Or.inl hut, Or.inl hvf ⇒
    apply Or.inl
    simp [hvf, hut, true_ne_false]
```

Third branch, after apply Or.inl (context excerpt)

```
hut : u = True
hvf : v = False
⊢ u ≠ v
```

u_def : U u	v_def : V v	Conclusion
Or.inr h, h : p	anything	Or.inr h — the p side
Or.inl _	Or.inr h, h : p	Or.inr h — the p side
Or.inl hut : u = True	Or.inl hvf : v = False	Or.inl: u ≠ v

- Four combinations but only three match branches: as soon as either is Or.inr, we already have p
- Last branch: simp replaces u by True, v by False, and the goal becomes True ≠ False — exactly true_ne_false
- No need yet to know whether p holds: the result is always u ≠ v or p

Diaconescu (3): p implies u = v, then conclude

```
have p_implies_uv : p → u = v :=
  fun hp ⇒
  have hpred : U = V :=
    funext fun x ⇒
      have hl : (x = True ∨ p) → (x = False ∨ p) :=
        fun _ ⇒ Or.inr hp
      have hr : (x = False ∨ p) → (x = True ∨ p) :=
        fun _ ⇒ Or.inr hp
      show (x = True ∨ p) = (x = False ∨ p) from
        propext (Iff.intro hl hr)
  have h₀ : ∀ exU exV,
      @choose _ U exU = @choose _ V exV := by
    rw [hpred]; intros; rfl
  show u = v from h₀ _ _
```

```
  match not_uv_or_p with
  | Or.inl hne ⇒
    exact Or.inr (mt p_implies_uv hne)
  | Or.inr h   ⇒
    exact Or.inl h
```

1. Assume hp : p. For every x, both x = True ∨ p and x = False ∨ p hold by Or.inr hp, so they are equivalent; propext turns that into an equality
2. funext combines the pointwise equalities into hpred : U = V
3. h_0: after rw [hpred] both sides are choose over V; the two existence proofs are equal by proof irrelevance ⇒ rfl
4. Conclusion: from u ≠ v, mt (modus tollens) gives ¬p; from p, Or.inl

noncomputable in practice: the left inverse linv

```
open Classical
noncomputable def linv [Inhabited α] (f : α → β) :
    β → α :=
  fun b : β ⇒
    if ex : (∃ a : α, f a = b) then choose ex
    else default

theorem linv_comp_self {f : α → β} [Inhabited α]
    (inj : ∀ {a b}, f a = f b → a = b)
    : linv f ∘ f = id :=
  funext fun a ⇒
    have ex  : ∃ a₁ : α, f a₁ = f a := ⟨a, rfl⟩
    have feq : f (choose ex) = f a  := choose_spec ex
    calc linv f (f a)
      _ = choose ex := rfl
      _ = a         := inj feq
```

- **choice is used twice:** (1) the scoped instance propDecidable, enabled by open Classical, makes if ex : (∃ a, f a = b) meaningful — ∃ over an arbitrary α is not algorithmically decidable; (2) choose ex extracts a
- Depending on Classical.choice ⇒ Lean cannot generate code, so noncomputable is mandatory; drop the keyword and Lean reports a compilation error
- **Proof:** funext reduces to each a; ex has witness a; feq comes from choose_spec; calc: linv f (f a) = choose ex by definition, then = a by inj feq
- Classically, linv is a function. Constructively it is rejected, as there is no general implementation

Depending on Classical.choice ⇒ noncomputable is required

Error 1 — extracting an element with choose (illustration)

```
open Classical in
def pick (h : ∃ n : Nat, n > 5) : Nat :=
  choose h
```

```
error: failed to compile definition,
consider marking it as 'noncomputable'
because it depends on 'choose',
which is 'noncomputable'
```

choose is defined via Classical.choice, so it is itself noncomputable; the error spreads to every definition that uses it to produce data.

Error 2 — if on an undecidable proposition (illustration)

```
def isZeroFn (f : Nat → Nat) : Bool :=
  if ∀ n, f n = 0 then true else false
-- failed to synthesize instance of type class
--   Decidable (∀ (n : Nat), f n = 0)
-- with open Classical in added above:
-- … because it depends on 'propDecidable'
```

Situation	Remedy
Used only in proofs	Write noncomputable def …
Must be executable	Find a real algorithm; require [DecidablePred p] instead of open Classical
Only a proposition is needed	Return Prop instead of Bool

Consequences of em, and checking axioms with #print axioms

```
open Classical
theorem propComplete (a : Prop) : a = True ∨ a = False :=
  match em a with
  | Or.inl ha ⇒
    Or.inl (propext (Iff.intro (fun _ ⇒ True.intro) (fun _ ⇒ ha)))
  | Or.inr hn ⇒
    Or.inr (propext (Iff.intro (fun h ⇒ hn h) (fun h ⇒ False.elim h)))
```

#print axioms (illustration)

```
#print axioms thm₂
-- 'thm₂' depends on axioms: [propext]
#print axioms Classical.em
-- 'Classical.em' depends on axioms:
--  [propext, Classical.choice, Quot.sound]
```

- propComplete: em + propext ⇒ every proposition equals True or equals False
- Adding choice: every proposition is decidable (propDecidable); Decidable p is equivalent to Sum p (¬p), which eliminates into any type
- Quot.sound appears via funext; a theorem using only the computation rule of Quot.lift (rfl) depends on no axioms

CHAPTER 12 · CHAPTER SUMMARY

Axioms and Computation on one page

- Axiom = a constant with no body: the kernel trusts it, it does not reduce and cannot run
- Three standard axioms: propext, Quot.sound, Classical.choice; funext and em are **theorems** derived from them
- A quotient type merges equivalent elements into one; functions on a quotient must respect the relation
- Axioms block kernel reduction (#reduce) but not compiled code (#eval) — except choice
- Data produced via choice ⇒ noncomputable; using it in proofs is harmless

Name	Role
`propext`	(a ↔ b) → a = b
`funext`	Pointwise equal ⇒ equal
`Quot.mk, Quot.lift`	Into the quotient; functions on it
`Quot.sound, Quot.ind`	Same class ⇒ equal; induction
`Setoid, Quotient`	Quotient by an equivalence relation
`Classical.choice, choose`	Extract an element from an existence proof
`Classical.em`	p ∨ ¬p (Diaconescu)
`noncomputable`	Marks “no code generated”
`#print axioms t`	Lists the axioms t uses

CHAPTER 12 · QUESTIONS TO REFLECT ON

Four questions about axioms and computation

1. Why does Quot.lift demand a proof that f respects the relation? What would go wrong if fun n => n could be lifted to Mod7?

2. Why does #reduce val get stuck on funext, while #eval val still prints 0?

3. Does using Classical.em in a proof make any definition noncomputable? When does it?

4. Why can you not write a function Nonempty α → α without an axiom, even though Inhabited α → α is easy?

Suggested answers to the four questions

1. Quot.sound gives Mod7.mk 0 = Mod7.mk 7. Applying the lifted function g to both sides gives g (mk 0) = g (mk 7), which by the computation rule is 0 = 7 — a contradiction. The respect condition is the price of making lift compatible with sound.

2. The kernel only reduces by definitions; Eq.recOn unfolds only when the proof reduces to Eq.refl, and a proof resting on the axiom Quot.sound has no body to reduce. The compiler erases all proofs, so it never needs an axiom's content.

3. No, if em appears only inside proofs (in Prop, erased). Yes, when a data definition branches on an undecidable proposition — that requires propDecidable, which rests on choice.

4. Nonempty α is a Prop: it eliminates only into Prop, so you cannot "open the box" and take the element out to build data. Inhabited α is data that carries a default. The only bridge from Prop to data here is the axiom of choice.

SUMMARY

A map of the ground covered in 12 chapters

I · FOUNDATIONS (1–4)

Propositions are types, proofs are terms. ∀ is a dependent function type, ∃ is the Prop version of Σ; equality, rfl, ▸ and calc.

II · PRACTICE (5–6)

Tactics turn proofs into interactive scripts: intro, apply, cases, rw, simp. Namespaces, attributes, notation, display options.

III · DATA AND RECURSION (7–9)

Every concrete type is an inductive type with a recursor. The equation compiler, structural and well-founded recursion, functional induction; structures and inheritance.

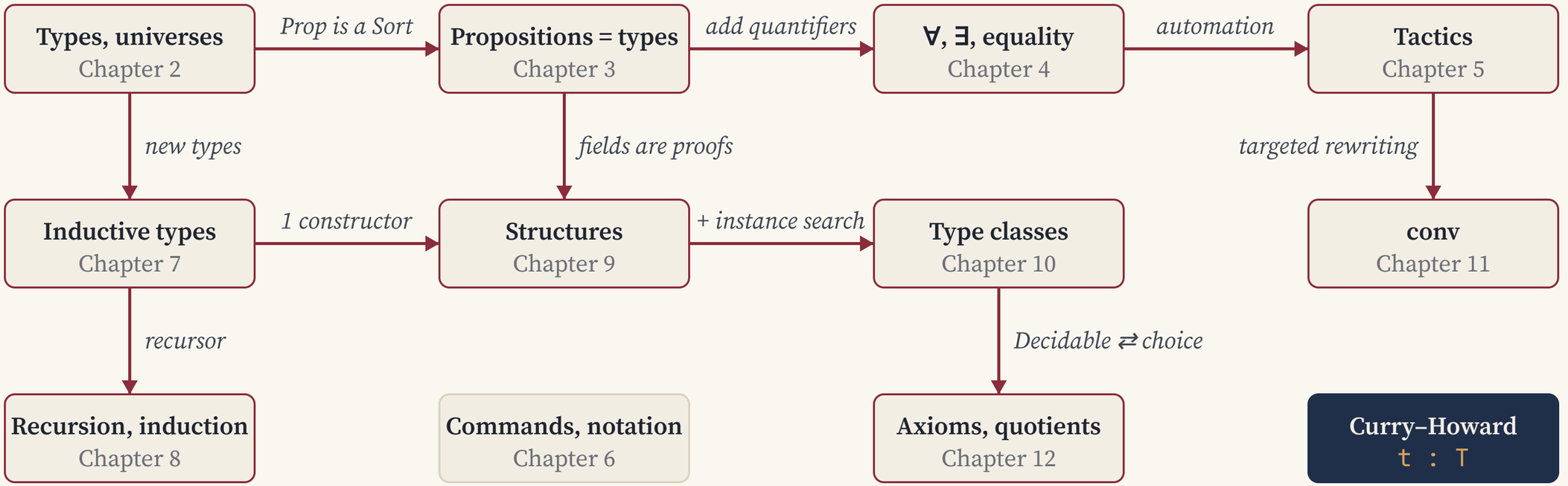
IV · ADVANCED (10–12)

Type classes for ad-hoc polymorphism and coercions; conv for targeted rewriting; propext, Quot, choice and their computational cost.

One sentence to remember: in Lean, writing programs and proving theorems are the same activity — constructing a term of the right type.

THE ROAD TO EXPERTISE · CONCEPT MAP

How the chapters connect



> *The common thread is the dark box: everything is a term with a type. Each chapter adds either a new way to build types (7, 9, 12) or a handier way to build terms (5, 8, 10, 11); chapter 6 (light box) is a toolkit used everywhere.*

THE ROAD TO EXPERTISE · ROADMAP

Three stages: newcomer → practitioner → expert

STAGE 1 · NEWCOMER

- Install Lean and VS Code; read the infoview: context, ⊢, error messages
- Finish the Natural Number Game
- Re-prove the examples of chapters 3–4 both ways: term and tactic
- Become fluent with intro, exact, apply, rw, simp, cases, induction
- **Milestone:** prove length (xs ++ ys) = length xs + length ys on your own

STAGE 2 · PRACTITIONER

- Define your own inductive types and recursive functions, then prove their properties
- Work through Functional Programming in Lean or Mathematics in Lean
- Use Mathlib: find lemmas with exact?, Loogle, the API docs
- Write structures, classes and instances for your own problem domain
- **Milestone:** specify and verify a small algorithm, e.g. insertion sort

STAGE 3 · EXPERT

- Understand elaboration, unification, universes and well-founded recursion well enough to read hard errors
- Write your own macros and tactics (Metaprogramming in Lean 4)
- Design class hierarchies that avoid instance conflicts
- Contribute to Mathlib or the team's library; answer questions on Zulip
- **Milestone:** a pull request accepted into Mathlib

> *Each stage is measured by what you can do, not by time: move on only after reaching the previous stage's milestone.*

What to read, play and where to ask next

Resource	For	Address
Natural Number Game	Stage 1: practising tactics through a game	`https://adam.math.hhu.de/#/g/hhu-adam/NNG4`
Functional Programming in Lean	Lean as a programming language	`https://lean-lang.org/functional_programming_in_lean/`
Mathematics in Lean (J. Avigad, P. Massot)	Mathematics with Mathlib	`https://leanprover-community.github.io/mathematics_in_lean/`
Metaprogramming in Lean 4	Stage 3: macros, tactics, elaborators	`https://github.com/arthurpaulino/lean4-metaprogramming-book`
Mathlib and its API docs	Source code; looking up definitions and theorems	`https://github.com/leanprover-community/mathlib4` `https://leanprover-community.github.io/mathlib4_docs/`
Lean reference manual	The full language specification	`https://lean-lang.org/doc/reference/latest/`
Lean Zulip	Q&A with the community	`https://leanprover.zulipchat.com/`
The community's **learning index**	All of the above and more	`https://leanprover-community.github.io/learn.html`

Finding lemmas and reading Mathlib source

Built-in search commands in Lean (run with Lean 4.33, illustration)

```
example (a b : Nat) : a + b = b + a := by
  exact?
-- Try this: exact Nat.add_comm a b

example (a b : Nat) (h : a ≤ b) :
    a < b + 1 := by
  apply?
-- Try this: exact Nat.lt_succ_of_le h

example (xs : List Nat) :
    (xs ++ []).length = xs.length := by
  simp?
-- Try this: simp only [List.append_nil]
```

- **exact?, apply?, rw?, simp?** suggest a proof step via “Try this:”; paste the result in and delete the search command, since searching is slow
- **Guess names** by convention: the name describes the statement — add_comm, mul_one, zero_add, lt_succ_of_le; check with #check Nat.add_comm
- **Loogle** (https://loogle.lean-lang.org/): search by constant (Real.sin), by pattern (_ * (_ ^ _)), by name ("differ") or by conclusion (|- …)
- **API docs** mathlib4_docs: search by name; every entry links to its source
- **Reading source:** “Go to Definition” in VS Code; read the docstring /-- … -/, the variables at the top of the file, then the statement; use #print, #check @name, #synth to see definitions, implicit arguments and instances

THE ROAD TO EXPERTISE · HANDS-ON PROJECTS

Project ideas for a software development team

Project	Specification to prove	Chapters used
Small pure functions: max, clamp, safe division	Precondition is an argument h : b ≠ 0; postcondition is a theorem about the result	3–5, 8
Insertion sort	The output is sorted and is a permutation of the input (same length, same elements)	7–8: induction on List
Binary search tree	insert preserves the invariant “left < root < right”; every inserted element is found	7–9: inductive, structure
Encoder / decoder	Round trip: decode (encode x) = some x	8, 10: ToString, class
State machine of a business process	Every transition preserves the invariant; forbidden states are unreachable	7–8: inductive, induction

Start small: write the specification first, the proof later; one function, one theorem at a time.

Work in pairs: one person writes the specification, the other proves it — a wrong specification shows up fast.

Stay disciplined: never merge code containing sorry; check #print axioms on main theorems.

A starting template: preconditions and postconditions in Lean

Postcondition = a theorem about the result (illustration)

```
def maxOf (a b : Nat) : Nat :=
  if a ≤ b then b else a

-- the result is at least each argument
theorem maxOf_ge (a b : Nat) :
    a ≤ maxOf a b ∧ b ≤ maxOf a b := by
  unfold maxOf
  split <;> omega

-- and it is one of the two arguments
theorem maxOf_eq (a b : Nat) :
    maxOf a b = a ∨ maxOf a b = b := by
  unfold maxOf
  split <;> simp
```

Precondition = a proof argument (illustration)

```
def safeHead (xs : List Nat)
    (h : xs ≠ []) : Nat :=
  match xs, h with
  | x :: _, _ ⇒ x

#eval safeHead [3, 1, 2] (by decide)  -- 3
example := safeHead [] (by decide)
-- Tactic `decide` proved that the
-- proposition [] ≠ [] is false
```

- No [] branch needed: Lean sees h : [] ≠ [] is absurd and drops that case itself
- A bad call is rejected **at compile time**, not at runtime; h is erased in generated code
- split separates the two branches of the if; omega solves linear inequalities over Nat

SUMMARY

References

[1] J. Avigad, L. de Moura, S. Kong, S. Ullrich, with contributions from the Lean Community. *Theorem Proving in Lean 4* (edition for Lean 4.33.0). Apache License 2.0. https://lean-lang.org/theorem_proving_in_lean4/

[2] Source code of [1] (Apache License 2.0). https://github.com/leanprover/theorem_proving_in_lean4

[3] *Functional Programming in Lean* — Lean as a programming language. https://lean-lang.org/functional_programming_in_lean/

[4] J. Avigad, P. Massot. *Mathematics in Lean*. https://leanprover-community.github.io/mathematics_in_lean/

[5] *Metaprogramming in Lean 4*. https://github.com/arthurpaulino/lean4-metaprogramming-book

[6] *Lean reference manual*. https://lean-lang.org/doc/reference/latest/ — installation guide: https://lean-lang.org/install/

[7] *Mathlib* — source code: https://github.com/leanprover-community/mathlib4

[8] Mathlib API documentation. https://leanprover-community.github.io/mathlib4_docs/

[9] *Natural Number Game*. https://adam.math.hhu.de/#/g/hhu-adam/NNG4

[10] Loogle — search lemmas by name or pattern. https://loogle.lean-lang.org/

[11] Lean Zulip — community Q&A. https://leanprover.zulipchat.com/

[12] The community's index of Lean learning resources. https://leanprover-community.github.io/learn.html

Thank you — questions and discussion welcome.

SUMMARY

References (continued)

[13] A. Edelman. “The Mathematics of the Pentium Division Bug.” *SIAM Review* 39(1):54–67, 1997.

[14] X. Leroy. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52(7):107–115, 2009.

[15] X. Yang, Y. Chen, E. Eide, J. Regehr. “Finding and Understanding Bugs in C Compilers.” *PLDI* 2011.

[16] G. Gonthier. “Formal Proof — The Four-Color Theorem.” *Notices of the AMS* 55(11):1382–1393, 2008.

[17] T. Hales et al. “A Formal Proof of the Kepler Conjecture.” *Forum of Mathematics, Pi* 5:e2, 2017.

[18] Liquid Tensor Experiment, Lean community (completed 2022). https://github.com/leanprover-community/lean-liquid

[19] L. de Moura, S. Ullrich. “The Lean 4 Theorem Prover and Programming Language.” *CADE-28*, LNCS 12699, 2021.

[20] W. A. Howard. “The Formulae-as-Types Notion of Construction.” In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980.

[21] R. Diaconescu. “Axiom of Choice and Complementation.” *Proceedings of the AMS* 51(1):176–178, 1975.

[22] J.-Y. Girard. *Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur.* Thèse d’État, Université Paris VII, 1972.

[23] Apache License, Version 2.0 (license of [1]). http://www.apache.org/licenses/LICENSE-2.0

> [1] Primary source — Jeremy Avigad, Leonardo de Moura, Soonho Kong and Sebastian Ullrich, with contributions from the Lean Community. *Theorem Proving in Lean 4*. Apache License 2.0.
> `https://lean-lang.org/theorem_proving_in_lean4/ · https://github.com/leanprover/theorem_proving_in_lean4`

APPENDIX · LICENSE OF [1]

Apache License, Version 2.0 — full text (1/3)

Apache License Version 2.0, January 2004
http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is

Apache License, Version 2.0 — full text (2/3)

filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

(a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other

Apache License, Version 2.0 — full text (3/3)

Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.